Integer multiplication below $n \log n$

An illustrated reading edition

Original manuscript: OpenAI

Added explanations and figures prepared with OpenAI Codex

Second illustrated edition, October 12, 2026

Original manuscript dated September 23, 2026

Abstract

A guided explanation of the original argument, with 24 explanatory figures, worked arithmetic examples, a map of the recursive construction, and explicit distinctions between source theorem claims, illustrative calculations, and conditional proof interfaces. The exposition begins with digit convolution and builds toward frame geometry, tape recurrences, transforms, and exact recovery. It is not a new proof or an official revision of the source paper.

Full annotated edition. Added explanatory chapters precede the relevant original sections. The original proofs and numerical claims are reproduced from the frozen source, with their original attribution. Their reproduction is not an independent endorsement of the asserted theorem.

Contents

Reader guide: Start with the problem, not the symbols **4**

Reader guide: The question, the claim, and how to read it **5**

Reader guide: See exactly which digit products get added **7**

Reader guide: From long multiplication to convolution **8**

Original abstract **10**

1 Introduction **10**

1.1 The two tape operations . . . 10
1.2 Linear networks and the recursive saving . . . 11
1.3 From transforms to the exact product . . . 12
1.4 Other arithmetic operations . . . 12
1.5 Historical context and inherited methods . . . 13
1.6 The finite-network ingredients . . . 14
1.7 Lower-bound scope and matrix transposition . . . 14
1.8 Companion application to exact Fourier transforms . . . 15

Reader guide: A matrix transpose is a physical rearrangement **15**

Reader guide: Why moving data is part of the mathematics **17**

2 The machine model and elementary streams **18**
2.1 Array order and elementary scans . . . 18

Reader guide: Vectors, rank, and a frame you can draw **21**

Reader guide: Borrowed workspace and changing representations **22**

Reader guide: How unwanted sources cancel out **25**

3 Two finite networks with a rank saving **26**
3.1 Scalar operations and restored auxiliary values . . . 26
3.2 From scalar values to arrays . . . 28
3.3 Subspaces attached to the gates . . . 28
3.4 The rational matrix interface . . . 32
3.5 The binary phase interface . . . 33
3.6 Explicit rational bounds for the two recurrences . . . 34

Reader guide: Why more calls can still mean less work **35**

Reader guide: How a finite rank saving becomes an exponent **36**

4 Faster interchange of address chunks **37**
4.1 A matrix decomposition adapted to tape order . . . 37
4.2 Transferring a finite circuit to address permutations . . . 38
4.3 Removing width and row restrictions . . . 41

Reader guide: What a transform actually changes **43**

Reader guide: The smallest transform step, applied on many axes **45**

5 Fast simultaneous synthetic butterfly layers **46**
5.1 Phase frames for a simultaneous butterfly layer . . . 47
5.2 Packed changes of selected address bits . . . 48
5.3 Batching a common layer on a fixed number of tapes . . . 52
5.4 An explicit guard-width bound . . . 55

Reader guide: Making roots of unity into signed shifts **59**

6 Synthetic transforms and their tape layout **60**
6.1 Roots, butterflies, and stored frequency order . . . 61
6.2 A layout on which the parallel layers apply . . . 62

7 Exact normalized polynomial products **66**
7.1 The normalized convolution interface . . . 68

Reader guide: Changing the grid without sorting everything **70**

8 Resampling with the permutations left in the transform **71**
8.1 The one-dimensional interface . . . 72
8.2 Growing dimension and the padded array . . . 75
8.3 The middle transform and the final permutation . . . 76

Reader guide: Small error must stay small after rescaling **77**

Reader guide: Why improving one part may change nothing **78**

Reader guide: Putting exactness and the running time together **79**

9 Exact integer multiplication and its cost **81**
9.1 A fixed rational choice . . . 81
9.2 Input and transform sizes . . . 81
9.2.1 The padded box and working precision . . . 82
9.2.2 Prime source lengths . . . 82
9.2.3 The radix-digit polynomials . . . 83
9.3 An explicit cyclic-convolution layout . . . 84
9.4 Complex convolution by a coefficient twist . . . 85
9.4.1 Computed complex convolution . . . 86
9.5 Chirp and source-transform procedures . . . 87
9.5.1 The source transform . . . 87
9.6 The multiplication algorithm . . . 87
9.7 Precision and exact recovery . . . 88
9.7.1 Scales and disk margins . . . 88
9.7.2 The final integer coefficients . . . 90
9.8 The complete time bound . . . 92

Reader guide: What a successful check actually tells us **94**

Reader guide: What follows, and what still needs checking **96**

10 Exact division and integer square root **97**

11 A consequence for matrix transposition **99**

A Packed additions on grouped rectangles **100**

Reader guide: Notation and source map **102**

Start with the problem, not the symbols

Added explanation First pass: ordinary arithmetic is enough for this opening.

You have two long strings of binary digits. You want their exact product. The research question is how the required work grows when the strings become longer. It is not a question about whether multiplication is possible, and it is not a promise that a new program will beat your calculator tomorrow.

Think of three bills. There is a bill for combining numbers, a bill for moving the right pieces next to one another, and a bill for making sure the final answer is exact. This paper proposes a way to reduce the first two bills together while still paying the third. The illustrations below show actual data paths: the left picture changes where records are stored; the right picture connects pairs that are combined at successive transform levels.

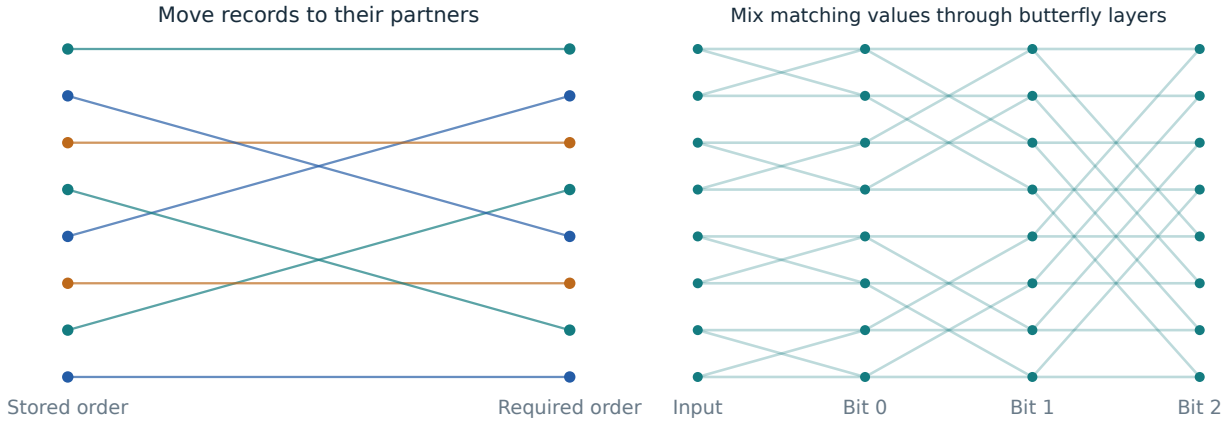


Figure E1. Two distinct jobs inside fast multiplication. Left: records must be brought into a useful order. Right: an eight-entry butterfly network combines entries differing in one address bit per level. These are small structural illustrations, not the paper's enormous circuit.

What is a bit? A bit is a zero or a one. In decimal, the places are ones, tens, hundreds. In binary, they are ones, twos, fours, eights. Thus binary 1011 means $1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 11$. This input has four bits. The paper writes n for the bit length, not the number's value. Leading zeroes are allowed: 0011 is also a four-bit input, even though its value is three.

What does a logarithm count? For powers of two, $\log_2 n$ counts how many doublings take one to n. Since $1, 2, 4, 8, 16$ takes four doublings, $\log_2 16 = 4$. The distinction between n and $\log_2 n$ is crucial. When a bit string doubles in length, n doubles but $\log_2 n$ increases by just one.

What does a running-time formula mean? Schoolbook multiplication pairs each of roughly n digits with roughly n others, suggesting n^2 elementary digit products. More sophisticated algorithms avoid doing all those pairs separately. A bound such as $O(n \log n)$ says that, beyond a threshold, the work is at most a fixed constant times $n \log n$. It does not specify the constant, the threshold, or the number of seconds on a particular computer.

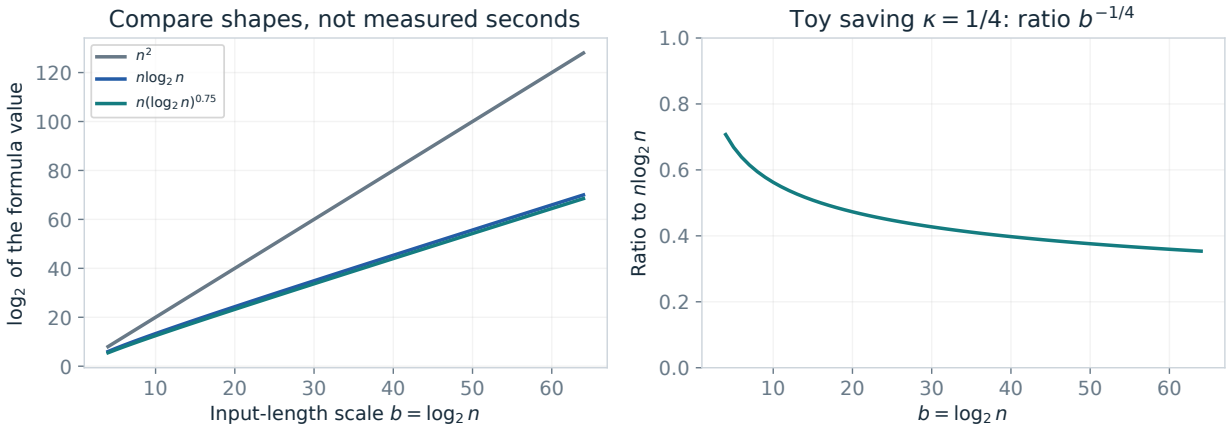


Figure E2. Growth rates need the right scale. Left: the vertical axis is the logarithm of a formula's value, so exponentially different values fit on one plot. Right: dividing the toy improved formula by $n \log_2 n$ exposes its smaller growth. The saving $1/4$ is deliberately large enough to see; the original paper uses 2^{-182}. Neither panel is a measurement of a multiplication program.

Two ways to read this edition. For a first pass, follow the worked examples, pictures, and "What to carry forward" paragraphs. They build the vocabulary before the technical argument uses it. For a second pass, read the equations and then the corresponding original proofs in the expanded paper. Understanding why the pieces fit is a different goal from checking every theorem. The guide supports the first goal and provides a route toward the second; it does not turn the full proof into elementary arithmetic.

A quick check. If an input grows from 16 bits to 32 bits, n doubles and $\log_2 n$ grows from four to five. The value of the input integer can be vastly larger than 32. Keeping these three quantities separate prevents many later confusions.

What to carry forward. The paper tries to save a little work at an enormous number of scales. To understand it, follow both the arithmetic and the movement of data.

The question, the claim, and how to read it

Added explanation Start here; no Fourier analysis is assumed.

Multiplication looks finished when we learn it at school. But the number of elementary steps needed to multiply two extremely long binary strings is a different question. The input length is the number of bits, not the numerical value of the input. A number with a million bits is one input of length a million; the fact that its value may be near $2^{1,000,000}$ does not change that length.

The manuscript studied here is OpenAI's *Integer multiplication below $n \log n$*, dated September 23, 2026, at source commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`. It states a worst-case bound

$$T(n) = O(n(\lg n)^{1-\kappa}), \qquad \kappa = 2^{-182}.$$

Here $\lg n = \max\{\lceil \log_2 n \rceil, 1\}$. The machine must output the exact product for every input, including inputs with leading zeroes. The machine model uses a fixed finite alphabet and a fixed finite number of one-dimensional tapes. Reading, writing, and moving data all cost time.

What this edition does and does not establish. The original text states its result as a theorem. This edition explains that argument; it does not independently establish its validity. The community repository treats its improvements as conditional on the original framework and retained implementation and analytic interfaces. That distinction matters: checking an exact finite network or a list of rational inequalities does not by itself prove that a single tape machine implements the complete algorithm within the asserted bound for all input sizes. Original theorem statements are retained as source text in the full edition, not recast as new findings by this guide. This is an independently prepared explanatory edition, not an official OpenAI revision or endorsement.

Why a tiny positive exponent matters. Ignoring constants only for this comparison, divide the proposed bound by $n \log_2 n$. The quotient is $(\log_2 n)^{-\kappa}$, which tends to zero for every fixed $\kappa > 0$. This is an asymptotic distinction, however tiny κ is. It is not a prediction of a useful speedup on a computer. Even reducing this idealized ratio to $1/2$ requires

$$(\log_2 n)^{\kappa} = 2, \qquad \log_2 n = 2^{1/\kappa}, \qquad n = 2^{\,2^{1/\kappa}}.$$

With $\kappa = 2^{-182}$ this is an extraordinary scale. Unknown constants, thresholds, and the actual implementation make a practical comparison still more demanding. A larger κ means a better exponent saving, not an equal percentage improvement in measured runtime.

The argument in one paragraph. Turn digit multiplication into convolution. Use transforms to turn convolution into smaller pointwise products. Make two expensive ingredients cheaper: rearranging the addresses of stored data, and applying many two-point transform steps. Both improvements come from finite linear networks whose intermediate representations are chosen carefully. Prove a recursive saving after charging every auxiliary role and every representation change. Then show that layouts, resampling, precision, and exact recovery fit under the same final bound.

A route through the expanded paper. The new foundation chapters introduce each prerequisite with a concrete example before its technical use. First read the digit example and the tape explanation. Next read restoration, common frames, and the recurrence: these explain where the proposed saving comes from. The butterfly and polynomial examples explain what consumes the saving. The final chapters explain why reordering and rounding cannot be afterthoughts. In the full edition, each added explanation precedes the original technical material it illuminates. Added figures use the prefix E; original section, equation, theorem, and figure numbering are preserved.

What to carry forward. The central question is whether the entire computation, including data movement and exact recovery, can save a positive power of $\log n$.

Source and attribution. The original paper and its bibliography are preserved under the Apache-2.0 license in the accompanying source package. Its historical discussion credits Schönhage–Strassen, Harvey–van der Hoeven, Nussbaumer and Quandalle, and other prior work. The subsequent CrocSwap repository preserves community contributions separately. New explanatory prose, figures, and example checks in this edition were prepared with OpenAI Codex assistance on October 12, 2026. The source snapshot is frozen; this document is not a live record leaderboard.

See exactly which digit products get added

Added explanation Convolution without new machinery.

Every term in long multiplication has a destination place. In 23×14, the 3×4 product contributes to the ones place. Both 3×1 and 2×4 contribute to the tens place, because one factor in each product is a tens digit. Finally 2×1 contributes to the hundreds place. The diagonal colors in the grid identify those destinations.

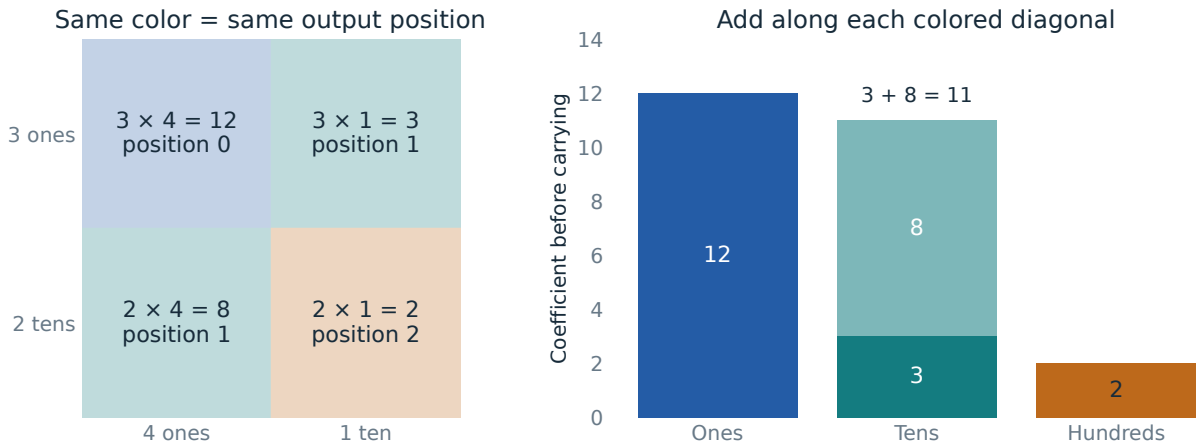


Figure E3. Convolution is addition along equal-place diagonals. The two teal cells contribute 3 and 8 to the same coefficient, giving 11. The coefficient list is $[12, 11, 2]$, before carrying. A cell is a specific digit product, not a generic stage of an algorithm.

Number digit positions starting at zero. A digit in position i multiplies a digit in position j into position $i + j$, because $10^i 10^j = 10^{i+j}$. That is the whole reason for the rule "add products with the same position sum." The word *convolution* names this rule. It is the same rule in binary or any other radix.

Why change to a polynomial? Replace the radix ten by a variable x. The two inputs become $A(x) = 3 + 2x$ and $B(x) = 4 + x$. Their product is a quadratic: it can be written $C(x) = c_0 + c_1 x + c_2 x^2$ for three unknown coefficients. At $x = 10$ it must give the integer product. But ten is not the only useful place to evaluate it. Easier sample points can reveal its coefficients.

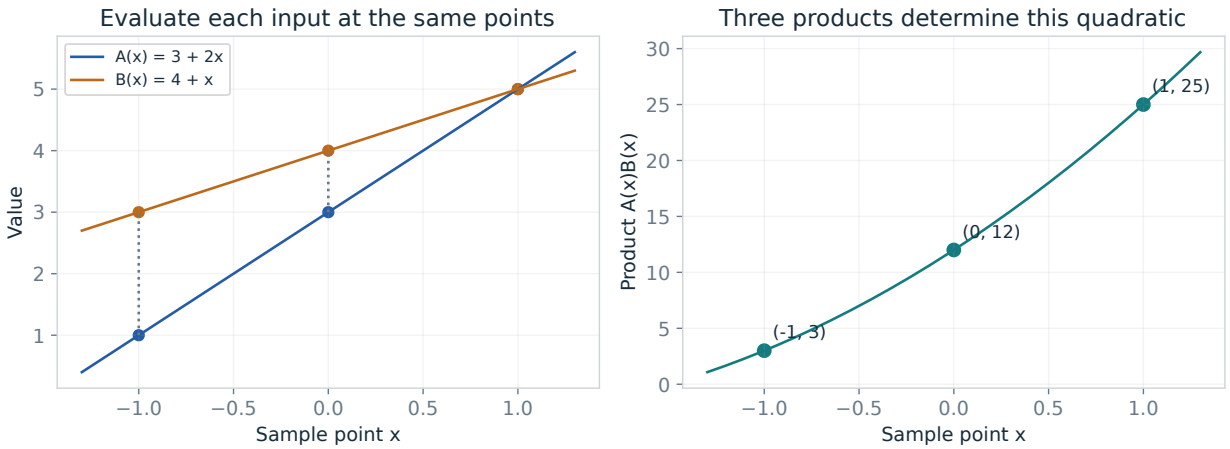


Figure E4. Compute values first, then recover coefficients. At $x = -1, 0, 1$, the input values multiply to $3, 12, 25$. Exactly one polynomial of degree at most two passes through those three distinct sample points. The curves are these specific polynomials, not a metaphor for a transform.

At zero, $C(0) = c_0 = 3 \times 4 = 12$. At one, $C(1) = 5 \times 5 = 25$. At minus one, $C(-1) = 1 \times 3 = 3$. Thus

$$c_0 = 12, \qquad c_1 = \frac{25 - 3}{2} = 11, \qquad c_2 = \frac{25 + 3}{2} - 12 = 2.$$

Subtracting the last two sampled equations isolates the odd-power term; adding them isolates the even-power terms. Evaluating at ten now gives $12 + 11 \cdot 10 + 2 \cdot 100 = 322$.

Where this example stops. This tiny interpolation example is not the paper's fast algorithm. It shows why "evaluate, multiply matching values, reconstruct" can work at all. For huge inputs, evaluation and reconstruction must themselves be fast and well controlled. Fourier transforms choose highly structured sample points so that those steps can share work. The next foundation chapter explains what their patterns look like.

What to carry forward. Convolution groups equal-place products. A transform provides another way to compute the same coefficients; it does not change the product.

From long multiplication to convolution

Added explanation A complete small example.

Write an integer in radix B as $a_0 + a_1 B + \cdots + a_{q-1} B^{q-1}$. The digits are listed from least significant to most significant. For the explanation take $B = 10$; the paper uses a power of two so that digit splitting is a binary operation. Introduce a formal variable x and form $A(x) = \sum_i a_i x^i$. Then the original integer is exactly $A(B)$.

For two inputs, the coefficients of the polynomial product satisfy

$$A(x)B(x) = \sum_k c_k x^k, \qquad c_k = \sum_{i+j=k} a_i b_j.$$

The second input polynomial is denoted $B(x)$ here; the unparenthesized B still denotes the radix. The sum defining c_k is *convolution*. It groups digit products that have the same place value. Unlike digits, the coefficients c_k can exceed the radix. Carry propagation repairs that after the coefficients have been computed.

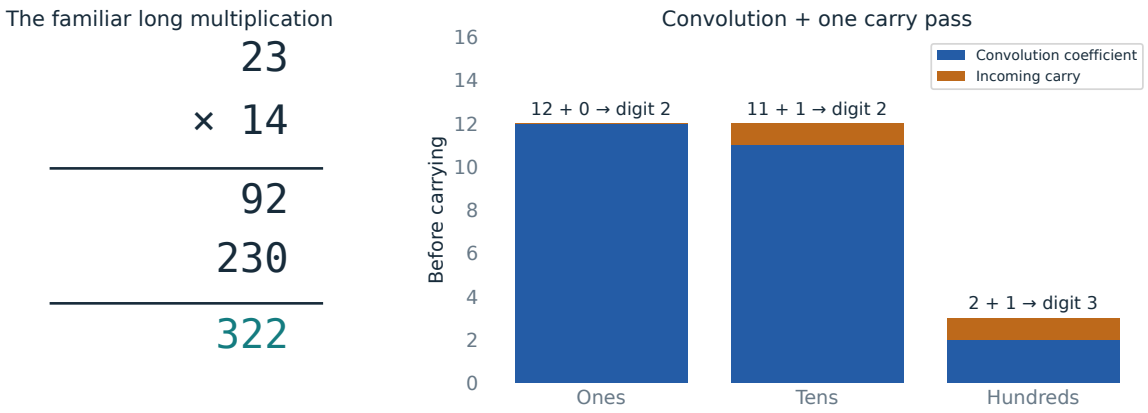


Figure E5. One multiplication, three representations. The coefficient list $[12, 11, 2]$ belongs to powers x^0, x^1, x^2. The carry is passed from lower to higher positions. The result is 322.

For $23 \cdot 14$, the coefficient of x is $3 \cdot 1 + 2 \cdot 4 = 11$. At position zero, 12 writes digit 2 and carries 1. Position one receives $11 + 1 = 12$, again writing 2 and carrying 1. Position two receives $2 + 1 = 3$. This is ordinary multiplication reorganized, with no approximation so far.

Why transforms enter. For a suitable transform F, convolution can be recovered from products of corresponding transform entries. Evaluating two polynomials at many roots of unity, multiplying those values, and transforming back replaces many cross-products by structured additions and pointwise products. The benefit depends on computing the transforms and arranging their inputs cheaply. The paper's transforms are normalized, so known scale factors must be put back; the slogan "transform, multiply, inverse transform" hides those factors.

Why padding is necessary. A cyclic convolution treats positions modulo a length S. A contribution at position S then wraps to position zero. That is useful algebraically but would corrupt this integer product. For degree at most $2q - 2$, choosing $S > 2q - 2$ and padding unused entries with zero prevents nonzero wraparound. The original assembly proves its chosen prime-axis product has enough capacity. It also bounds each coefficient before choosing its storage precision.

What to carry forward. The hard task is now to compute all convolution coefficients cheaply and accurately enough to recover the integers, then carry once.

Original source anchors: `00-introduction.tex; 08-assembly.tex, eq:capacity`.

Original abstract (reproduced source claim)

The following is the original author's abstract, preserved verbatim. See the opening explanation for this edition's verification scope.

We give a deterministic algorithm that multiplies two n-bit integers in $O(n(\lg n)^{1-\kappa})$ worst-case time, with $\kappa = 2^{-182}$, on one fixed finite-alphabet Turing machine with a fixed finite number of one-dimensional tapes. The algorithm is exact for every input length and disproves the $n \log n$ optimality conjecture of Schönhage and Strassen in this model.

Original manuscript continues below.

1 Introduction

Schönhage and Strassen proved in 1971 that two n-bit integers can be multiplied in $O(n \log n \log\log n)$ time on a multitape Turing machine, and proposed $n \log n$ as the optimal order of growth [25, Section 1]. After a sequence of improvements, Harvey and van der Hoeven reached the unconditional $O(n \log n)$ upper bound [15]. We obtain a strict power saving in the logarithmic factor, in the same fixed finite-tape model. The algorithm uses linear combinations of intermediate data to reduce the cost of both address rearrangement and Fourier-transform layers.

For a binary string x of length n, let $\mathrm{val}(x)$ be its nonnegative integer value, with the leftmost bit most significant. For $0 \le z < 2^k$, let $\mathrm{bin}_k(z)$ denote its binary representation padded on the left to length k. We consider exact multiplication: on input $x\#y$, with $x, y \in \{0,1\}^n$, the output must be $\mathrm{bin}_{2n}(\mathrm{val}(x)\,\mathrm{val}(y))$. Write $\lg n = \max\{\lceil \log_2 n \rceil, 1\}$.

Theorem 1. *There is one deterministic Turing machine A, with a fixed finite alphabet and a fixed finite number of one-dimensional tapes, that computes this exact product for every $n \ge 1$ and every pair of n-bit inputs. Its worst-case running time satisfies*

$$T_A(n) = O\Big(n(\lg n)^{1-\kappa}\Big), \qquad \kappa = 2^{-182}.$$

The bound is asymptotic through all input lengths. The constants and thresholds in the construction are extremely large; the algorithm is intended to establish a bit-complexity bound. The theorem refutes an eventual $\Omega(n \log n)$ lower bound for every fixed multiplication machine in this model.

1.1 The two tape operations

Splitting the inputs into radix digits reduces multiplication to convolution of short integer coefficients. Our starting point is the Gaussian-resampling reduction of Harvey and van der Hoeven [15]: it replaces the Fourier transforms needed for this convolution by multidimensional transforms whose axis lengths are powers of two. After a further conversion to convolution, one coordinate is stored as the coefficient list of a polynomial modulo $y^r + 1$, with r a power of two. Powers of y then serve as roots of unity in the remaining axes, and multiplication by such a root is a signed shift of the coefficient list. These are the synthetic polynomial transforms of Nussbaumer and Quandalle and Nussbaumer [21, 20].

In this representation, the transform calculation consists of address rearrangements, signed shifts, and two-point butterfly operations. Scanning an array of $\Theta(n)$ stored bits at each of $\Theta(\log n)$

transform levels already costs $\Theta(n \log n)$. We therefore need savings in both movement and arithmetic. Two tape procedures supply them; in both bounds, V denotes the stored bit volume.

The first procedure interchanges two disjoint contiguous address fields of u bits in an array whose addresses form a complete Cartesian product. Its cost is $O(Vu^{\tau})$ for a fixed $\tau < 1$. Although its final effect is an exact permutation, its intermediate steps form pointwise XOR combinations of data. The final permutation is exact for arbitrary initial auxiliary values.

The second procedure applies

$$H_0(u, v) = ((u + v)/2, (u - v)/2)$$

simultaneously on selected coordinate bits of the numerical arrays. Each record is a polynomial with complex fixed-point coefficients, and H_0 acts coefficientwise. On the layouts used in the multiplication algorithm, one selects the same bit position in each of at most d equal-width address chunks, and the cost is $O(Vd^{\lambda'})$ for a fixed $\lambda' < 1$. Intermediate values are exact Gaussian dyadics, whose real and imaginary parts have power-of-two denominators. A single truncation follows the completed contractive layer. Section 4 proves the address bound, and Section 5 gives the numerical layout, precision hypotheses, and layer bound.

1.2 Linear networks and the recursive saving

Each procedure arises from its own fixed linear network that exchanges two banks of values, with prescribed signs, and restores every auxiliary value. A wire's fixed place in the network is called its role. To use the network on an array, regard a row as the complete ordered suffix, including all active address fields, at a fixed prefix and row index. Distribute rows cyclically among the W roles, padding the row count for each prefix to a multiple of W. Let V now denote the node volume after padding. All role streams have the same local address set and each contains V/W bits. A scalar gate is then applied pointwise to matching addresses on the roles that it touches.

The representation of a role's array may change along the network. At a source, gate, or sink v, choose an invertible linear operator D_v on the space of arrays indexed by the common address set. We call D_v its frame: the stored array is $D_v g$ when the logical array is g. All incidences of one gate have the same frame. Along an edge from u to v, the operator $D_v D_u^{-1}$ changes the stored array from $D_u g$ to $D_v g$. At a pointwise linear gate G, the common frame commutes with the gate:

$$G(D_v g_1, \ldots, D_v g_t) = D_v G(g_1, \ldots, g_t).$$

The supplied input needs no preliminary change of frame: its logical interpretation is defined by applying the inverse input frame. Thus the intermediate frames cancel, and a route from input role w to output role $\rho(w)$ applies $D_{\mathrm{sink}(\rho(w))} D^{-1}_{\mathrm{source}(w)}$, together with its prescribed scalar sign. Undoing the role exchange and those signs leaves the endpoint operator on every role array, including the roles that carried auxiliary scalar values.

For the bit procedure, that endpoint operator is an address shear: two fields, written as vectors H, D over a common residue ring, are updated by $(H, D) \mapsto (H + D, D)$. Two further linear-cost address updates complete their interchange. For the numerical procedure, the endpoint operator becomes the simultaneous H_0 layer after diagonal phase operations and a dyadic scale. The intermediate frames are chosen so that every edge can be implemented by smaller calls to the same procedure.

The finite construction in Section 3 attaches subspaces of a fixed m-dimensional space to its gates and terminals. The subspaces at adjacent vertices are nested. Their dimension changes count the smaller operations needed on an edge, with an additional source contribution in the bit

construction. The construction uses three-element subsets: gather and scatter create coefficients depending on intersection size, and auxiliary wires cancel the unwanted coefficients. Orthogonality of the corresponding indicator vectors permits the subspace assignments. The decisive property is that the total number s of smaller calls satisfies $s < Wm$.

A recursive call at parameter mf therefore uses s calls at parameter f, each on a stream of volume V/W. The parameter counts field digits for address interchange and selected axes for a numerical layer. After dividing cost by volume, multiplying the parameter by m increases the recursive contribution by $s/W < m$. This strict inequality is the source of the power saving. The later tape and arithmetic estimates preserve it, including the work of changing numerical address coordinates. Those changes are implemented by whole-chunk moves and controlled shifts, followed by a deterministic correction of the explicitly identified exceptional addresses.

1.3 From transforms to the exact product

The remaining proof places these procedures inside the multiplication reduction and accounts for every change of representation. First, the radix-digit coefficient list is placed on a product of distinct prime cyclic axes by a Chinese-remainder map that preserves convolution. Section 9 implements this map through triangular controlled shifts and charged axis moves. Gaussian resampling then expresses each source Fourier transform through a power-of-two tensor transform. Section 8 retains the source and target coordinate permutations in the transform identity, so they need not be executed as separate full sorts.

Next, Bluestein's chirp identity converts the middle transform, including its target permutation, into normalized cyclic convolution. A phase twist changes the last cyclic coordinate into a polynomial modulo $y^r + 1$; its negative wrap gives precisely the original cyclic wrap after the twist is removed. Sections 6 and 7 compute convolution on the remaining axes over this polynomial ring. The two forward synthetic transforms retain the same frequency order through their pointwise product, and the opposite transform consumes that order. Their polynomial products use signed packing and the established $O(n \log n)$ multiplier as an ordinary subroutine.

Finally, the source permutation retained during resampling cancels across the two forward source transforms, their pointwise product, and the opposite-sign transform. Section 9 removes the known normalizations and digit scales, proves that the recovered coefficient error is strictly below $1/2$, and propagates carries to obtain the exact binary product. A compatible choice of rational parameters makes all terms in its cost table fit Theorem 1. Figure 1 summarizes these reductions.

1.4 Other arithmetic operations

Corollary 2 (Exact division and integer square root)**.** *There are deterministic machines with fixed finite alphabets and fixed finite numbers of one-dimensional tapes performing the following tasks in $O(n(\lg n)^{1-\kappa})$ worst-case time, for every $n \geq 1$. For $x, y \in \{0,1\}^n$, put $a = \mathrm{val}(x)$ and $b = \mathrm{val}(y)$. If $b > 0$, the division machine takes $x\#y$ and returns $\mathrm{bin}_n(q)\#\,\mathrm{bin}_n(r)$, where $q = \lfloor a/b \rfloor$ and $r = a - bq$. The square-root machine takes x and returns $\mathrm{bin}_n(\lfloor \sqrt{a} \rfloor)$. Leading zeroes are allowed, and κ is as in Theorem 1.*

The classical Newton reductions described by Brent [5, Lemmas 2.1–2.4] compute reciprocal and square-root approximations at geometrically increasing precisions. Their multiplication costs are bounded by a geometric sum controlled by the last precision, and exact remainder and square tests recover the required integer answers. Section 10 gives the complete fixed-tape implementation.

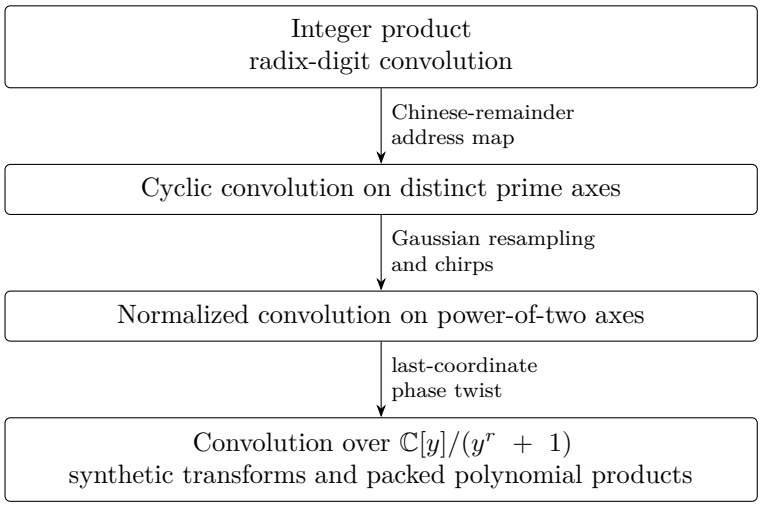


Figure 1: The reductions connecting the tape procedures to multiplication. The arrows describe changes of problem and representation. Fast address interchanges organize the layouts, and simultaneous butterfly layers accelerate the synthetic transforms. Solving the final polynomial-ring convolution evaluates the transforms needed in the preceding stages. Known rescalings, rounding, and carry propagation then recover the exact integer product.

1.5 Historical context and inherited methods

Karatsuba's recursive construction, published jointly with Ofman in 1962, gave a binary multiplication circuit of size $O(n^{\log_2 3})$ [18]. Toom's subsequent use of polynomial evaluation and interpolation gave circuit size $n \exp(O(\sqrt{\log n}))$, and hence $O(n^{1+\eta})$ for every fixed $\eta > 0$ [26]. These early results were formulated in terms of automata and logical networks. Cook adapted Toom's method to the Turing-machine setting; the contemporary account of Schönhage and Strassen credits this adaptation explicitly [25, Section 1, p. 282].

In the multitape Turing model, Schönhage and Strassen proved the bound $O(n \log n \log\log n)$ in 1971 [25]. Fürer improved it to $n \log n \, 2^{O(\log^* n)}$ in 2007, with a full journal account in 2009 [10, 11]; here $\log^* n$ counts iterated logarithms until a constant threshold is reached. De, Kurur, Saha and Saptharishi developed a modular-arithmetic route to the same scale [7]. Later improvements made the exponential factor $8^{\log^* n}$ and then $4^{\log^* n}$ [17, 14]. Harvey and van der Hoeven proved the unconditional $O(n \log n)$ bound [15].

The multitape bounds just cited concern that fixed model. Schönhage obtained linear-time multiplication on storage-modification machines [24]. Groff later gave a smaller-than-$\log n$ factor in a unit-cost RAM model with constant-time arbitrary memory access, after polynomial preprocessing whose cost is excluded from that bound [13, Sections 2 and 4]. Neither result establishes the bound in the fixed finite-tape model of Theorem 1.

The transform methods behind these bounds also underlie the present construction. Schönhage–Strassen multiplication uses roots of unity in Fermat-type residue rings, where certain multiplications become shifts. Fürer combines cheap polynomial roots of unity with more general roots, reducing the frequency of expensive multiplications. Harvey and van der Hoeven use Gaussian resampling to pass to multidimensional transforms with power-of-two dimensions, then evaluate them using polynomial transforms developed by Nussbaumer and Quandalle and subsequently by Nussbaumer [21, 20, 15]. The Gaussian-gridding work of Dutt and Rokhlin is an earlier conceptual precursor [8]; Harvey and van der Hoeven discuss the connection in [15, Section 4.4.3]. The

quantitative resampling and bit-cost estimates used here are those of the latter paper.

More specifically, the proof adapts Harvey and van der Hoeven's Gaussian-resampling identity, contraction bounds, and short-convolution procedures [15, Theorem 4.2, Proposition 4.7 and Lemmas 4.8–4.12]. Their framework already permits arbitrary dimension. Here the source and target permutations remain in the transform identity, while the faster tape procedures perform the required axis movements. Bluestein's conversion to convolution [4] and the synthetic-root convolution algorithm are retained; the latter follows the polynomial-transform construction in [15, Section 2.4 and Section 3]. Signed coefficient packing is a form of Kronecker substitution, with the fixed-tape version of [15, Lemma 2.5] as a direct antecedent. Calls to the established $O(n \log n)$ multiplier are ordinary subroutine calls; they do not invoke the improved theorem recursively.

1.6 The finite-network ingredients

The subset-intersection construction in Section 3 has an earlier source outside multiplication. Alon's presentation of the Frankl–Wilson construction [9] gives low-degree representations of complementary intersection graphs over different fields [2, Section 3, remark after the proof of Theorem 1.1]. At the prime $p = 2$, its subsets have size three and the two polynomial evaluations are the intersection count modulo two and the intersection count minus one. These are the two pairing formulas used here. We change the ground-set size and express the second pairing through a rational bilinear form.

The gather and scatter operations, together with the side wires, have the form of linear index-code decoding: a low-rank linear summary gives the wanted coordinate plus contributions from known neighboring coordinates, and those contributions are subtracted. Bar-Yossef, Birk, Jayram and Kol establish the fitting-matrix formulation and this decoder [3, Section 3, proof of Theorem 5]. Lubetzky and Stav treat decoding over other fields and use related intersection representations to separate their possible ranks [19, Proposition 2.1 and Section 2.3]. These comparisons explain the algebraic role of the two fields here; they do not supply the subspace labels on the subsequent network.

Restoring scratch registers with arbitrary initial contents also has a precise precedent. The transparent-computation construction of Buhrman, Cleve, Koucký, Loff and Speelman subtracts an initial register value, performs a computation, adds the resulting value, and reverses the computation [6, Section 3]. Our scratch schedule is the linear-readout version of that construction. The rank reduction used for address shears is a singular-matrix Bruhat factorization: after reversing the coordinate order, Grigor'ev's upper-triangular factors become the two lower-triangular factors needed here [12, Appendix A.2, Proposition 14(c)].

The finite networks and their labels are proved directly in Section 3. The additional requirements are the three-stage bank exchange, nested subspaces with a strict total rank or dimension deficit, and endpoint frames acting on every role. The tape implementations then turn that deficit into the volume-normalized recurrences of Sections 4 and 5, including the costs of addressing, padding, coefficient growth, and finite precision. The cited decoding, restoration, and factorization results provide ingredients; these further properties are established in the present proof.

1.7 Lower-bound scope and matrix transposition

Schönhage and Strassen conjectured $n \log n$ as the optimal order of multiplication cost [25, Section 1]. In the present fixed-tape model, an eventual lower bound for every multiplication machine means that, for each fixed machine A, there are constants $c_A > 0$ and N_A such that $T_A(n) \geq c_A n \lg n$ whenever $n \geq N_A$. Its negation requires one machine satisfying $\liminf_{n\to\infty} T_A(n)/(n \lg n) = 0$. The

power saving proved here gives a limit of zero through all input lengths, which is a stronger conclusion.

Harvey and van der Hoeven consider a binary transposition machine supplied an integer $m \geq 1$ and a $k \times k$ binary matrix, where $k = \lfloor \sqrt{m} \rfloor$. They proved that if every fixed such machine has worst-case cost $\Omega(m \lg m)$ through all sufficiently large supplied values of m, then every fixed multiplication machine has the corresponding eventual lower bound [16, Theorem 1.2]. Combining their quantitative reduction [16, Corollary 6.2] with Theorem 1 gives a deterministic fixed-tape algorithm that transposes a row-major $k \times k$ binary matrix in $O(k^2(\lg k)^{1-\kappa})$ time, with κ as in Theorem 1. Section 11 proves the more general rectangular-matrix bound.

This comparison concerns unrestricted computation on a fixed number of one-dimensional tapes. Our address procedure forms XOR combinations of intermediate data, even though its final effect is a permutation. It therefore lies outside models restricted to moving data without such computations; see [16, Section 1.4] for the distinction.

1.8 Companion application to exact Fourier transforms

The complex phase network of Proposition 8 is also used in the companion manuscript [22, Section 1.1, Theorem 1.1 and Corollary 1.2]. That manuscript gives a single algorithm for the exact discrete Fourier transform at every positive transform length n. As $n \to \infty$, its cost is $O(n(\log n)^{1-10^{-13}}) = o(n \log n)$.

The cost is measured in an exact complex-register model that assigns unit cost to exact complex field operations. The algorithm accepts every $x \in \mathbb{C}^n$, with no bound on its entries or on intermediate complex magnitudes. Its computation on the input uses only addition, subtraction, and multiplication by prepared input-independent scalars. The magnitudes of those scalar coefficients are unrestricted as well.

Integer operations and random access on $O(\log(n+2))$-bit words have unit cost, with a fixed number of words for each address or integer. The model supplies one specified Fourier root ζ_{D_*} of explicitly computable order $D_* < 1024n^3$. Scalar preparation, schedule construction, and index work are charged. This is a separate application of the finite network, with its own uniform array implementation. It is neither a bit-complexity consequence nor a stable floating-point FFT consequence of the fixed-tape multiplication theorem, Theorem 1.

A matrix transpose is a physical rearrangement

Added explanation Follow all nine records.

Read the 3×3 grid row by row: $0, 1, 2, 3, 4, 5, 6, 7, 8$. Now read it column by column: $0, 3, 6, 1, 4, 7, 2, 5, 8$. The records have not changed internally. Their positions in the stream have. The crossing paths show the exact permutation, including the records whose positions stay fixed.

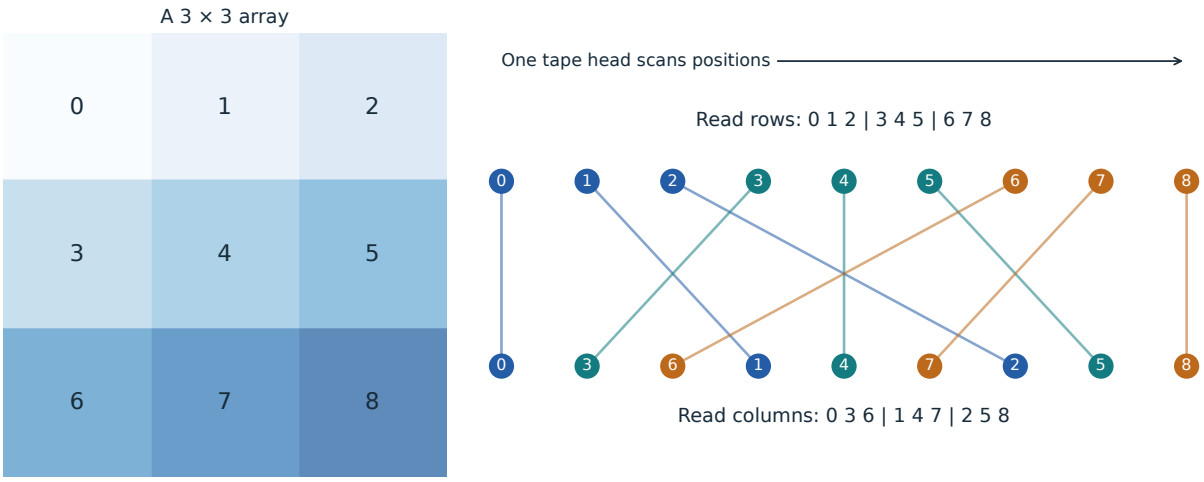


Figure E6. The geometry of one concrete data move. Colors identify original rows; every numbered record is connected to its destination. The top scan arrow represents a tape head moving through consecutive positions. It cannot teleport to the next useful record.

A computer program can write a new indexing formula in one line. That does not prove that the underlying records have been physically rearranged in one step. In the paper's machine model, a head moves at most one tape cell per step. Large data movement must therefore be implemented and charged.

Why a calculation can help perform a permutation. For a pair of bits, three XOR updates exchange their values:

$$x \leftarrow x \,\mathrm{XOR}\, y, \quad y \leftarrow y \,\mathrm{XOR}\, x, \quad x \leftarrow x \,\mathrm{XOR}\, y.$$

For initial $(x, y) = (1, 0)$ the states are $(1, 0), (1, 1), (0, 1)$. XOR is addition modulo two: equal bits give zero and different bits give one. Intermediate values are combinations, yet the final result is a pure swap. The large construction also uses intermediate linear combinations to achieve an overall array operation. This two-bit identity only explains the possibility; it does not establish the large construction's speed.

Keep two meanings of geometry separate. The grid here represents positions on an array. The subspaces and ranks used later represent algebraic directions of operators on such arrays. A picture of a short physical path is not by itself a low-rank certificate, and a low-rank identity is not by itself a tape-time bound. The argument needs a precise connection between them.

What to carry forward. The stored order is part of the problem. A useful reordering must be realized by actual steps, not just by renaming indices.

Why moving data is part of the mathematics

Added explanation Read before the machine-model section.

Imagine a long row of labeled boxes, with a head that can move one box per step. The label is a conceptual address; the machine does not get free random access just because we can write $A[i, j]$ on paper. A multidimensional array is flattened into one ordered stream. In lexicographic order, the last coordinate changes fastest. For a 2×3 array the order is

$$(0,0), (0,1), (0,2), (1,0), (1,1), (1,2).$$

Changing to the opposite coordinate order rearranges the records. On this machine, the cost includes getting them into those positions.

A role is a place in a circuit; a row is a block of data. The finite network has W roles, counting both data roles and scratch roles. At a recursion node, complete rows are distributed cyclically among them. A row contains the entire suffix of active address fields. That completeness ensures that each role sees the same local address set. After suitable padding, each role receives exactly a $1/W$ fraction of the logical volume.

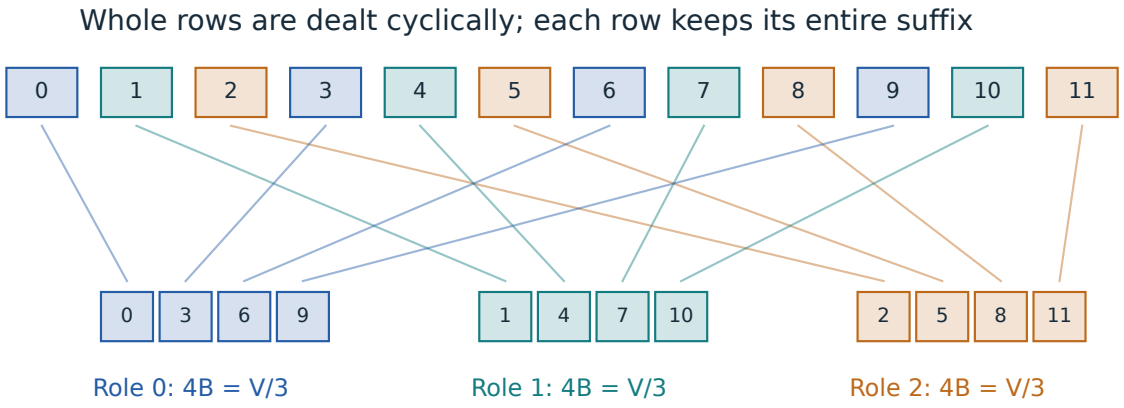


Figure E7. The volume division that drives the recurrence. Each illustrative row has B bits. Twelve rows give $V = 12B$; three role streams each hold $4B$. The actual paper uses much larger fixed networks.

Why auxiliary roles are included. Suppose we counted only useful output rows and forgot workspace. A construction could appear cheap merely by hiding a large amount of temporary data. The network's W counts all roles. Scratch roles can hold arbitrary existing data because the scalar circuit restores them. They are not presumed free zeroed memory. The later framed array operation must also be correct on these roles.

Logical volume is not permission to ignore copying. V names the data volume used to normalize a recursive call. Physical copies, padding, counter updates, tape-head returns, and cleanup still consume time. They are charged as overhead. When a child is active, the paper's schedule parks its parent's streams and restores them on return. The claimed accounting requires that it does not repeatedly sweep through every parked ancestor.

A fixed machine can still process growing descriptors. Input lengths, axis lengths, and call states are stored as data. The number of tapes and the alphabet cannot grow with n. This is why a proof about a finite circuit is only one ingredient: it must be transferred to uniform stream operations with explicit costs for manipulating those descriptors.

What to carry forward. Watch both the number of recursive calls and the amount of data each call processes. A saving in one can disappear if the other is miscounted.

Original source anchors: `02-streams.tex, lem:elementary-streams`.

Original manuscript continues below.

2 The machine model and elementary streams

A machine has one fixed finite alphabet and one fixed finite number of one-dimensional tapes. A step reads and writes at most one cell on each tape and moves each head by at most one cell. Fixed tracks and finitely many work tapes change time by a fixed factor. We distinguish moving one digit in a fixed radix, which needs a bounded number of scans, from interchanging fields whose numbers of digits grow. The latter permutation will be realized in Section 4 by computing XOR combinations of intermediate data streams. The present section supplies the elementary scans it uses. Every scan includes its counters and the return of its tape heads.

2.1 Array order and elementary scans

For a positive integer a write $[a] = \{0, \dots, a-1\}$. An array with address set $[a_1] \times \cdots \times [a_t]$ is stored in lexicographic order, with the first coordinate most significant. Its entries are bits unless a record width is specified. A trailing record of B bits can equivalently be regarded as a final address field $[B]$. An address is implicit in an entry's position in this ordered stream; it is not stored beside every data bit. The *logical volume* V is the number of stored data bits. Work tapes and duplicate streams are charged in time but do not increase V in recurrences. All the fields discussed below are nonempty. A call with an empty array is handled directly.

An address permutation f moves the entry at x to $f(x)$. Thus its action on an array A is $(fA)(f(x)) = A(x)$. Adjacent fields which are irrelevant to an operation may be replaced by a single field whose length is their product. In particular, the elementary procedures below have a fixed number of formal address fields even when a surrounding algorithm has a growing list of coordinates. Lengths and positions are supplied in canonical binary on descriptor tapes; no additional tape head is attached to an address coordinate. Each elementary call receives a fixed number of collapsed canonical lengths. Constructing them from a longer surrounding list is work of the caller; recursive calls below construct their own collapsed descriptors within their stated bounds.

When one address field is designated as a row field, collapse the fields before it to a prefix $[P]$ and those after it to a suffix $[B]$. If the row field has range $[R]$, a *row* at prefix p and row index r is the complete consecutive block $\{p\} \times \{r\} \times [B]$. Every row therefore contains B bits in the same suffix order.

For headers that concatenate a list of positive integer parameters, we use the self-delimiting code

$$b_v = \lfloor \log_2 v \rfloor + 1, \qquad \Gamma(v) = 0^{b_v - 1} \operatorname{bin}_{b_v}(v), \qquad |\Gamma(v)| = 2b_v - 1 \quad (v \geq 1). \tag{1}$$

The initial zeroes specify how many further bits follow the first one, so concatenated codes can be read in order. We use this code when an interface specifies a header; elementary calls still receive the fixed number of canonical binary lengths described above.

Lemma 3 (Elementary stream operations)**.** *Fix a radix $q \geq 2$. Each of the following operations on an array of volume V takes $O(V)$ steps on a fixed number of one-dimensional tapes:*

1. *any fixed permutation of the pair of digit values at two specified radix-q positions, whether or not the positions are adjacent;*
2. *moving one specified radix-q digit to another position, preserving the order of the intervening positions;*
3. *splitting on a specified radix-q digit, applying a fixed pointwise map to aligned streams, and merging them back;*
4. *for positive integers P, R, B, within each prefix of an array on $[P] \times [R] \times [B]$, splitting whole rows cyclically into a fixed number W of streams, and, when W divides R, merging those streams in any fixed order of the row classes.*

The bounds include shape preparation, counter initialization and resets, and returning or clearing the visited portions of work tapes. Their constants may depend on the fixed radix, the fixed pointwise map, and the fixed number of row classes.

Proof. For the first operation split into q^2 streams according to the two specified digits. With these digits fixed, the order of all remaining digits is unchanged by the permutation. Enumerate output addresses in their required order and read the stream prescribed by the two preimage digits. For a digit move use q streams instead. The same observation shows that splitting on one digit gives streams with the same remaining address set in the same order. Scan their aligned entries, apply the fixed pointwise map, and merge according to the output digit; this proves the third operation. For the fourth, write $r = Wg + w$, where $0 \leq w < W$, and send the whole row to stream w. If W divides R, each stream has address set $[P] \times [R/W] \times [B]$, indexed by the prefix, g, and the unchanged suffix. Thus the streams have the same shape and each has volume V/W. The inverse merge, with any fixed permutation of the classes, reads them in the specified cyclic order.

Here are the counter details used in these scans and later ones. A binary counter is stored with its head at the least significant end. For a counter of width w, incrementing or decrementing and returning this head over L consecutive counts costs $O(L + w)$: the number of visits to position j is $O(1 + L/2^j)$. The width term is paid when initializing the counter and is absorbed by its complete range scan. A countdown of length L can be initialized, reset, and tested for underflow in $O(\log(2L)) = O(L)$ time. Keep a local copy of its length on a dedicated track, so a reset does not search a larger descriptor. For a nested scan of a fixed number of nonempty ranges, the reset of a range is charged to the scan of that range. Induction on the number of ranges therefore gives linear total counting cost. A fixed number of empty pieces in a split can be charged to the adjacent nonempty combined range.

Only a fixed number of collapsed lengths are needed. Computing these lengths and the initial counter values by ordinary binary arithmetic takes $O((\log(2V))^C)$ steps for a fixed C. This is $O(V)$ uniformly in $V \geq 1$, after increasing a fixed constant. Each stream is scanned a fixed number of times, and every rewind, copy, or erasure is over a region whose length was charged to that scan. Delimiters and the fixed number of tracks require only a fixed alphabet enlargement. □

The next operations are linear because their target field follows their control fields. This order will be preserved whenever they are invoked.

Lemma 4 (Controlled shifts and fixed rational scalings)**.** *Fix a prime q, a fixed number of control fields, and finitely many rational coefficients whose denominators are prime to q. Whenever a coefficient is used as a unit, also require its numerator to be prime to q. Put $Q = q^b$, $b \geq 1$. Suppose a target field $y \in [Q]$ follows its control fields, each control used in an offset having $O(b)$ digits. An update*

$$y \longmapsto y + a(\mathit{controls}) \pmod{Q}$$

costs $O(V)$ if the offset can be prepared in $b^{O(1)}$ time from the fixed number of control values. In particular, this holds for a fixed rational linear combination of them, specialized modulo Q. Multiplication of y by any of the specified rational units, and its inverse, also costs $O(V)$. Arbitrarily long intervening spectator fields and suffix records are permitted.

Proof. Fix all fields before the target, and let the suffix length be B bits. The resulting fiber consists of Q consecutive blocks of length B. For an offset $a \in [Q]$, split these blocks at $Q - a$. Append the two pieces to two tapes, doing so successively for all prefix fibers. In the merge read the second piece and then the first piece for each prefix. The pieces have become the two consecutive ranges $[0, a)$ and $[a, Q)$ of the new target coordinate. Recompute a when starting that prefix in the merge. Initializing these two piece lengths and returning the heads of the relevant control counters is included in the $b^{O(1)}$ offset work. Since

$$b^C = O(q^b) = O(Q)$$

for each fixed C, this work is absorbed by the $QB \geq Q$ bits of the fiber. Unrelated spectator counters are incremented by ripple counting; their entire contents are not consulted when computing the offset.

For positive fixed integer c prime to q, the permutation $y \mapsto cy \pmod{Q}$ has c increasing pieces, classified by

$$j = \left\lfloor \frac{cy}{Q} \right\rfloor, \qquad 0 \leq j < c.$$

Prepare their endpoints once for this Q, split each fiber into these c streams, and enumerate the output coordinate $y' = 0, \ldots, Q-1$. The required stream is the unique $j \in [c]$ for which $y' + jQ \equiv 0 \pmod{c}$. Uniqueness follows from $\gcd(Q, c) = 1$; the corresponding $y = (y' + jQ)/c$ lies in $[Q]$. The stream choice is maintained by a finite residue counter. Reversing the roles of this split and merge implements multiplication by c^{-1} modulo Q.

Negation leaves the block at zero fixed and reverses the order of the other $Q - 1$ blocks. Push these blocks consecutively to a temporary tape. Popping gives their order reversed, but also reverses the bits in each block. Reverse each popped block on a second temporary tape before writing it, restoring its internal order. Every bit is moved a bounded number of times. The same procedure is repeated for successive fibers; the temporary stack is emptied and its head returned locally. A signed rational unit is a composition of these integer multiplications, inverse multiplications, and possibly negation. Its description is fixed.

The nested scan and reset analysis of Lemma 3 applies to all these operations. In particular, copying a suffix-length counter at every target position costs $O(\log(2B)) \leq O(B)$ at that position. It does not introduce a factor $\log V$ per bit. □

When coefficients have a denominator d prime to q, their reductions modulo q^b can be computed by the extended Euclidean algorithm using ordinary tape arithmetic. The dimension and all original

coefficients are fixed, so these preparations have polynomial cost in b. Nothing in Lemma 4 assumes unit-cost arithmetic.

For later use, fixed-width signed fixed-point binary addition, subtraction, sign change, and multiplication or division by a fixed power of two are streaming operations of linear bit cost, provided enough integer and fractional guard bits have been allocated for exact results. If an input record is most-significant-bit first, reverse it onto the arithmetic work tape. With least significant bits first on that tape, propagate carries using finite control. Reverse the completed record if its required external order is opposite. This convention separates the order of bits inside a record from the lexicographic order of records. Each required reversal is a linear pass and is charged.

Vectors, rank, and a frame you can draw

Added explanation The geometry behind the technical vocabulary.

A vector can be as simple as a pair of numbers, such as $(1, 2)$. Draw it as an arrow from the origin to that point. Adding vectors adds their coordinates. Multiplying one by a number scales the arrow. A subspace is a collection closed under these operations: a line through the origin is one example, and the whole plane is another.

A matrix is a rule for combining input coordinates into output coordinates. Its columns tell you where the basic input directions go. If every output lies on a single line, there is only one independent output direction, even if the matrix has many nonzero entries. That number of independent directions is its rank.

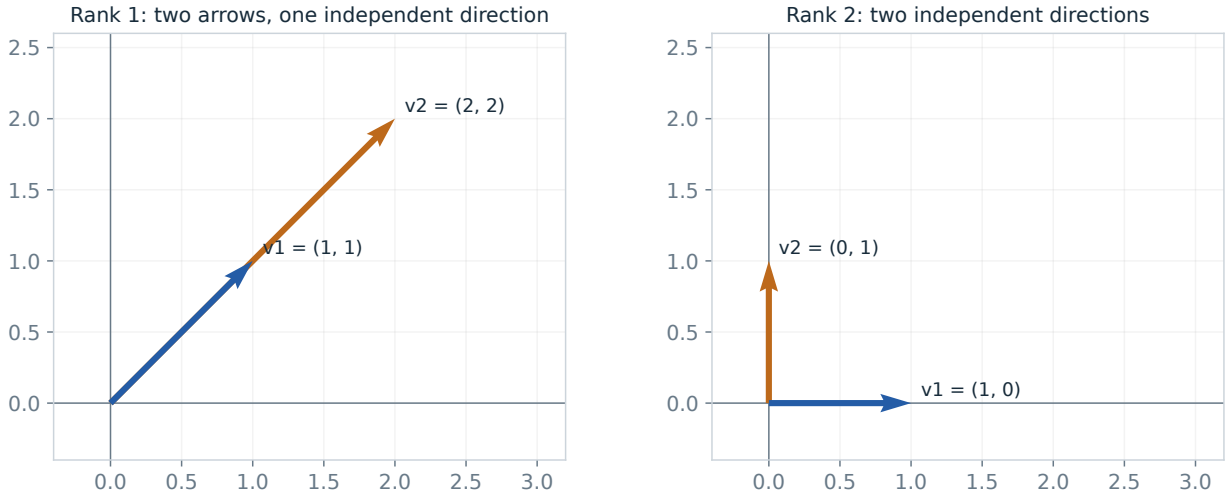


Figure E8. Count independent directions, not arrows. On the left, $(2, 2)$ is twice $(1, 1)$: both arrows lie on one line. On the right, no multiple of $(1, 0)$ reaches $(0, 1)$: both directions are needed. These represent ranks one and two respectively.

The matrices

$$\begin{pmatrix} 1 & 2 \\ 1 & 2 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

have ranks one and two. The first has four nonzero entries but lower rank. For that first matrix, input (s, t) produces $(s + 2t, s + 2t)$, always on the diagonal line. The second can produce every point in the plane. This is why counting matrix entries would miss the saving sought by the paper.

A frame is a consistent representation rule. Take D to rotate every vector a quarter-turn counterclockwise. The logical value $(1, 0)$ is stored as $(0, 1)$. The logical value $(0, 1)$ is stored as $(-1, 0)$. Adding the stored values gives $(-1, 1)$, exactly the rotation of the logical sum $(1, 1)$. Both inputs used the same rule, so addition remained meaningful.

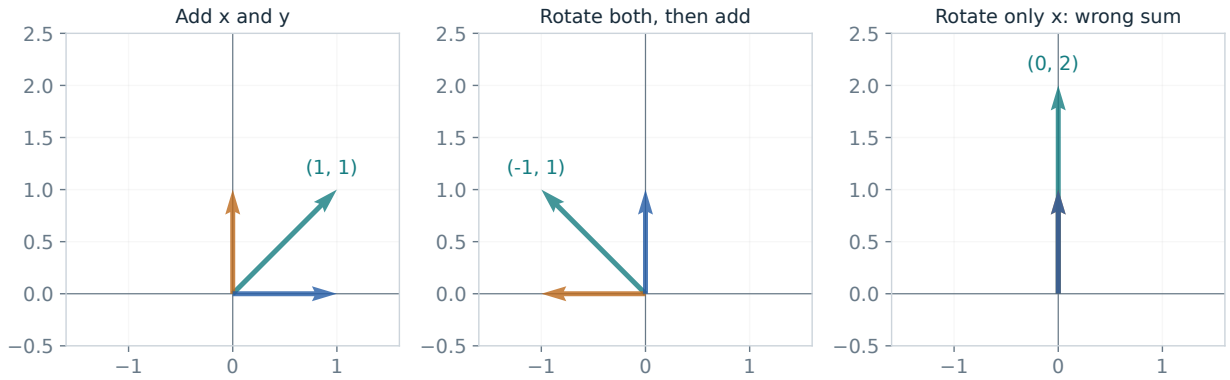


Figure E9. Mixing representations changes the answer. Blue and gold are the two inputs; teal is their sum. Rotating both inputs rotates the correct sum. Rotating just the first produces $(0, 2)$, which is not the intended rotated result $(-1, 1)$. Overlapping arrows in the right panel deliberately show the two equal directions.

In the paper, a frame acts on an entire address-indexed array rather than just a two-number vector. The same law of linearity applies. The later two-by-two tile diagram gives a concrete array version: rotate the address grid of each input and then add matching cells. You get the rotated sum.

Why changing frames can be worthwhile. One representation may make a particular gate cheap while another makes the next gate cheap. The algorithm can switch, provided it pays for switching and maintains the interpretation consistently. Its endpoint frames determine the intended final operation; its intermediate frames are chosen to keep the sum of transition costs small. The actual proof uses more sophisticated operators than rotations, and verifies their residual ranks explicitly.

What to carry forward. Rank measures independent directions. A frame is an invertible representation. Shared frames preserve linear gates; unpriced frame changes would hide real work.

Borrowed workspace and changing representations

Added explanation Read before the finite-network proof.

Two ideas that initially seem unrelated work together here. First, a linear calculation can borrow a register without knowing its contents, provided it removes the influence of those contents and restores them. Second, arrays can be represented in different coordinates at different places in a circuit. The cost of changing representation can be smaller than directly performing the desired overall array operation.

A four-update restoration example. Let a be an arbitrary scratch value. Starting from (x, y, a), perform

$$y \leftarrow y - a, \quad a \leftarrow a + x, \quad y \leftarrow y + a, \quad a \leftarrow a - x.$$

The final state is $(x, y + x, a)$. The initial a cancels from the target, and the scratch register returns exactly to its initial value. This is a toy identity over an additive ring, not a proposed implementation of the full network. In characteristic two, addition and subtraction both become XOR.

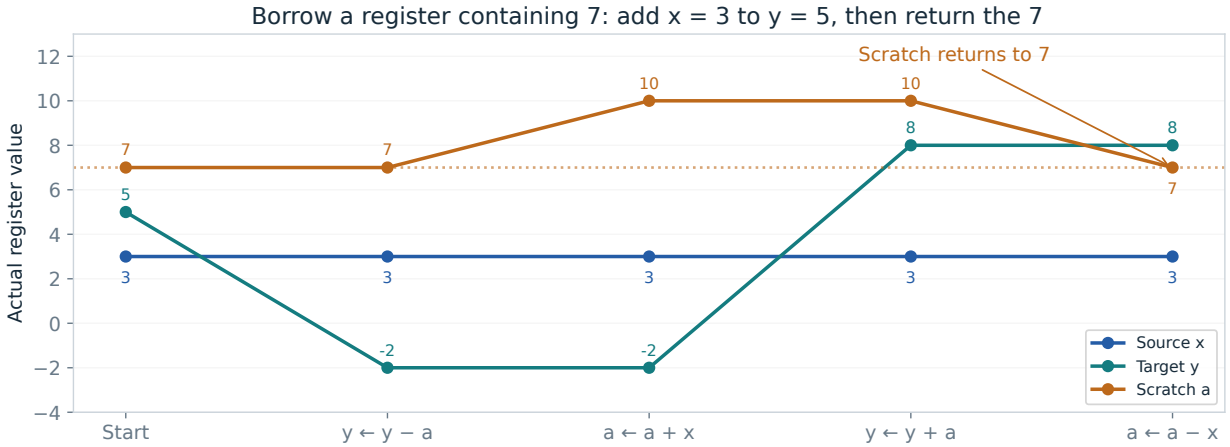


Figure E10. Unknown does not mean unusable. The plotted registers start at $x = 3, y = 5, a = 7$. The target ends at eight, while source and scratch return to three and seven. The symbolic identity above explains why this works for arbitrary initial values, not just this trace.

How the actual network isolates the right source. The original construction indexes values by triples $S, T \subseteq [h]$. Gathering through coordinates shared by the triples gives a coefficient depending on $|S \cap T|$. A side circuit cancels unwanted cross-terms. In the bit version the central coefficient is $|S \cap T|$ modulo two: intersection sizes zero and two already vanish, intersection one is canceled, and the self-intersection three leaves one. Thus the net target update is $y_S \leftarrow y_S + x_S$. For the complex version, the central coefficient is $(|S \cap T| - 1)/2$; side terms cancel the distinct intersection-zero and intersection-two cases. In both cases the scratch-restoration schedule removes the old auxiliary values.

Three bank updates yield

$$(x, y) \to (x, y + x) \to (-y, y + x) \to (-y, x).$$

This is a signed swap; over bits the minus sign disappears. The network alone has not yet accomplished a fast transform. It supplies an algebraic scaffold.

What a frame means. Let g be the logical array on one role. A frame is an invertible linear operator D on its address-indexed entries. The stored array is Dg. A pointwise gate, in contrast, combines different roles at the same address. For a gate adding two roles, a common frame gives

$$Dg_1 + Dg_2 = D(g_1 + g_2).$$

This is ordinary linearity, but it requires the *same* D on every incidence of that gate. With unrelated frames D_1, D_2, the desired identity does not follow.

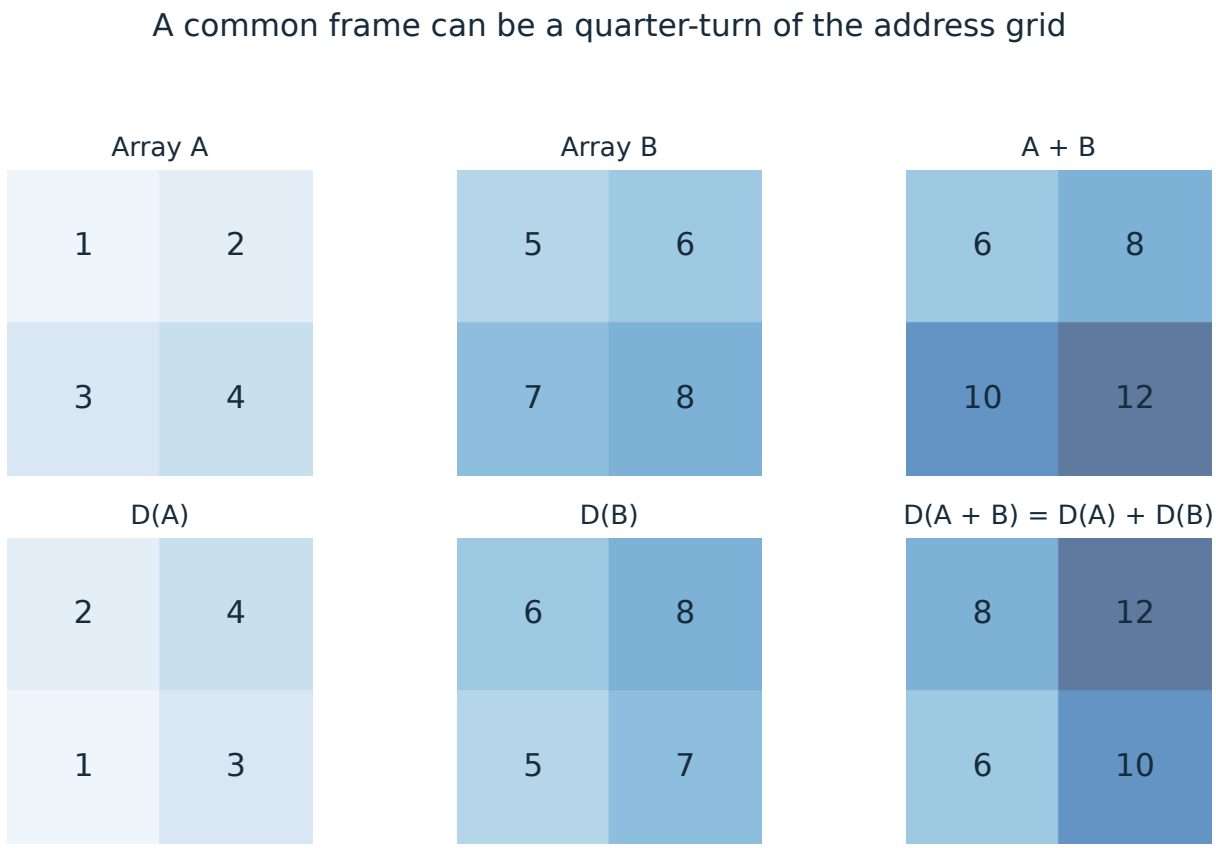


Figure E11. The diagram commutes because the frame is shared. Each panel is an actual two-by-two array. The bottom row rotates the address grid of the top row. Adding the bottom-left arrays gives the bottom-right array, exactly the rotation of the top-row sum.

Between vertices with frames D_u, D_v, the change is $D_v D_u^{-1}$. Following the invariant through the circuit gives the implemented operator

$$D_{\text{out}}\, \mathsf{S}_\Omega\, D_{\text{in}}^{-1},$$

where S_Ω is the scalar circuit applied pointwise. If the scalar circuit is a signed role permutation, each route sees only its endpoint frames and its scalar sign. Intermediate frames cancel algebraically, but their physical changes must still be paid for. Choosing them cheaply is the design problem.

Why the geometry matters. The paper attaches subspaces to vertices. Comparable neighboring subspaces give residual directions whose dimensions count smaller operations. Low rank means fewer independent directions to implement. A matrix with many nonzero entries may still have rank one; for example, $\left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ moves only one independent combination. Conversely, a sparse identity matrix has full rank. The accounting therefore depends on the actual residual operators, their bases, and endpoint contributions, not just the number of visible gates.

The bit circuit combines scalars in $\mathbb{F}_2$ while using rational subspace labels for address operators. The complex circuit combines Gaussian dyadic scalars while using binary subspace labels for phase operators. These are two different uses of a field; confusing scalar values with address labels obscures the construction.

What to carry forward. Restoration makes existing rows usable as workspace. Common frames

turn a scalar swap into an array operation. Residual ranks price the changes between those frames. All three properties are needed.

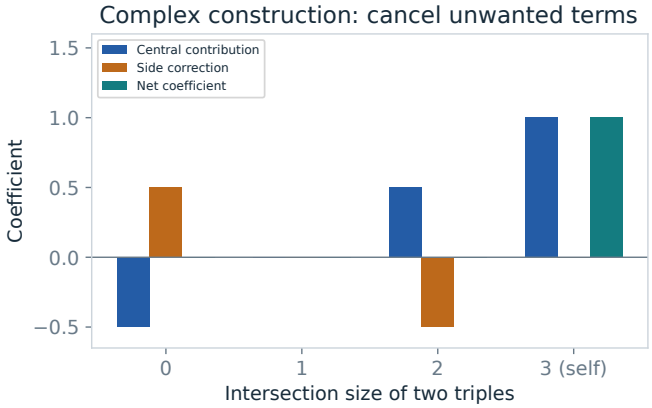
Original source anchors: `03-motifs.tex, lem:scalar-motifs; eq:common-frame-identity`.

How unwanted sources cancel out

Added explanation Read the coefficients, not just the circuit's arrows.

The source circuit temporarily combines many values. The challenge is to leave precisely the source meant for each target. The original construction labels a source and a target with sets of three elements. Their overlap can have size zero, one, two, or three. Two sets of size three overlap in three elements only when they are the same set.

For example, $\{1, 2, 3\}$ and $\{3, 4, 5\}$ overlap once, while $\{1, 2, 3\}$ and $\{2, 3, 4\}$ overlap twice. The central part of the complex construction gives coefficient $(q - 1)/2$ when the overlap is q. That is $-1/2, 0, 1/2, 1$ across the four possibilities. Side contributions cancel the two unwanted nonzero cases. The chart shows both contributions and their sum.



Bit construction: arithmetic modulo two

Overlap	Central	Side XOR	Net
0	0	0	0
1	1	1	0
2	0	0	0
3 (self)	1	0	1

Figure E12. Only the self-contribution survives. The complex coefficients are ordinary signed numbers. The bit construction instead uses modulo-two arithmetic, where $1+1 = 0$; its side contribution cancels the overlap-one case. Both leave net coefficients $0, 0, 0, 1$.

For a target labeled S, every distinct source T therefore has zero net coefficient, and source S has coefficient one. The target receives exactly its intended value even though many other values were used internally. This is stronger than testing one convenient input. It describes every source's contribution to the output.

Where workspace restoration enters. The cancellation table concerns the newly introduced source values. It does not by itself remove arbitrary values that were already in scratch registers. The separate restoration schedule cancels those initial values and returns them. The source proof needs both properties: correct new contribution and unchanged borrowed data. The register-trajectory figure makes the latter property visible for one small example.

What to carry forward. A crowded network can implement a simple map because unwanted coefficients cancel. The proof must account for every contribution, including the values initially occupying its workspace.

Original manuscript continues below.

3 Two finite networks with a rank saving

We construct two fixed linear networks that exchange two banks of values while restoring every auxiliary value. When the values are replaced by arrays, a common representation on all wires at a gate lets that gate act pointwise without disturbing the representation. We explain this invariant before constructing the subspace labels that make changes of representation cheap. The first network uses bit values and rational subspaces; the second uses Gaussian dyadic values and binary subspaces. The scalar domain specifies the values combined at a gate, while the subspace labels are used to construct operators on each wire's address-indexed array.

Put

$$h = 100, \qquad v = \binom{h}{3} = 161700, \qquad N = v^3 = 4227952113000000, \qquad m = h^3 = 1000000.$$

Let $\mathcal{T}$ be the set of three-element subsets of $[h]$. There are two data banks X_a, Y_a, indexed by $a = (a_1, a_2, a_3) \in \mathcal{T}^3$. A wire carries one scalar. A gate is a fixed linear map on a specified set of wires; untouched wires retain their values. The vertices on a wire are its source, the gates that touch it in time order, and its sink; its edges join consecutive vertices. We call the name of a wire its *role*, so that we can distinguish a fixed place in the network from the value currently carried there.

3.1 Scalar operations and restored auxiliary values

There are three stages. At stage j, fix the other two coordinates and operate on the v values in each bank obtained by varying coordinate j. Thus there are v^2 separate invocations at each stage. Each such invocation has its own auxiliary wires, distinct from all other invocations, including those in other stages. There are

$$I = 3v^2 = 78440670000$$

invocations. In a forward invocation, write x_T for the data used as controls and y_S for the data to be updated; $S, T \in \mathcal{T}$ are their varying jth coordinates. These are the logical source and target banks of the invocation. For each ordered pair (S, T) that is neighboring under the rule below, there is an auxiliary wire A_{ST}. There are also central wires C_i, $i \in [h]$, and in the second construction a wire C_*. We call all these auxiliary wires *scratch wires*; their initial values may be arbitrary.

The following maps specify the construction.

- In the bit construction, scalars lie in $\mathbb{F}_2$, and S, T are neighbors when $|S \cap T| = 1$. The copy map adds x_T to A_{ST} for each neighbor pair. The side injection adds $\sum_{T:\,|S\cap T|=1} A_{ST}$ to y_S. The gather adds $\sum_{T \ni i} x_T$ to C_i, and the scatter adds $\sum_{i \in S} C_i$ to y_S.
- In the complex construction, scalars lie in $\mathbb{Z}[i, 1/2]$, and neighbors are distinct triples with even intersection. Copy is unchanged. Side injection adds

$$-\sum_{T \text{ neighboring } S} \frac{|S \cap T| - 1}{2} A_{ST}$$

to y_S. Gather adds $\sum_{T \ni i} x_T$ to C_i and $\sum_T x_T$ to C_*. Scatter adds $(\sum_{i \in S} C_i - C_*)/2$ to y_S.

Each update leaves its control wires unchanged. The first two rows of the following schedule remove the effect of the old scratch values; the next four introduce the source values and add their effect to the target; the last two restore the scratch values. This cancellation of arbitrary initial scratch values is a linear instance of the restoration construction for transparent computation in Buhrman, Cleve, Koucký, Loff and Speelman [6, Section 3]. The forward schedule is

row	operation	gate grouping
0	subtract side injection	one gate per target
1	subtract scatter	one gate on targets and center
2	copy	one gate per source
3	gather	one gate on sources and center
4	scatter	one gate on targets and center
5	side injection	one gate per target
6	undo gather	one gate on sources and center
7	undo copy	one gate per source.

(2)

Lemma 5. *For arbitrary initial values of its auxiliary wires, a forward invocation changes y_S to $y_S + x_S$ and leaves every other value unchanged. Performing a forward invocation from X to Y at stage 1, the inverse forward schedule from Y to X at stage 2, and a forward invocation from X to Y at stage 3 sends*

$$(X_a, Y_a) \longmapsto (-Y_a, X_a)$$

and restores every auxiliary value. In the bit construction this is the permutation exchanging X_a and Y_a.

Proof. Write $\mathcal{V}$ for copy, $\mathcal{G}$ for gather, $\mathcal{J}$ for side injection and $\mathcal{R}$ for scatter. If the initial side and central vectors are a, c, the first two rows subtract $\mathcal{J}a + \mathcal{R}c$ from y. Rows 2 and 3 change them to $a + \mathcal{V}x, c + \mathcal{G}x$. Rows 4 and 5 therefore add $\mathcal{R}(c + \mathcal{G}x) + \mathcal{J}(a + \mathcal{V}x)$. Rows 6 and 7 restore c, a. The net target change is $(\mathcal{J}\mathcal{V} + \mathcal{R}\mathcal{G})x$, independently of a, c.

In the bit case, the central coefficient from x_T to y_S is $|S \cap T| \bmod 2$. The side contribution cancels exactly the intersection-one cases; distinct triples with intersection zero or two already have coefficient zero, and the self coefficient is one. In the complex case, the central coefficient is $(|S \cap T| - 1)/2$. The side cancels it for distinct triples with intersection zero or two; intersection one contributes zero, and the self coefficient is one. Thus both net maps are $y \leftarrow y + x$.

The inverse schedule with logical source Y and target X effects $X \leftarrow X - Y$. Starting from (x, y), the three stages give $(x, y + x)$, then $(-y, y + x)$, then $(-y, x)$. Each invocation restores its own auxiliary wires. □

A fixed triple has

$$z_{\mathrm{b}} = 3\binom{97}{2} = 13968, \qquad z_{\mathrm{c}} = \binom{97}{3} + 3 \cdot 97 = 147731$$

neighbors in the two constructions, respectively. The second count separates intersection zero and intersection two. Ordered pairs must be counted: there are vz side wires in each invocation, not $vz/2$. With $c_{\mathrm{b}} = 100$ and $c_{\mathrm{c}} = 101$ central wires per invocation, the total wire counts are

$$W_{\mathrm{b}} = 2N + I(vz_{\mathrm{b}} + c_{\mathrm{b}}) = 177176569091445000000, \tag{3}$$

$$W_{\mathrm{c}} = 2N + I(vz_{\mathrm{c}} + c_{\mathrm{c}}) = 1873807244643542670000. \tag{4}$$

All these numbers are fixed, independently of any eventual input length.

3.2 From scalar values to arrays

We record the representation principle before choosing the labels. Let R be the scalar ring of either network and let Ω be a finite address set common to all wires. Replace each scalar by an array in R^Ω, and apply each scalar gate pointwise at every address. Write S_Ω for the resulting network on the vector of wire arrays.

At each source, gate and sink v, assign an invertible R-linear operator D_v on R^Ω, called its *frame*. A frame acts on the values indexed by addresses within one wire's array and may mix them; a gate instead combines wire values at each fixed address. All incoming and outgoing incidences of one gate use the same D_v. On an edge from u to v, apply $D_v D_u^{-1}$ to that wire's array. Define the operators on all input and output roles by

$$D_{\mathrm{in}} = \bigoplus_w D_{\mathrm{source}(w)}, \qquad D_{\mathrm{out}} = \bigoplus_w D_{\mathrm{sink}(w)}.$$

For a supplied physical input f, define its logical interpretation to be $g = D_{\mathrm{in}}^{-1} f$; this definition requires no physical preprocessing. At a vertex v, the invariant is that the stored array equals D_v applied to the logical array there. It holds at the sources by definition. On an edge whose logical array is φ, the edge operator changes $D_u\varphi$ to $D_v\varphi$. At a gate with logical input arrays φ_j, R-linearity gives $\sum_j c_j D_v\varphi_j = D_v(\sum_j c_j\varphi_j)$ for each output combination, with $c_j \in R$. Thus the common frame commutes with the pointwise gate, and the invariant continues to the sinks. The implemented operator is therefore

$$D_{\mathrm{out}}\, \mathsf{S}_\Omega\, D_{\mathrm{in}}^{-1}. \tag{5}$$

In particular, if the scalar network is a signed role permutation that sends input role w to output role $\rho(w)$ with scalar sign ε_w, that route applies $\varepsilon_w D_{\mathrm{sink}(\rho(w))} D_{\mathrm{source}(w)}^{-1}$ to its input array. All roles, including the auxiliary roles, enter this identity. Once the scalar routing is fixed, the endpoint frames determine the array operator on each route; the intermediate frames can then be chosen to make each edge change inexpensive.

Figure 2 summarizes this invariant.

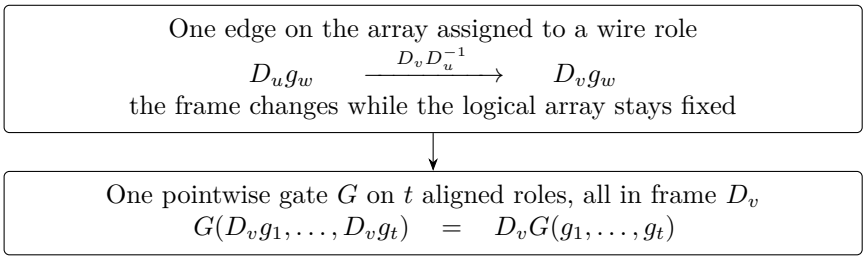


Figure 2: An edge changes the representation of one role's logical array g_w. At the next gate G, all t input arrays have the common frame D_v, so the pointwise linear gate commutes with that frame. The same D_v acts on each output component.

3.3 Subspaces attached to the gates

The scalar network already has the required signed exchange. We now choose its intermediate labels to make the changes of frame inexpensive. A label is a nondegenerate subspace of a fixed bilinear space. On every edge one label will contain the other. For such an edge, the *orthogonal residual* is the orthogonal complement of the smaller label inside the larger one; its dimension is

the absolute change in label dimension. The saving will come from keeping the total dimension lost on decreasing edges small relative to N.

For the bit network, let $F = \mathbb{Q}^h$ with bilinear form

$$\langle x, y\rangle = x^{\mathsf{T}}(I - J/9)y,$$

where J is the all-one matrix. This form has eigenvalues 1 and $1 - h/9 = -91/9$, so it is nondegenerate. For a triple T let t_T be its indicator vector. Then

$$\langle t_S, t_T\rangle = |S \cap T| - 1, \qquad \langle t_T, t_T\rangle = 2.$$

For the complex network, take instead $F = \mathbb{F}_2^h$ with the ordinary dot product; here $t_T \cdot t_T = 1$. Thus every triple line is nondegenerate, and neighboring triple lines are orthogonal, in the label field belonging to either construction. The scalar domain need not equal the label field. These degree-one intersection formulas specialize the construction described by Alon [2, Section 3, remark after Theorem 1.1] to the prime 2, with the ground-set size changed to $h = 100$.

The ambient label space is $\mathcal{F} = F^{\otimes 3}$ with its tensor bilinear form. It has dimension m and is nondegenerate. Write

$$u_a = t_{a_1} \otimes t_{a_2} \otimes t_{a_3}, \qquad U_a = \langle u_a\rangle.$$

The prescribed terminal labels are

role	source label	sink label
X_a	U_a	$\mathcal{F}$
Y_a	0	$U_a^{\perp}$
each scratch role	0	$\mathcal{F}$.

Write W for the wire count in the construction under consideration. The source dimensions sum to N and the sink dimensions to $Wm - N$. Thus the signed dimension changes will sum to $Wm - 2N$, because the contributions at gates cancel. If decreasing edges lose a total dimension L, the sum of absolute changes is $Wm - 2N + 2L$. We will obtain $L < N/2$. The rational interface below will require an additional N source ranks, so this inequality will leave a strict saving in both constructions.

At stage j, fix an invocation. Put

$$A = F^{\otimes(j-1)}, \qquad P = \langle t_{a_1} \otimes \cdots \otimes t_{a_{j-1}}\rangle, \qquad B = P^{\perp} \subset A.$$

An empty tensor product is the one-dimensional ground field; thus $B = 0$ at stage 1. Let $Q = \langle t_{a_{j+1}} \otimes \cdots \otimes t_{a_3}\rangle \subset F^{\otimes(3-j)}$, again using the empty tensor product at stage 3. Since P is nondegenerate, $A = B \perp P$. We prescribe the following transition for each data wire in the stage, where t is the indicator of that wire's jth triple:

	incoming label	outgoing label
X	$A \otimes \langle t\rangle \otimes Q$	$(A \otimes F) \otimes Q$
Y	$B \otimes \langle t\rangle \otimes Q$	$((B \otimes F) \perp (P \otimes t^{\perp})) \otimes Q$.

The outgoing Y label can also be written $(P \otimes \langle t\rangle)^{\perp}_{A\otimes F} \otimes Q$. These transitions expose one tensor factor at each stage. To see that they fit together, for $j < 3$ write $Q = \langle t_{a_{j+1}}\rangle \otimes Q'$. The next stage has

$$A' = A \otimes F, \qquad P' = P \otimes \langle t\rangle, \qquad B' = (P')^{\perp}_{A'} = (B \otimes F) \perp (P \otimes t^{\perp}).$$

Thus the prescribed outgoing labels are $A' \otimes \langle t_{a_{j+1}} \rangle \otimes Q'$ on X and $B' \otimes \langle t_{a_{j+1}} \rangle \otimes Q'$ on Y, exactly the next incoming labels. At stage 1 they start at U_a on X_a and zero on Y_a; at stage 3 they end at $\mathcal{F}$ on X_a and $U_a^\perp$ on Y_a, as prescribed at the terminals.

The gate labels are organized by the decomposition

$$A \otimes F = (B \otimes F) \perp (P \otimes F),$$

with the common summand $B \otimes F$ and the remaining summand $P \otimes F$ of dimension h. Suppress the one-dimensional factor Q for the moment. On a side wire joining neighboring physical triples with indicators t_X, t_Y, the common component grows to $B \otimes F$, while the remaining component passes through

$$P \otimes \langle t_X \rangle \;\subset\; P \otimes t_Y^\perp \;\subset\; P \otimes F.$$

The first inclusion is possible because $t_X \perp t_Y$. For the central gates we assign $A \otimes F$ on the physical X bank and $B \otimes F$ on the physical Y bank. A central wire returns once from the former label to the latter, losing exactly $P \otimes F$. The gate order below confines all decreases to this return, while the other edges realize the prescribed stage transitions.

Number the successive times in an invocation by $r = 0, \ldots, 7$. At stages 1 and 3, time r uses row r of the scalar schedule (2). At stage 2 it uses the inverse of row $7 - r$, with the logical banks exchanged. In either case the gates touch the physical wires in the order

$$Y\text{-side}, \quad Y\text{-center}, \quad X\text{-side}, \quad X\text{-center}, \quad Y\text{-center}, \quad Y\text{-side}, \quad X\text{-center}, \quad X\text{-side}.$$

A side wire still joins neighboring physical triples in stage 2 because the neighbor relation is symmetric. The following table assigns the common gate labels. It suppresses the tensor factor Q; for a gate indexed by a physical bank triple, t denotes that triple's indicator.

time	physical wires	common gate label
0	Y, its side wires	$B \otimes \langle t \rangle$
1	Y, all center wires	$B \otimes F$
2	X, its side wires	$(B \otimes F) \perp (P \otimes \langle t \rangle)$
3	X, all center wires	$A \otimes F$
4	Y, all center wires	$B \otimes F$
5	Y, its side wires	$(B \otimes F) \perp (P \otimes t^\perp)$
6	X, all center wires	$A \otimes F$
7	X, its side wires	$A \otimes F$.

(6)

All labels are tensored by Q and viewed as subspaces of $\mathcal{F}$. Every displayed summand is nondegenerate.

All scratch sources have label zero and all scratch sinks have label $\mathcal{F}$, as prescribed above. The next lemma verifies the stage boundaries and all intervening edge changes, including these scratch edges.

Lemma 6. *Every edge joins comparable nondegenerate labels. The only decreasing edges are the central-wire edges from time 3 to time 4; their dimension loss is h. Consequently, if c is the number of central wires per invocation and W the total number of wires, then*

$$L = Ich, \qquad \sum_e \left|\dim U_{\mathrm{head}(e)} - \dim U_{\mathrm{tail}(e)}\right| = Wm - 2N + 2L. \tag{7}$$

For the binary label space, every nonzero orthogonal residual of an edge has an orthonormal basis.

Proof. Write $a = \dim A = h^{j-1}$ and $f = h^{3-j}$. In the table below, in denotes the preceding vertex on that data wire: its source at stage 1, and its last gate in the preceding stage otherwise. The interstage calculation above gives its label. The table lists the orthogonal residual and its dimension for every edge except the final data edges to their sinks, whose labels agree and whose residuals are zero. Tensor each residual by Q, except in the last row. The symbols t_X, t_Y are the indicators of the neighboring physical triples joined by a side wire.

wire and edge	orthogonal residual	dimension
$X:$ in $\to 2$	$B \otimes t_X^\perp$	$(a-1)(h-1)$
$X:\ 2 \to 3$	$P \otimes t_X^\perp$	$h-1$
$X:\ 3 \to 6,\ 6 \to 7$	0	0
$Y:$ in $\to 0$	0	0
$Y:\ 0 \to 1$	$B \otimes t_Y^\perp$	$(a-1)(h-1)$
$Y:\ 1 \to 4$	0	0
$Y:\ 4 \to 5$	$P \otimes t_Y^\perp$	$h-1$
center : source $\to 1$	$B \otimes F$	$(a-1)h$
center : $1 \to 3$	$P \otimes F$	h
center : $3 \to 4$	$P \otimes F$	h
center : $4 \to 6$	$P \otimes F$	h
side : source $\to 0$	$B \otimes \langle t_Y \rangle$	$a-1$
side : $0 \to 2$	$(B \otimes t_Y^\perp) \perp (P \otimes \langle t_X \rangle)$	$(a-1)(h-1)+1$
side : $2 \to 5$	$P \otimes \langle t_X, t_Y \rangle^\perp$	$h-2$
side : $5 \to 7$	$P \otimes \langle t_Y \rangle$	1
scratch : last $\to$ sink	$(A \otimes F) \otimes Q^\perp$	$ah(f-1)$.

The last gate on a central wire is at time 6, and the last gate on a side wire is at time 7. Both have label $(A \otimes F) \otimes Q$. Enlarging it to $\mathcal{F}$ at the sink gives the last residual in the table, with $Q^\perp$ taken in the entire future tensor space. The side edge $2 \to 5$ uses the neighbor relation essentially. Since $t_X \perp t_Y$, we have $\langle t_X \rangle \subset t_Y^\perp$. Hence its two labels satisfy

$$(B \otimes F) \perp (P \otimes \langle t_X \rangle) \ \subset\ (B \otimes F) \perp (P \otimes t_Y^\perp).$$

The complement of $\langle t_X \rangle$ inside $t_Y^\perp$ is $\langle t_X, t_Y \rangle^\perp$, of dimension $h-2$; tensoring by P and then by Q gives the residual in the table. The other entries follow directly by splitting $A = B \perp P$ and $F = \langle t \rangle \perp t^\perp$. Label dimension is nondecreasing along every edge except center $3 \to 4$, which removes $P \otimes F \otimes Q$. The table includes the interstage edges through in, and the initial and final data labels match their terminals, so no other dimension changes occur.

The signed sum of dimension changes telescopes at every gate: each touched wire has one incoming and one outgoing incidence with the same gate label. The sum of source dimensions is N, while the sum of sink dimensions is $Wm - N$. The signed increase is therefore $Wm - 2N$. There are Ic decreasing edges, each of loss h; replacing signed changes by absolute changes adds $2L$.

Now work over $\mathbb{F}_2$. A nondegenerate symmetric bilinear space is nonalternating precisely when it contains a vector of norm one. The lines P, Q have such vectors. If $B \neq 0$, a coordinate unit outside the support of the earlier tensor vector belongs to B: that support has size $3^{j-1} < h^{j-1}$ when $j > 1$. The same argument gives a unit vector in $Q^\perp$ when it is nonzero. A triple complement, and the complement of two neighboring triple lines, contain coordinate units outside supports of sizes at most three and six, respectively. Full tensor spaces contain coordinate units as well. In each nonzero tensor summand, tensoring these chosen norm-one vectors gives a norm-one vector; placing it in that orthogonal summand gives one in the whole residual. Each residual is nondegenerate by the decompositions above. Thus every nonzero residual is nonalternating.

For completeness, every nondegenerate nonalternating binary symmetric space has an orthonormal basis. Split off unit lines until the remaining space is alternating. A nonzero nondegenerate alternating space splits into planes with bases a, b satisfying $a \cdot a = b \cdot b = 0$, $a \cdot b = 1$; this follows by choosing a nonzero a, choosing b with $a \cdot b = 1$, and taking the orthogonal complement of their nondegenerate plane. Retain one unit vector w from the lines already split off. A plane orthogonal to w can be absorbed into that line: the three vectors

$$w + a, \qquad w + b, \qquad w + a + b$$

are independent, pairwise orthogonal, and have norm one. Use one of these new unit vectors in place of w for the next plane; the remaining planes are still orthogonal to it. Repeating this operation absorbs every alternating plane. □

The loss ratios have the exact values

$$\frac{L_{\mathrm{b}}}{N} = \frac{100}{539}, \qquad \frac{L_{\mathrm{c}}}{N} = \frac{101}{539}.$$

In particular, both are smaller than $1/2$. We now turn the dimension count into the two interfaces needed later.

3.4 The rational matrix interface

For a nondegenerate rational subspace $U \subset \mathcal{F}$, write P_U for the projection onto U along $U^{\perp}$. Assign P_U to each gate or sink with label U. Assign the source matrix $-P_{U_a}$ to X_a and zero to every other source. These are fixed rational $m \times m$ matrices; write M_v for the matrix at vertex v. Define $\rho(X_a) = Y_a$, $\rho(Y_a) = X_a$, and let ρ fix every auxiliary wire.

In the tape application, two address chunks are written as vectors H, D of m fields. For a matrix M, the frame Φ_M moves the array entry at (H, D) to $(H + MD, D)$, leaving the other address fields unchanged. The fields use a radix coprime to the fixed denominators, and addition is modulo their common range. Hence Φ_M is an invertible $\mathbb{F}_2$-linear permutation of array entries, and $\Phi_{M'}\Phi_M = \Phi_{M'+M}$. Using $D_v = \Phi_{M_v}$ in (5), the endpoint matrix difference on each routed role is exactly the shear applied to that role. The following identity makes it the same shear $H \leftarrow H + D$ on every role; Section 4 turns that shear into a field interchange and implements an edge change with one smaller interchange per unit of its rational rank.

Proposition 7. *The bit network has W_{b} wires, uses only pointwise XOR gates, and sends its input vector to the permutation ρ of that vector. Its rational matrices are common to all incidences of a gate and obey*

$$M_{\mathrm{sink}(\rho(w))} - M_{\mathrm{source}(w)} = I_m \quad \textit{for every input role } w.$$

The sum of the rational ranks of all edge differences is

$$s_{\mathrm{b}} = \sum_e \mathrm{rank}_{\mathbb{Q}}(M_{\mathrm{head}(e)} - M_{\mathrm{tail}(e)}) \tag{8}$$

$$= W_{\mathrm{b}}m - N + 2L_{\mathrm{b}} = 177176569088785861287000000 < W_{\mathrm{b}}m. \tag{9}$$

Proof. Lemma 5 gives the scalar permutation, including all auxiliary roles. For $U \subset V$ nondegenerate, $P_V - P_U$ is the identity on $V \cap U^{\perp}$ and zero on its orthogonal complement. Its rank is $\dim V - \dim U$; reversing the edge negates the matrix without changing its rank. The only exceptions are the N negative source projections. At stage 1, $B = 0$, and the first gate on X_a has label

exactly U_a. Its first edge matrix is therefore $2P_{U_a}$, of rational rank one, where the corresponding dimension change was zero. Lemma 6 consequently gives the stated rank sum. Its deficit from $W_{\mathrm{b}}m$ is $N - 2L_{\mathrm{b}} > 0$.

For an input X_a the endpoint difference is $P_{U_a^{\perp}} - (-P_{U_a}) = I_m$. Every other role has zero source matrix and identity sink matrix. This proves the interface. □

3.5 The binary phase interface

Use the complex scalar network and binary labels. For a binary vector x, let $\mathrm{wt}(x)$ denote its integer Hamming weight, and write $[b] \in \{0, 1\}$ for the integer representative of $b \in \mathbb{F}_2$. For a label U, let

$$q_U(x) = \mathrm{wt}(P_U x) \pmod 4.$$

The projection here is over $\mathbb{F}_2$. Let

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad C = H \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} H = aI + bX, \qquad a = \frac{1+i}{2}, \quad b = \frac{1-i}{2},$$

where X interchanges the two coordinates. On functions of $x \in \mathbb{F}_2^m$ define

$$\mathcal{C}_U = H^{\otimes m} \operatorname{diag}_x(i^{q_U(x)}) H^{\otimes m}.$$

The use of H in this definition is an identity of matrices. Each entry of $\mathcal{C}_U$ and its inverse is a Gaussian integer divided by 2^m, so both preserve arrays over $\mathbb{Z}[i, 1/2]$. The kernels implemented below have Gaussian dyadic coefficients.

To apply the scalar network to these functions, replace the scalar on each wire w by an array $f_w : \mathbb{F}_2^m \to \mathbb{Z}[i, 1/2]$. At each address, a gate applies its scalar linear map to the values on the wires it touches. The frame in (5) is $D_v = \mathcal{C}_U$ at a vertex with label U. On an edge from label U to label V, apply $\mathcal{C}_V \mathcal{C}_U^{-1}$ to that wire's array. All frames are diagonalized by the same matrix $H^{\otimes m}$. For an integer $k \geq 1$, an address for k columns is $(x^{(1)}, \ldots, x^{(k)}) \in (\mathbb{F}_2^m)^k$; every frame and edge operator is then the tensor product of its one-column version over these k coordinates. Each scalar gate is still applied once at each joint address, to the vector of values on the roles it touches.

Proposition 8. *For each edge of the complex network, write U, V for its tail and head labels. Its phase-frame difference $\mathcal{C}_V \mathcal{C}_U^{-1}$ is a product of one translation kernel $aI + bX_v$, or its inverse, per vector in an orthonormal basis of the edge residual. Here $(X_v f)(x) = f(x + v)$. The sum of these basis lengths is*

$$s_{\mathrm{c}} = W_{\mathrm{c}}m - 2N + 2L_{\mathrm{c}} = 187380724463667126730800000 0 < W_{\mathrm{c}}m.$$

After source and sink diagonal corrections, and undoing the signed bank exchange, telescoping these phase frames gives $C^{\otimes m}$ on every role, including auxiliary roles. The same assertion holds on any positive number of columns by taking tensor products over columns.

Proof. For binary vectors z, w,

$$\mathrm{wt}(z + w) = \mathrm{wt}(z) + \mathrm{wt}(w) - 2|\operatorname{supp}(z) \cap \operatorname{supp}(w)|.$$

If $z \cdot w = 0$, the intersection cardinality is even, so weight is additive modulo four. If $V = U \perp E$ and $\mathcal{B}$ is an orthonormal basis of E, it follows that

$$q_V(x) - q_U(x) = \sum_{v \in \mathcal{B}} \mathrm{wt}(v)[v \cdot x] \pmod 4.$$

For a decreasing edge the right side is negated. Each v has norm one, so $\mathrm{wt}(v)$ is odd. Its coefficient in the phase, including the sign for the edge direction, is therefore some $\varepsilon \in \{1, -1\}$ modulo four. The function $i^{\varepsilon[v\cdot x]}$ equals

$$\frac{1+i^{\varepsilon}}{2} + \frac{1-i^{\varepsilon}}{2}(-1)^{v\cdot x}.$$

The identity $H\,\mathrm{diag}(1,-1)H = X$, tensored over the coordinates in $\mathrm{supp}(v)$, sends the diagonal character $(-1)^{v\cdot x}$ to translation X_v. The displayed phase therefore becomes $aI + bX_v$ or its inverse. The nonzero residuals have such bases by Lemma 6; for a zero residual use the empty basis. The same lemma gives the count s_{c}. The deficit is $2N - 2L_{\mathrm{c}} > 0$.

For an edge e, write $\mathcal{B}_e$ for its residual basis and $K_{e,v}$ for the forward or inverse translation kernel associated with $v \in \mathcal{B}_e$. Taking tensor products preserves their product, so on k columns

$$\left(\mathcal{C}_V \mathcal{C}_U^{-1}\right)^{\otimes k} = \prod_{v\in\mathcal{B}_e} K_{e,v}^{\otimes k}.$$

Thus each residual vector contributes one factor acting on all k columns, and there are s_{c} such factors over the network.

Let ρ denote the underlying permutation of the complex roles: it exchanges X_a, Y_a and fixes each scratch role. By Lemma 5, the pointwise scalar network sends an input array at X_a to Y_a with sign $+1$, an input at Y_a to X_a with sign -1, and each scratch input to its own output with sign $+1$. Applying (5) with $D_v = \mathcal{C}_{U_v}$, where U_v is the label at vertex v, shows that an input role w reaches output role $\rho(w)$ with that sign and with operator $\mathcal{C}_{U_{\mathrm{sink}(\rho(w))}}\mathcal{C}^{-1}_{U_{\mathrm{source}(w)}}$. For k columns, use $D_v = \mathcal{C}_{U_v}^{\otimes k}$ on the joint address set in the same identity. The route operator is then the kth tensor power of the displayed address operator, multiplied by its scalar route sign once. The zero label has frame I, while the full label has frame $C^{\otimes m}$ because $q_{\mathcal{F}}(x) = \mathrm{wt}(x)$. Every pair except $X_a \to Y_a$ has these zero and full labels. For the remaining pair, write $u = u_a$. Its binary norm is one and its weight is $3^3 = 27$. Orthogonal weight additivity yields

$$\begin{aligned} q_{U_a^{\perp}}(x) - q_{U_a}(x) &= \mathrm{wt}(x) - 2\,\mathrm{wt}(u)[u\cdot x] \\ &= \mathrm{wt}(x) + 2[u\cdot x] \pmod 4. \end{aligned}$$

Since $2[u\cdot x] \equiv 2\sum_{j\in\mathrm{supp}(u)} x_j \pmod 4$, the operator on $X_a \to Y_a$ is the tensor of C^{-1} on the 27 coordinates of $\mathrm{supp}(u)$ and C elsewhere. Put $Z = \mathrm{diag}(1,-1)$. Direct multiplication gives

$$C = iZC^{-1}Z, \qquad C^{-1} = -iZCZ.$$

For k columns, let $Z_{a,k}$ be the tensor product of Z on those 27 coordinates in every column and the identity elsewhere. Apply $Z_{a,k}$ before the network at X_a, and apply $i^{27k}Z_{a,k}$ after the network at Y_a; the latter scalar is applied once to the whole role. There are $27k$ inverse factors, so the identity $C = iZC^{-1}Z$ turns each of them into C. The operator on this pair is therefore $C^{\otimes mk}$ as well. Only the physical X outputs still carry the minus sign of the scalar exchange. Negate those outputs and send output role $\rho(w)$ back to role w. Every role, including every scratch role, now has the required forward tensor product. □

3.6 Explicit rational bounds for the two recurrences

The relative rank deficits are

$$\eta_{\mathrm{b}} = \frac{W_{\mathrm{b}}m - s_{\mathrm{b}}}{W_{\mathrm{b}}m} = \frac{339}{22587335000000}, \qquad \eta_{\mathrm{c}} = \frac{W_{\mathrm{c}}m - s_{\mathrm{c}}}{W_{\mathrm{c}}m} = \frac{73}{19906842167500}.$$

For both recurrences we may use the fixed rational exponent

$$\tau = \sigma = 1 - 2^{-50}. \tag{10}$$

Both exact deficits displayed above are greater than $20 \cdot 2^{-50}$. Since $m < 2^{20}$ and $\log 2 < 1$, we have $\log m < 20$, whence

$$m^{1-2^{-50}} = m \exp(-2^{-50} \log m) > m(1 - 20 \cdot 2^{-50}) > m(1 - \eta)$$

for either $\eta = \eta_{\mathrm{b}}$ or $\eta = \eta_{\mathrm{c}}$. Consequently

$$\frac{s_{\mathrm{b}}}{W_{\mathrm{b}}} < m^{\tau}, \qquad \frac{s_{\mathrm{c}}}{W_{\mathrm{c}}} < m^{\sigma}, \qquad 0 < \tau = \sigma < 1.$$

Here σ is a rational upper bound for the logarithmic exponent of the complex recurrence.

Why more calls can still mean less work

Added explanation Count the size of each job as well as the jobs.

Suppose a width-doubling step makes three smaller calls, each on half the data volume. Three calls sounds worse than two, but it does not mean three full-size jobs. Their total visited volume is $3 \cdot (V/2) = 1.5V$. At the next depth there are nine calls on volume $V/4$ each, totaling $2.25V$. The circle areas in the tree encode those shrinking volumes.

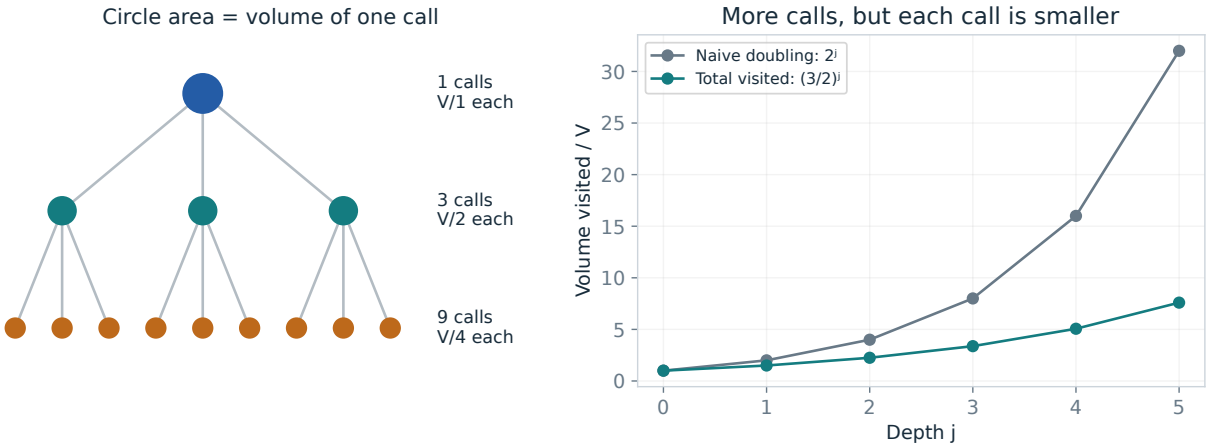


Figure E13. The growth factor is calls times relative volume. This toy tree uses $m = 2$, $W = 2$, $s = 3$. Its first three levels visit V, $1.5V$, and $2.25V$. The right chart compares this with doubling at every level. Calls can run sequentially and reuse storage; the total visited volume describes work, not peak memory.

After j levels, there are 3^j calls and each has volume $V/2^j$. Multiplying gives $V(3/2)^j$. Meanwhile the width factor is 2^j. To express the work as a power of width, solve $2^q = 3/2$. This gives $q = \log_2(3/2)$, about 0.585, below one. The corresponding toy cost grows more slowly than the width itself.

What the real proof must add. The tree charges only the recursive contributions shown. A real algorithm also scans, copies, arranges, and restores records at its nodes. It must bound that overhead, handle widths that do not fit the tree exactly, and keep one fixed machine description. The source reserves strict slack in its recurrence inequality for these tasks. Its numerical saving is much smaller than the one drawn here.

A diagnostic question. What if every one of the three children actually processed the whole volume V? The multiplier would be three, not $3/2$, and this argument would no longer beat a factor of two. This is why the paper's division by W is an implementation obligation rather than a cosmetic normalization.

What to carry forward. A recurrence counts weighted work. The weight is the data volume of each child, including the effects of the required layout and workspace.

How a finite rank saving becomes an exponent

Added explanation Read before address interchange.

Let $C(V, f)$ be a recursive cost for volume V and width parameter f. One schematic recurrence underlying the argument is

$$C(V, mf) \le s\, C(V/W, f) + O(V).$$

The full paper includes layout and descriptor conditions; this display isolates the essential scaling. The parent invokes s smaller operations, each acting on a role stream of volume V/W. Divide by V and write $a(f)$ for a uniform bound on cost per unit volume. Then

$$a(mf) \le (s/W)a(f) + O(1).$$

If $a(f)$ behaves like f^q, its recursive contribution scales by m^q. The critical comparison is therefore

$$\frac{s}{W} < m^q < m, \qquad q < 1.$$

The final proof uses strict inequalities so that overhead can be absorbed.

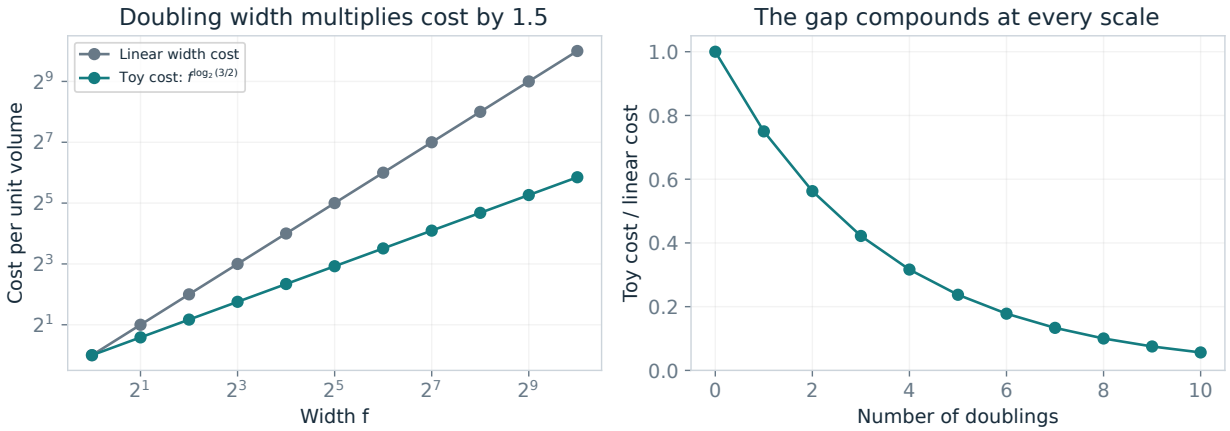


Figure E14. Small children can outweigh a large call count. The plot is an illustrative recurrence with $m = 2$, $W = 2$, $s = 3$; it is not the paper's network and is not a runtime benchmark. Its recursive exponent is $\log_2(3/2) \approx 0.585$.

Why $s < Wm$ is decisive. A width that grows by m would normally cost a factor m. The recursive contribution grows by s/W instead. If $s = Wm$, this comparison gives no strict power saving. If $s < Wm$, an exponent below one can sit strictly above $\log_m(s/W)$. The source constructs two such inequalities, one for address interchange and one for numerical layers. Their rational upper exponents are called τ and σ.

A tiny deficit still gives a positive gap. Define $\eta = (Wm - s)/(Wm)$. Algebra gives

$$\log_m(s/W) = 1 + \frac{\log(1-\eta)}{\log m}.$$

For $0 < \eta < 1$, the logarithm in the numerator is negative. When η is small, the gap is approximately $\eta/\log m$. Approximation helps intuition; the source uses exact rational deficits and strict exponential inequalities to certify a rational exponent. Its original choice is $\tau = \sigma = 1 - 2^{-50}$, not the visibly large saving in the toy plot.

Why a shear is easier to build than a swap. An address shear updates one block by a linear combination of another: $(H, D) \mapsto (H + AD, D)$. The paper factors a rank-a rational matrix A into two invertible triangular factors and a partial permutation with a pivots. Triangular updates follow tape order cheaply. Each pivot needs one smaller interchange. Thus the expensive-call count is the rank a. A suitable fixed odd prime makes the required rational coefficients and inverses valid modulo its powers. Denominators and invertibility cannot be ignored when transferring a rational identity to modular addresses.

The unglamorous conditions are essential. Real field widths need not be powers of m. Row counts may not divide by W. Some layouts contain spectators between active fields. The following source section removes these restrictions and pays for padding and cleanup. The toy recurrence alone is not a proof of those implementation contracts.

What to carry forward. The finite certificate supplies a strict deficit. The tape implementation must preserve its V/W normalization for that deficit to become an exponent saving.

Original source anchors: `03-motifs.tex, eq:explicit-motif-exponents; 04-swap.tex`.

Original manuscript continues below.

4 Faster interchange of address chunks

We use the bit network to interchange two equal address chunks on tape. An edge matrix of rank a will require a smaller interchanges. Because the sum of these ranks is less than the number of wires times m, the resulting recurrence saves a power of the chunk width.

4.1 A matrix decomposition adapted to tape order

Lemma 4 updates a later field using earlier fields in linear time. Lower triangular changes of coordinates can be performed entirely in this direction. The required factorization is the Bruhat factorization for arbitrary matrices, with coordinate order reversed; see Grigor'ev [12, Appendix A.2, Proposition 14(c)]. We give its elementary elimination proof in this order.

Lemma 9 (Lower triangular factorization)**.** *Every rational $m \times m$ matrix A of rank a has a factorization*

$$A = E_1 \Pi E_2,$$

where E_1, E_2 are invertible lower triangular rational matrices and Π has exactly a entries equal to one, with at most one in each row and column, and all remaining entries zero.

Proof. If $A = 0$, take $E_1 = E_2 = I$ and $\Pi = 0$. Otherwise choose the topmost nonzero row and its rightmost nonzero entry. Use that pivot column to clear entries to its left by right multiplication by lower triangular elementary matrices. There are no nonzero entries to its right in the pivot row. Clear entries below the pivot by left multiplication by lower triangular elementary matrices, and scale the pivot to one. All rows above the chosen row were zero in the still active columns.

Delete the pivot row and column from the active index sets, retaining their original orders, and repeat. Elementary operations on the active indices are still lower triangular in the original order. They do not change any earlier pivot row or column, whose other entries have already been cleared. Each step isolates a rank-one block and leaves the remaining rank in the active submatrix. Thus the final partial permutation matrix has exactly a ones. If $LAR = \Pi$ is the resulting identity, then $E_1 = L^{-1}$ and $E_2 = R^{-1}$ have the required form. □

Lemma 10 (The cost of a rational matrix shear)**.** *Let $\mathcal{A}$ be a fixed finite collection of rational $m \times m$ matrices. There is a fixed odd prime q such that the following holds. Let $H = (H_1, \dots, H_m)$ and $D = (D_1, \dots, D_m)$ be two ordered groups of fields, every field ranging over $[q^b]$, with every H_i before every D_j. For $A \in \mathcal{A}$ of rational rank a, the permutation*

$$(H, D) \longmapsto (H + AD, D) \quad \text{over } \mathbb{Z}/q^b\mathbb{Z}$$

can be performed with exactly a interchanges of pairs of b-digit fields and $O(V)$ further work. Every interchange is between an H field and a D field. Spectators between any of the formal fields are allowed.

Proof. Choose the factorizations of Lemma 9 for all matrices in $\mathcal{A}$ once and for all. Include their factors and inverses in a finite rational table. Choose an odd prime avoiding every denominator in the table and every numerator of a diagonal entry of an invertible factor. For example, a prime exceeding the absolute values of all its nonzero numerators and denominators suffices. Rational elimination and trial division construct this finite table and a suitable prime in a finite computation independent of b. They may therefore be compiled into one fixed machine description. All these rational identities then specialize to $\mathbb{Z}/q^b\mathbb{Z}$, and the triangular factors remain invertible there.

A lower triangular transformation of H, or of D, costs $O(V)$: update coordinates in descending physical order. At coordinate i, multiply by its diagonal unit and add a fixed linear combination of earlier coordinates. These earlier coordinates still have their original values. Apply Lemma 4; the number of coordinates is the fixed number m. Inverses are also lower triangular.

For $A = E_1 \Pi E_2$, first transform D by E_2 and H by E_1^{-1}, then add ΠD to H, and finally transform H by E_1 and D by E_2^{-1}. Each one in position (i, j) of Π calls for $H_i \leftarrow H_i + D_j$. Implement it by

$$D_j \leftarrow D_j + H_i, \qquad (H_i, D_j) \leftarrow (D_j, H_i), \qquad D_j \leftarrow H_i - D_j.$$

The first and last updates target a later field controlled by an earlier field. On an initial pair (h, d) this sequence yields $(h + d, d)$. It uses just one interchange. There are a ones, which proves the claim. In particular, the number of recursive calls is the rational rank a; temporary tapes used for the other operations do not multiply that count. □

4.2 Transferring a finite circuit to address permutations

Fix integers $m, W \geq 2$. The circuit has W wires, including its scratch wires, with separate input and output terminals. Its fixed, pointwise bit gates induce a permutation ρ of the W scalar inputs at completion: the input value at role w appears at output role $\rho(w)$. Assign a rational $m \times m$

matrix to each terminal and to each gate, the latter assignment being common to every wire at that gate. A wire segment between consecutive assigned vertices is called an edge. For an edge e put

$$A_e = M_{\mathrm{head}(e)} - M_{\mathrm{tail}(e)}, \qquad s = \sum_e \operatorname{rank}_{\mathbb{Q}} A_e.$$

The required conditions are

$$M_{\mathrm{out}(\rho(w))} - M_{\mathrm{in}(w)} = I_m \quad (1 \le w \le W), \qquad s < Wm. \tag{11}$$

All these data are finite and fixed. Proposition 7 supplies these conditions with $W = W_{\mathrm{b}}$ and $s = s_{\mathrm{b}}$. Its negative source projection is essential to the endpoint identity and has already been included in the rank sum.

Proposition 11 (Power-width interchange with rows)**.** *Assume the fixed circuit data satisfy* (11)*, and let q be a prime as in Lemma 10 for its edge matrices, also avoiding the denominators of every assigned terminal and gate matrix M. Choose a rational number $\tau \in (0,1)$ with $s/W < m^{\tau}$. For every $e = m^k$, $k \ge 0$, consider two disjoint contiguous address chunks of e radix-q digits each. Suppose a row field preceding both chunks has length divisible by W^k. There may be additional fields before and after the row field, between the chunks, and after them. The chunks can be interchanged in $O(Ve^{\tau})$ time, preserving every other address field, on a fixed number of finite-alphabet tapes. The constant is independent of all field lengths and of the data.*

Proof. For $k = 0$ apply the single-digit operation of Lemma 3. For $k > 0$, write each chunk as m consecutive $b = e/m$ digit fields, giving vectors H, D over $\mathbb{Z}/q^b\mathbb{Z}$. We first construct the shear $H \leftarrow H + D$.

Split whole rows cyclically into W role streams within each prefix preceding the row field. Write the original row number as $r = Wg + (w-1)$ with $1 \le w \le W$; stream w has row number g. These streams have exactly the same remaining shape, including the full ranges of H and D. Each has logical volume V/W, and its row count is divisible by W^{k-1}.

For a rational matrix M, let Φ_M act on an array by moving its entry at address (H, D) to $(H + MD, D)$, with every spectator unchanged. Thus Φ_M is the array permutation induced by that address map. Its inverse is Φ_{-M} and $\Phi_{M'}\Phi_M = \Phi_{M'+M}$. On every circuit edge apply Φ_{A_e} to that role stream; implement it by Lemma 10, recursively interchanging each pivot pair of b-digit fields. Execute each circuit gate pointwise at matching addresses of its role streams.

The row split supplies the common address set used in the frame identity (5). Here the frame at a vertex with matrix M is the address permutation Φ_M, and every edge changes it by $\Phi_{M'-M}$. The same invariant proof applies because a common address permutation commutes with every pointwise bit gate. For the physical input arrays f_w, the logical arrays are $g_w = \Phi_{-M_{\mathrm{in}(w)}} f_w$; this only specifies their interpretation, with no initial tape operation. The logical circuit routes g_w to role $\rho(w)$. Thus its physical output satisfies

$$f'_{\rho(w)} = \Phi_{M_{\mathrm{out}(\rho(w))}} \Phi_{-M_{\mathrm{in}(w)}} f_w = \Phi_{I_m} f_w,$$

by (11). Merge by returning row g of output role $\rho(w)$ to the original row $Wg + (w-1)$ within the same preceding prefix. This undoes ρ and restores row order. The result is $H \leftarrow H + D$ on every original row, including rows assigned to scratch roles, whose initial values were arbitrary.

To interchange the two full chunks, perform

$$D \leftarrow D - H, \qquad H \leftarrow H + D, \qquad D \leftarrow H - D,$$

componentwise. Only the middle shear uses the circuit. The other two updates target the later group and have linear cost. The transformation sends (H, D) to (D, H).

The number of recursive calls is the sum of the edge ranks, exactly s. Each acts on one role stream of volume V/W, retaining all other fields as spectators. Splitting, merging, gates, and triangular operations have total cost $O(V)$ because their number is fixed. Below we verify that scheduling the recursive calls on fixed tapes also costs $O(V)$ at this node. Thus if F_k is a uniform upper bound for time divided by logical volume, then

$$F_0 = O(1), \qquad F_k \le (s/W)F_{k-1} + O(1). \tag{12}$$

For $a = s/W < m^\tau$ the geometric sum $\sum_{j=0}^{k} a^j$ is $O(m^{k\tau})$. If $a \le 1$, the sum is at most $k + 1 = O(m^{k\tau})$; if $a > 1$, it is $O(a^k)$. Hence $F_k = O(e^\tau)$.

A fixed-tape depth-first schedule. Here we complete the implementation assertion used in (12). Reserve W role tapes, a fixed set of I/O and elementary-operation work tapes, one parking stack tape, and a separate descriptor stack. Although W is large, it is fixed. At a call boundary all reusable work tapes, other than the current I/O and local descriptor, are empty and their heads are at their origins. The parking and descriptor heads are at the tops of their stacks.

Before a recursive call, append the $W - 1$ inactive role streams to the parking stack in a fixed order, with separators. Copy the active stream to the child I/O area, and erase the parent role tapes while traversing their current contents. Push the parent descriptor and its finite program counter to the descriptor stack. The role and work tapes are now available to the child, using the same tapes at every depth. On return, copy the child output into the designated active role. Pop the inactive streams in reverse order. To recover a stream in its original direction, prepare its destination to the known length and write the popped symbols from the destination's right end toward its left end. Erase the popped stack cells as they are left. This uses linear time and ends with the parking head at the previously saved top. No scan passes through a parked ancestor. Cleanup is always limited to the region visited by the current call; a reused tape is not swept to the largest position ever visited by an ancestor.

Each push, pop, copy, positioning operation, or erasure moves $O(V)$ symbols at a parent node. There are only the fixed number s of child calls. This gives precisely the additional $O(V)$ term claimed above, even though each child itself has volume V/W. Temporary storage is therefore not substituted for logical volume in the factor s/W.

For completeness, descriptor processing also fits this term when the payload is one bit. Let V_0 be the volume at the root of one power-width call with width $e_0 = m^{k_0}$, and $V_j = V_0/W^j$ the volume of any depth-j child. Here $j \le k_0 = \log_m e_0$. The root row count is at least W^{k_0}, so at depth j it is still at least one. Both original chunk ranges remain present: digits outside the current subchunks become spectators rather than being removed. Hence

$$V_j \ge q^{2e_0}, \qquad j \log W \le (\log_m e_0) \log W = O(e_0) = O(\log V_j).$$

It follows that $\log V_0 = \log V_j + j \log W = O(\log(2V_j))$. A local descriptor lists the row field, at most the current $m + m$ fields, and a fixed number of intervals formed from intervening spectator fields. Computing products of their lengths, subdividing target fields, copying lengths, and storing the current instruction take $(\log(2V_0))^{O(1)} = O(V_j)$ time at that child, since every fixed power of $\log(2V_j)$ is $O(V_j)$. Before descending, construct the child's descriptor by retaining its two target fields and combining each consecutive group of spectator fields into one interval. The ancestor's subdivision need not remain active in the child. Stack entries contain finite instructions and binary

integers, not new alphabet symbols. These observations establish one finite machine for every recursion depth and every choice of spectator lengths.

□

We use the rational value $\tau = 1 - 2^{-50}$ established in (10). Its verification uses fixed integer comparisons and the displayed exponential inequality; it needs no real-arithmetic oracle.

4.3 Removing width and row restrictions

We have proved the fast operation for power widths with enough rows. Rows for a general array can be supplied by a few of its own high digits. Their movement will cost less than the final bound.

Lemma 12 (Arbitrary-width interchange)**.** *There exist fixed $0 < \tau < 1$ and one fixed finite-alphabet multitape procedure with the following property. For positive integers P, G, B and $u \geq 1$, it transforms an arbitrary bit array on*

$$[P] \times [2^u] \times [G] \times [2^u] \times [B]$$

by the address permutation

$$(p, h, g, d, z) \longmapsto (p, d, g, h, z)$$

in $O(Vu^\tau)$ steps, where $V = PGB\,2^{2u}$. The integers specifying the shape are part of the input. The time includes their processing, all padding and unpadding, and fixed-tape workspace cleanup. In particular, $B = 1$ is allowed.

Proof. First suppose the two ranges are $[q^e]$, where q is the fixed prime already chosen. Set

$$k = \lceil \log_m e \rceil, \qquad \rho = \left\lceil \frac{k \log W}{2 \log q} \right\rceil.$$

For all sufficiently large e, $1 \leq \rho < e$ and $\rho = O(\log(2e))$. Handle the bounded remaining set of widths by e individual digit interchanges; the constant in $O(Ve^\tau)$ absorbs their cost.

Write each chunk as its top ρ digits followed by its remaining digits, denoted h^+, h^- and d^+, d^-, respectively. Interchange the corresponding high digits one pair at a time. The high positions now contain d^+ and h^+; move the digits of the latter, preserving their order, into consecutive positions immediately after d^+. Suppressing the prefix and suffix, the field orders during the construction are

(h^+, h^-, g, d^+, d^-),	
(d^+, h^+, h^-, g, d^-)	after exchanging and joining the high parts,
(d^+, h^+, d^-, g, h^-)	after interchanging the remaining parts,
(d^+, d^-, g, h^+, h^-)	after separating the high parts again.

In the two middle orders, the first two fields form a row field of range

$$R = q^{2\rho} \geq W^k$$

before both remaining chunks. The joining moves preserve the order of the remaining digits and the gap field. Exchanging and joining the high parts costs $O(V\rho)$ by Lemma 3. Append zero rows, separately within each preceding prefix, until the row count is

$$R' = W^k \left\lceil R/W^k \right\rceil < 2R.$$

This changes the row field's length; it need not remain a power of q during the computation. The padded logical volume is less than $2V$.

Write the remaining width in base m:

$$e - \rho = \sum_{j=0}^{k} a_j m^j, \qquad 0 \le a_j < m.$$

Partition each remaining chunk into corresponding consecutive pieces, with a_j pieces of width m^j. Apply Proposition 11 to each corresponding pair. All these calls use the same row field of length R', which is divisible by each required W^j. At completion every call preserves its row address, so it neither mixes genuine rows with padded rows nor depends on the values used for padding. The padded zero rows remain zero at these completed-call boundaries. The sum of the costs is bounded by

$$O(V) \sum_j a_j m^{j\tau} = O(Ve^\tau).$$

The fixed bound $a_j < m$ and the inequality $m^k < me$ make this a geometric sum of the stated order. Scan away the padded rows, then reverse only the moves that joined the high parts. This is the last transition in the field-order display: h^+ returns to the later high position, while d^+ stays in the earlier one. The final order is therefore the required interchange of the two full chunks. The remaining movement cost $O(V \log(2e))$ is $O(Ve^\tau)$. Computing the piece list and every new shape is polynomial in $\log(2V)$ per piece. There are at most $(m-1)(k+1) = O(\log(2e))$ pieces; since $V \ge q^{2e}$, their total descriptor cost is a fixed power of $\log(2V)$ and hence $O(V)$. Each power-width call uses the fixed schedule proved above and leaves its work tapes clean.

Finally, for binary chunks let e be the least integer with $q^e \ge 2^u$. Pad each of the two chunk ranges numerically from $[2^u]$ to $[q^e]$, putting zero in an entry if either chunk coordinate is invalid. The increase in volume is less than q^2, a fixed factor. The padding is a nested scan through consecutive valid and invalid intervals, with their lengths prepared from the shape. It costs $O(V)$ with a constant depending on q. Lexicographic radix-q order of $[q^e]$ is its numerical order, just as binary order of $[2^u]$ is its numerical order. Padding therefore preserves the order of the existing entries without converting the digits of every address. The completed interchange preserves the set on which both chunk coordinates lie below 2^u. Scan away the other entries to recover the desired binary array. Since $e = O(u)$ for fixed q, the bound is $O(Vu^\tau)$. □

The lemma permits arbitrary prefix, gap, and suffix lengths. Thus every later interchange of equal bit chunks is an instance of its stated layout, after multiplying the irrelevant contiguous lengths. For widths differing by one bit, choose the leading bit of the longer chunk as the extra bit. Suppress the outer prefix and suffix, and write g for the intervening address field. If the first chunk is longer, write it as (h_0, H), so H and the second chunk D have equal width. The physical orders are

$$(h_0, H, g, D) \longmapsto (h_0, D, g, H) \longmapsto (D, g, h_0, H).$$

The first step interchanges H, D by the lemma; the second moves the single bit h_0 immediately before H. If the second chunk is longer, write it as (d_0, D), with H the first chunk. Use

$$(H, g, d_0, D) \longmapsto (d_0, H, g, D) \longmapsto (d_0, D, g, H).$$

Here the first step moves d_0 immediately before H, and the second interchanges H, D. In each case g remains between the two full chunks in the final order. If H, D are empty, the indicated equal-width interchange is the identity. Each case uses one single-bit move, costing $O(V)$ by Lemma 3,

in addition to the equal-width interchange. No unrestricted array transpose is being used in these operations.

What a transform actually changes

Added explanation Replace a list of values by weights of simple patterns.

A transform is a change in description. Consider the four numbers $[8, 4, 2, 2]$. Instead of listing those values directly, describe them as a sum of four simple patterns. The figure displays each contribution and the result. Add the first point from each of the first four plots: $4 + 3 + 0 + 1 = 8$. At the second point, $4 + 0 + 1 - 1 = 4$. Repeat at the remaining positions to get two and two.

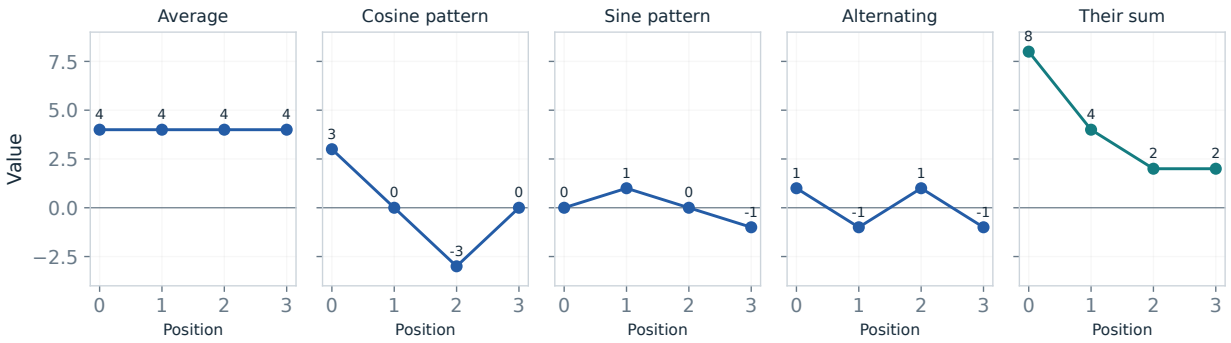


Figure E15. The same four numbers in a pattern description. Average $[4, 4, 4, 4]$, cosine contribution $[3, 0, -3, 0]$, sine contribution $[0, 1, 0, -1]$, and alternating contribution $[1, -1, 1, -1]$ sum exactly to $[8, 4, 2, 2]$. Lines connect four discrete samples; this is not measured sound.

Why call these frequencies? The average does not oscillate. The cosine and sine patterns complete one cycle over the four positions, with different starting phases. The alternating pattern changes sign at every step. In this real-pattern basis, the weights are $4, 3, 1, 1$. These are not the four complex Fourier coefficients in the paper's convention; they are an equivalent elementary way to display the same real data.

Where complex numbers come in. A complex number is a pair of real coordinates written $a + ib$. Addition works coordinate by coordinate. The rule $i^2 = -1$ makes multiplication by i turn the coordinate plane through a quarter-circle: (a, b) becomes $(-b, a)$. This is useful bookkeeping for oscillating patterns; it does not mean the input integers or their final product are imaginary.

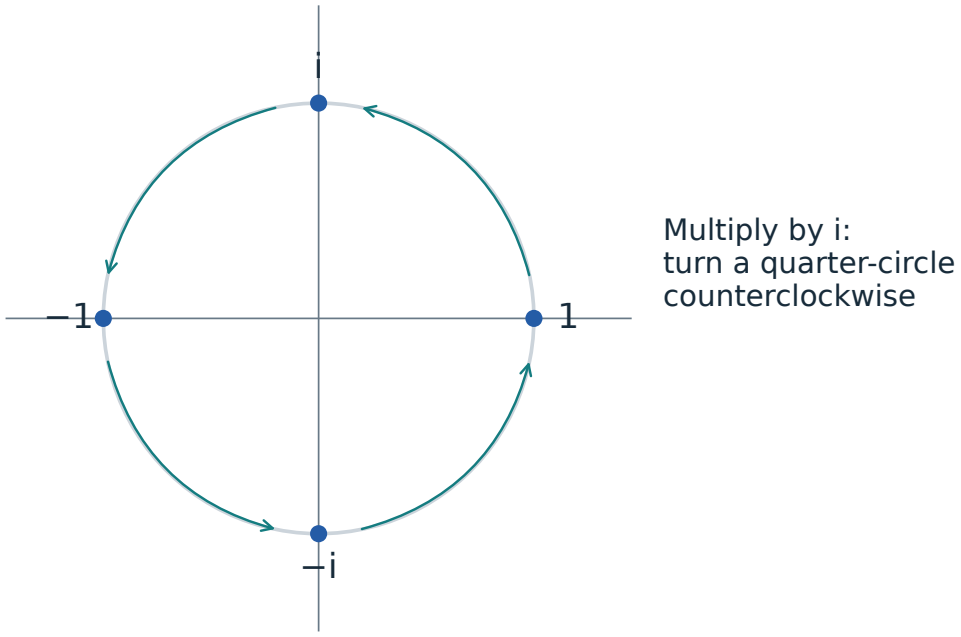


Figure E16. A phase is a position around a circle. Multiplication by i cycles through $1, i, -1, -i, 1$. Each point has magnitude one. The four turns provide the simplest visible example of roots of unity.

For four samples, the ordinary unnormalized Fourier coefficients of $[8, 4, 2, 2]$ are $[16, 6-2i, 4, 6+2i]$ with the negative-exponent convention. Dividing by four gives the normalized version $[4, 1.5 - 0.5i, 1, 1.5 + 0.5i]$. The average is four; the paired complex entries encode the cosine and sine contributions together. You do not need to derive these complex entries to follow the rest of the guide, but the distinction explains why transform scale factors cannot be guessed.

Why this helps multiplication. Shifting a list changes the phase of each frequency in a simple way. Convolution is built from weighted shifts, so in the frequency description it becomes multiplication of matching components, with the chosen normalization accounted for. The hard work is changing descriptions efficiently and reversing the change accurately. The full paper adds carefully chosen layouts and a polynomial representation to address those costs.

What a tensor product means here. First apply a two-entry operation to every horizontal pair in a 2×2 array, then to every vertical pair. Applying the same operation separately on both coordinate directions is its tensor-product action. The terminology becomes compact when there are many axes, but its meaning is still "do this operation in each selected direction while holding the others fixed."

What to carry forward. A transform changes the coordinates used to describe data. Structured patterns can make an operation cheap in those coordinates.

The smallest transform step, applied on many axes

Added explanation Read before simultaneous butterfly layers.

A two-point normalized Fourier transform sends

$$H_0(u, v) = \left(\frac{u+v}{2}, \frac{u-v}{2}\right).$$

The first output records an average; the second records a difference. For $(u, v) = (6, 2)$ the outputs are $(4, 2)$. Recover the original by $u = 4 + 2$, $v = 4 - 2$. The factor $1/2$ matters: this is not the unnormalized Hadamard matrix, nor the unitary matrix with factor $1/\sqrt{2}$.

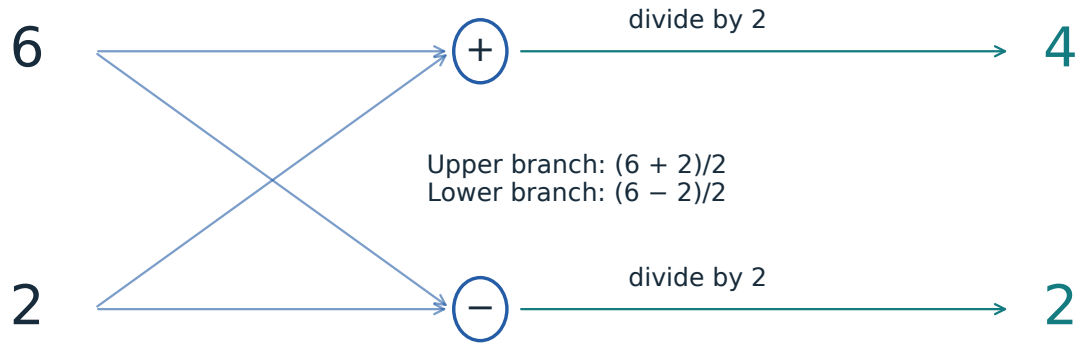


Figure E17. Follow the two arithmetic branches. Both inputs reach both operations. The plus branch returns four and the minus branch returns two after division by two. The entries are a worked example, not intermediate values from the large network.

For a two-axis example, start with the four entries $[8, 4, 2, 2]$ in address order $00, 01, 10, 11$. Applying H_0 to adjacent pairs gives $[6, 2, 2, 0]$. Applying it along the other bit pairs positions zero with two and one with three, giving $[4, 1, 2, 1]$. This is the action of $H_0 \otimes H_0$.

If every input has complex magnitude at most M, each output also does: $|u \pm v|/2 \le (|u| + |v|)/2 \le M$. Thus the completed operation is contractive in the maximum norm. Applying it on several address bits means pairing records that differ in a selected bit, for every setting of the other bits. A naive implementation makes one whole-array pass per selected axis. The proposed layer algorithm organizes those operations jointly.

Why the paper temporarily uses a different kernel. Define X to exchange two entries, $\mathsf{S} = \mathrm{diag}(1, i)$, $a = (1 + i)/2$, $b = (1 - i)/2$, and $C = aI + bX$. The useful identities are

$$C^2 = X, \quad C^4 = I, \qquad H_0 = b\mathsf{S}C\mathsf{S}.$$

The kernel C interacts well with the binary phase frames used to price residual directions. Once its tensor layer is complete, diagonal phases and a known dyadic complex factor produce the desired H_0 layer. These are classical linear identities; the use of phase matrices does not require a quantum computer.

One final rounding, not one rounding per internal gate. The internal values are exact Gaussian dyadics, numbers whose real and imaginary parts are integers divided by powers of two. Internal scratch can grow even though the completed H_0 layer is contractive. The proof therefore needs guard bits for internal magnitudes and denominators. It executes the whole layer exactly and truncates only at its boundary. Contraction alone would not justify discarding bits after every scratch update.

Why selected bits create exceptional addresses. An efficient packed address update may agree with a desired binary basis change on most addresses while carries disturb a small, explicitly described set. The paper isolates that set and repairs it deterministically. The proof needs agreement outside it, invariance of the exceptional set, and a bound on the cost of extracting, sorting, and reinserting full records. "Rare" does not mean random or ignorable: every input must still be correct.

What to carry forward. The second saving batches arithmetic across axes. It must survive basis changes, intermediate growth, and exact correction of exceptional records.

Original source anchors: `05-layers.tex, eq:phase-C; eq:phase-basic-identities`.

Original manuscript continues below.

5 Fast simultaneous synthetic butterfly layers

We now apply the complex network to a common butterfly layer on several axes. The array has complete address set

$$[P] \times [2^K]^D \times [S], \qquad 1 \le D \le d,$$

with $P, S \ge 1$, in lexicographic order. Its D consecutive K-bit chunks each select the bit at one common offset $0 \le \rho < K$, counted from the least significant end. Each address carries a polynomial record with r complex coefficients, initially in the unit disk on the grid $2^{-p}\mathbb{Z}[i]$, with $\Theta(p)$ bits per real or imaginary component. The intended operation is the tensor product of $H_0(u, v) = ((u + v)/2, (u - v)/2)$ on the D selected bits, acting coefficientwise and preserving all other address fields.

The parameters will satisfy

$$d = \Theta(p^\epsilon), \qquad K = \lfloor d^c \rfloor, \qquad r = 2^{\Theta(p/d)}, \qquad c > 0, \quad 0 < \epsilon < 1.$$

The exponents and comparison constants are fixed, and we impose their further restrictions in Proposition 14. Two consequences explain the layout. The polynomial length r grows faster than every fixed power of p, so a scan of a record pays for polynomial work on its address. Also $K/\log p \to \infty$, which will make the exceptional set in a packed change of address bits small enough to repair by sorting. We will obtain cost $O(Vd^{\lambda'})$, with $\lambda' < 1$ and V the logical bit volume, whereas separate butterfly factors would require D whole-array passes.

We first construct tensor products of the auxiliary two-point kernel C from Proposition 8. Its phase frames express a C tensor product using smaller groups of selected bits, separated by binary changes of basis. We implement those changes by moving whole address chunks. Splitting complete rows among the network's wire roles then reduces the volume of each recursive call. Finally, we bound the intermediate dyadic values and convert the completed C layer to the normalized H_0 layer. The entire layer is evaluated exactly before one final truncation.

5.1 Phase frames for a simultaneous butterfly layer

Hold the unselected address bits fixed and regard the stored complex values on one wire role as a function of the selected bits. The phase identities below express an edge's residual operator as a product of two-point kernels on selected directions.

Put

$$H_0 = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad H = \sqrt{2}H_0, \qquad \mathsf{S} = \operatorname{diag}(1, i), \qquad Z = \operatorname{diag}(1, -1).$$

For the bit-flip matrix $X = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, define

$$C = H\mathsf{S}H = aI + bX, \qquad a = \frac{1+i}{2}, \quad b = \frac{1-i}{2}. \tag{13}$$

Direct multiplication gives

$$H_0 = b\mathsf{S}C\mathsf{S}, \qquad C^2 = X, \qquad C^4 = I, \qquad C^{-1} = bI + aX = -iZCZ. \tag{14}$$

In particular, $C = iZC^{-1}Z$. These identities use only Gaussian dyadic coefficients. The Hadamard matrices in the definition of $\mathcal{C}_U$ are used to prove identities; they need not be executed by the machine.

Consider an edge from label U_- to label U_+, and choose an orthonormal basis $v_1, \dots, v_{\rho_{\text{edge}}}$ of its residual. Proposition 8 gives the frame change

$$\mathcal{C}_{U_+}\mathcal{C}_{U_-}^{-1} = \prod_{j=1}^{\rho_{\text{edge}}} (aI + bX_{v_j})^{\epsilon_j}, \qquad \epsilon_j \in \{1, -1\}.$$

Here $(X_v f)(x) = f(x + v)$, and we write $C_v = aI + bX_v$. The number of directional kernels is precisely ρ_{edge}, including on decreasing edges. The next step puts those directions into coordinate positions without performing a general transpose.

Implementing an arbitrary residual basis. For $M \in \mathrm{GL}(m, \mathbb{F}_2)$, let $\mathcal{P}_M$ be the address permutation that moves the value at x to Mx. Thus $(\mathcal{P}_M f)(x) = f(M^{-1}x)$, and

$$\mathcal{P}_M X_v \mathcal{P}_M^{-1} = X_{Mv}. \tag{15}$$

Extend $v_1, \dots, v_{\rho_{\text{edge}}}$ to a basis of $\mathbb{F}_2^m$ and take these vectors as the first ρ_{edge} columns of M. The displayed edge identity then becomes

$$\mathcal{C}_{U_+}\mathcal{C}_{U_-}^{-1} = \mathcal{P}_M \left(\prod_{j=1}^{\rho_{\text{edge}}} C_{e_j}^{\epsilon_j} \right) \mathcal{P}_M^{-1}, \tag{16}$$

where e_j is the jth standard basis vector. The matrix M need not preserve the dot product: equation (15) only uses its invertibility.

For the array layout used below, take a group of $e = mf$ consecutive axis chunks and divide it into m consecutive slots of f chunks, numbered in physical order. Number the chunks within each slot by $j = 0, \dots, f-1$, starting at its least significant end. Write $x_{h,j}$ for the selected bit in chunk j of slot h, where $1 \le h \le m$. Column j is the vector $(x_{1,j}, \dots, x_{m,j})$. A frame on this group is the tensor product of the same phase frame $\mathcal{C}_U$ of Proposition 8 on these f columns.

To implement (16) on the group, first apply $\mathcal{P}_M^{-1}$ separately to each column. For each $1 \le h \le \rho_{\mathrm{edge}}$, apply $(C^{\epsilon_h})^{\otimes f}$ to the f bits $x_{h,0}, \dots, x_{h,f-1}$. This is one child operation on the f axes in slot h. Finally apply $\mathcal{P}_M$ to every column. A negative directional kernel still requires only one forward child, because

$$(C^{-1})^{\otimes f} = (-i)^f Z^{\otimes f} C^{\otimes f} Z^{\otimes f}. \tag{17}$$

The rightmost phase is applied first. All inverse wrappers are address signs and constant fourth-root phases, so they add no recursive calls.

Gaussian elimination expresses any $M \in \mathrm{GL}(m, \mathbb{F}_2)$ as a product of at most $m(m-1) + 3m$ elementary row additions: one clears at most $m-1$ other entries per pivot, and each row swap is three row additions. Reversing the elimination gives the required product since each row addition is an involution. The number is a constant because m is fixed. Adding row h' to row h in every column applies $x_{h,j} \leftarrow x_{h,j} \oplus x_{h',j}$ for every $0 \le j < f$. Thus one row addition changes corresponding selected positions in two whole slots. The packed address operation proved below performs all of these XORs together.

The exact recursive contract. Apply the finite network with these column frames to a group of $e = mf$ selected axes. Summing its residual dimensions gives exactly s_{c} child calls, each on the $f = e/m$ axes of one slot. The source and sink $Z_{a,f}$ signs, together with the scalar i^{27f} at each affected sink, turn every endpoint kernel into a forward one. Negating the prescribed physical bank and undoing its exchange returns each result to its original role. Proposition 8 therefore gives $C^{\otimes e}$ on every role, including roles whose initial scratch values were arbitrary. Formula (17) supplies each inverse child from one forward call, and none of the corrections adds a child. The remaining operations are a fixed number of binary row additions and coefficientwise scalar operations.

Returning to normalized butterflies. For an address x in a full layer, write $x_1, \dots, x_D$ for its selected bits. The diagonal operator $\mathcal{S}_D$ multiplies the value stored at an address by the phase determined by those bits:

$$(\mathcal{S}_D z)(x) = i^{x_1 + \cdots + x_D} z(x).$$

Here the unselected address bits are unchanged, and the same phase is applied to every coefficient stored at that address. Applying (14) on all selected coordinates gives

$$H_0^{\otimes D} = b^D \mathcal{S}_D C^{\otimes D} \mathcal{S}_D. \tag{18}$$

The scalar is

$$b^D = (-i/2)^{\lfloor D/2 \rfloor} b^{D \bmod 2},$$

so it is a binary shift and fourth-root phase, with at most one further factor b. All phases, inverse wrappers, and scalar corrections above are exact Gaussian dyadic operations. Thus, once the physical binary basis changes and recursive stream layout are supplied, the complete parallel butterfly layer is exact up to its prescribed final truncation.

5.2 Packed changes of selected address bits

We implement a binary row addition on corresponding selected bits in two address chunks. A third chunk supplies temporary address bits; its arbitrary initial value must be restored. The construction first uses a constant number of cyclic shifts of whole chunk ranges, then corrects an explicitly bounded set of addresses. The correction depends only on addresses, so the time bound is worst-case over all payloads.

We use three previously established streaming operations. Lemma 12 exchanges two disjoint address chunks of equal width L in $O(VL^{\tau})$ time, where V is the current stream volume and $0 < \tau < 1$ is fixed. The construction in Lemma 4 cyclically shifts the complete range of a target chunk by an offset determined by preceding fields in $O(V)$ time apart from calculating the offsets. Finally, Lemma 3 applies any fixed permutation to the two-bit value at two specified address positions in $O(V)$ time. Each operation uses a fixed number of one-dimensional tapes and restores the order of all spectator fields.

The asymptotic simplification in the following lemma describes a fixed family of parameters as $p \to \infty$, after the exponents and comparison constants have been fixed. The statement that R exceeds every fixed polynomial in p means that $R/p^{C} \to \infty$ for each fixed C. Implied constants and eventual cutoffs may depend on the fixed choices and on when the family's asserted bounds begin to hold. In particular, each explicit bound is uniform over all of its declared inputs; its asymptotic simplification is uniform over the allowed row counts and payloads within a fixed family.

Lemma 13 (Packed selected-bit addition on one rectangle)**.** *Let $R \ge 1$, $K \ge 6$, $f \ge 1$, $L = fK$, and $0 \le \rho < K$. Let N_0, N_1, N_2, N_3 be positive integers, and let (χ_1, χ_2, χ_3) be a permutation of (x, y, z). Consider R-bit records in lexicographic order on*

$$[N_0] \times [2^L]_{\chi_1} \times [N_1] \times [2^L]_{\chi_2} \times [N_2] \times [2^L]_{\chi_3} \times [N_3].$$

The subscript names the chunk at that position. The input supplies $N_0, \dots, N_3, R, K, f, \rho$ as a fixed number of canonical binary integers, together with the finite permutation of the chunk names, as in Section 2. Put

$$M = 2^{3L} \prod_{h=0}^{3} N_h, \qquad V = MR, \qquad A_{\mathrm{rect}} = \lceil \log_2(2V) \rceil.$$

At the selected positions $j_i = \rho + iK$, numbered from the least significant bit, one fixed multitape procedure performs

$$y_{j_i} \longleftarrow y_{j_i} \oplus x_{j_i} \quad (0 \le i < f).$$

It preserves the spectator coordinates, all other chunk bits, and every record payload; in particular, it restores the entire chunk z for every initial value. With $\delta_{\mathrm{bad}} = \min\{1, 80(f-1)2^{-K}\}$, its time is

$$O\Big(V(L^{\tau}+1) + MA_{\mathrm{rect}}^{3} + \delta_{\mathrm{bad}} MA_{\mathrm{rect}}(R + A_{\mathrm{rect}})\Big). \tag{19}$$

When $A_{\mathrm{rect}} = O(p)$, R exceeds every fixed polynomial in p, $f \le d \le p$, and $K/\log p \to \infty$, this is $O(V(L^{\tau}+1))$, uniformly over the permitted lengths and payloads in the fixed family.

Proof. We first construct eight rotations that have the required effect away from a small set of addresses. We then correct exactly that set. For a Boolean control c and arbitrary integers u, w, perform the following eight operations in the displayed chronological order. Every predicate uses the current values of the integers.

$$\begin{aligned}
u &\leftarrow u - c[\, w \equiv 0 \pmod 2\,], && (1)\\
w &\leftarrow w - c[\, u \equiv 0 \pmod 2\,], && (2)\\
u &\leftarrow u + c[\, w \equiv 0 \pmod 2\,], && (3)\\
w &\leftarrow w + c[\, u \equiv 0 \pmod 2\,], && (4)\\
w &\leftarrow w + c[\, u \equiv 1 \pmod 2\,], && (5)\\
u &\leftarrow u + c[\, w \equiv 1 \pmod 2\,], && (6)\\
w &\leftarrow w - c[\, u \equiv 1 \pmod 2\,], && (7)\\
u &\leftarrow u - c[\, w \equiv 1 \pmod 2\,]. && (8)
\end{aligned} \tag{20}$$

If $c = 0$, these operations do nothing. If $c = 1$, the first four operations induce the cycle (00 01 10) in every square

$$\{2a, 2a+1\} \times \{2b, 2b+1\}, \qquad a, b \in \mathbb{Z},$$

where the corner labels record the parities of (u, w). The last four induce (01 10 11). Applying the first cycle and then the second gives (00 10)(01 11). These facts follow by substituting the four parity pairs into the displayed operations; translation by $(2a, 2b)$ does not change any predicate. Consequently the complete output is

$$(u', w') = (u + c(1 - 2(u \bmod 2)),\ w). \tag{21}$$

Here $u \bmod 2 \in \{0, 1\}$, also for negative u. Thus the parity of u is flipped precisely when $c = 1$, both quotients under division by two are preserved, and the arbitrary auxiliary integer w is restored. Each coordinate is updated four times by at most one, so its displacement from its initial value is at most four, even if an intermediate point leaves its starting quotient square. The weaker bound eight will suffice below.

Omit the most significant selected position j_{f-1} temporarily. For each $0 \le i < f - 1$, the bits in positions $j_i, \dots, j_i + K - 1$ form a K-bit segment. When $f > 1$, these segments are disjoint, and the highest one ends immediately below j_{f-1}. Write their initial values in y and z as

$$u_i = 2g_i + a_i, \qquad w_i = 2h_i + b_i, \qquad a_i, b_i \in \{0, 1\}.$$

The values $g_i, h_i \in \{0, \dots, 2^{K-1} - 1\}$ are the guard values. The control for this pair of segments is the selected bit $c_i = x_{j_i}$.

Apply each line of (20) simultaneously to all these segment pairs. Physically, a line updating y by sign ε and testing parity b of z is the single cyclic shift

$$y \longleftarrow y + \varepsilon \sum_{i=0}^{f-2} 2^{j_i} [\, x_{j_i} = 1 \,][\, z_{j_i} = b \,] \pmod{2^L}. \tag{22}$$

For a line updating z, interchange y and z in this formula. The shift offset never depends on its target chunk, so each line is a bijection of the rectangle. Let S_0 be their composition. It fixes the spectator coordinates.

Let $\mathcal{B}$ consist of the addresses for which at least one used guard value, in either y or z, fails

$$10 \le g \le 2^{K-1} - 11. \tag{23}$$

At an address outside $\mathcal{B}$, each segment initially lies in $[20, 2^K - 21]$. Even displacement eight keeps it inside $[0, 2^K - 1]$. Inductively, each packed addition therefore agrees with all the independent segment additions: no segment carries into or borrows from the next segment. When $f > 1$, the bits below j_0 and above the highest used segment are untouched as well. By (21), S_0 then agrees with the ideal permutation T_0 that performs the required XORs at $j_0, \dots, j_{f-2}$ and fixes all other address coordinates.

There are exactly twenty excluded values for each guard in (23). Since its complete range has 2^{K-1} values, the union bound over the $2(f - 1)$ guards gives

$$|\mathcal{B}| \le \delta_{\text{bad}} M. \tag{24}$$

Indeed, the complete y, z ranges give the same guard distribution at each choice of spectator coordinates. No record value enters this count.

The ideal map T_0 preserves all guard values and therefore preserves $\mathcal{B}$. Both T_0 and S_0 are bijections, and they agree off $\mathcal{B}$. Taking complements in the finite rectangle gives

$$S_0(\mathcal{B}^c) = T_0(\mathcal{B}^c) = \mathcal{B}^c, \qquad S_0(\mathcal{B}) = \mathcal{B}.$$

Thus $T_0 S_0^{-1}$ fixes $\mathcal{B}^c$ and permutes $\mathcal{B}$. It remains to implement the rotations and this correction on tapes.

For one rotation, move its target to the rightmost of the three chunk positions if necessary. Let P^{sw} be the product of the record field lengths before the earlier exchanged chunk, let G^{sw} be the product between the exchanged chunks, and let B^{sw} be R times the product after the later one. Empty products are one. The bit array has exactly the shape

$$[P^{\mathrm{sw}}] \times [2^L] \times [G^{\mathrm{sw}}] \times [2^L] \times [B^{\mathrm{sw}}].$$

The third chunk lies inside one collapsed spectator interval. Pass only L and these three positive canonical lengths to Lemma 12. Each length is at most V, so the local descriptor has $O(A_{\mathrm{rect}})$ bits. The swap costs $O(VL^{\tau})$, including its own padding and cleanup.

The equal chunk lengths leave the field-length list unchanged by a completed swap. Keep the permutation of the logical names x, y, z in finite local state. With the target now after its two controls, collapse the remaining intervals to obtain

$$[\Lambda_0] \times [2^L]_{\mathrm{control}\,1} \times [\Lambda_1] \times [2^L]_{\mathrm{control}\,2} \times [\Lambda_2] \times [2^L]_{\mathrm{target}} \times [B_{\mathrm{rot}}],$$

where B_{rot} includes the payload and every displayed spectator length is positive. Their product with the chunk lengths is V. Together with K, f, ρ and the finite name permutation, these fixed canonical lengths form an $O(A_{\mathrm{rect}})$-bit local descriptor. The offset routine receives the two current L-bit logical control values. A scan of at most $f \le L$ selected positions evaluates (22) in $L^{O(1)}$ time, without reading the target or spectator values. Lemma 4 therefore gives an $O(V)$ rotation: its offset work is absorbed by the 2^L nonempty target blocks in each fiber. Undo the swap after the rotation. The local descriptors and name permutation are updated at completed swap boundaries; the primitives use their own descriptors inside padded layouts. There are at most sixteen swaps and eight rotations in S_0.

To implement the correction, scan the stream with a record-rank counter. All chunk names are back in their original positions. Dividing the current rank through the fixed seven field lengths recovers the three chunk values and their mixed-radix record strides. The strides exclude the R payload bits. All involved integers have $O(A_{\mathrm{rect}})$ bits, so schoolbook arithmetic, descriptor copying, and resets take at most $O(A_{\mathrm{rect}}^3)$ per record. Test membership in $\mathcal{B}$ and extract just those records. For a current exceptional address q, compute $q' = T_0(S_0^{-1}(q))$ by reversing the eight modular additions with negated offsets, then applying the selected XORs directly. Each reversed mask uses the current controls of that reversed line. There are at most $f \le L \le A_{\mathrm{rect}}$ selected positions, so this calculation also fits $O(A_{\mathrm{rect}}^3)$ time.

If q_h, q_h' are the current and destination values of chunk $h \in \{x, y, z\}$ and σ_h is its record stride, the fixed spectator coordinates give

$$\mathrm{rank}(q') = \mathrm{rank}(q) + \sum_{h \in \{x,y,z\}} (q_h' - q_h)\sigma_h.$$

The differences are signed. Attach this destination rank, padded to $\lceil \log_2 M \rceil \le A_{\mathrm{rect}}$ bits, to the extracted record, and mark its hole on a fixed track of the full stream. Hold the full stream untouched while stable binary radix sorting scans only the extracted records and keys, least significant key

bit first. Since $T_0 S_0^{-1}$ permutes $\mathcal{B}$, these keys are distinct and are exactly the ranks of the marked holes. One forward full-stream scan reinserts the sorted records. This implements T_0 everywhere.

Finally apply the two-bit XOR to $x_{j_{f-1}}, y_{j_{f-1}}$ by Lemma 3, splitting those chunks at the selected positions and collapsing the other intervals. Its descriptor again has a fixed number of positive lengths, each at most V. This costs $O(V)$ and supplies the omitted selected position. When $f = 1$, S_0, T_0 are identities, $\mathcal{B}$ is empty, and this last step is the entire algorithm.

The movement costs $O(V(L^\tau + 1))$. A fixed number of full record scans costs $O(MA_{\mathrm{rect}}^3 + V)$, and the at most A_{rect} radix passes cost $O(|\mathcal{B}|A_{\mathrm{rect}}(R + A_{\mathrm{rect}}))$; skip them when the extracted bank is empty. The initial descriptor has a fixed number of canonical integers of $O(A_{\mathrm{rect}})$ bits, so its preparation is covered by $O(MA_{\mathrm{rect}}^3)$. All counters, positioning, rewinds, and cleanup are included. Equation (24) proves (19) on a fixed number of tapes. After division by $V = MR$, its additional terms are bounded by A_{rect}^3/R and $80(f-1)2^{-K}A_{\mathrm{rect}}(1 + A_{\mathrm{rect}}/R)$. Under the stated family bounds the first tends to zero, and the second is $O(p^2 2^{-K}(1 + A_{\mathrm{rect}}/R)) = o(1)$. This proves the final assertion. □

Application to a fixed binary change of basis. An invertible matrix over $\mathbb{F}_2$ of fixed order $m \geq 3$ is a product of a fixed number of elementary row additions; a row exchange is three such additions. In one invocation of the finite network on $e = mf$ axes, slot h consists of f consecutive K-bit axis chunks. It therefore has address-bit width $L = fK$, and its selected bits $x_{h,j}$ occupy positions $\rho + jK$ for $0 \leq j < f$. To add row h' to row h, use slot h' as x, slot h as y, and any third, distinct slot as z in Lemma 13. The lemma performs $x_{h,j} \leftarrow x_{h,j} \oplus x_{h',j}$ for every j in one operation and restores the third slot, whatever its initial address value.

Under the asymptotic hypotheses in the final assertion of that lemma, the complete change of basis and its inverse therefore cost $O(V((eK)^\tau + 1))$ on a stream of volume V. Here e counts axis chunks, while eK counts address bits; the fixed factor m between eK and the slot width does not affect the bound. The same conclusion holds after the row padding and splitting below, because every role stream retains a complete suffix with the same fixed field lengths in every row.

5.3 Batching a common layer on a fixed number of tapes

We now turn the finite complex network into an algorithm for a whole layer. Throughout this subsection its constants are denoted by m, W, s: the network has W wires, its binary label space has dimension m, and the sum of the dimensions of its edge residuals is $s < Wm$. These are the constants of the complex network; they need not equal the constants used to prove the chunk-swap lemma.

Use the complete layout and parameter family specified at the start of the section, with fixed rational exponents c, ϵ to satisfy the recurrence and guard conditions below. Store each polynomial in coefficient order with a common width $w = \Theta(p)$ per real or imaginary component, and assume the complete address and its binary shape descriptor have length $O(p)$. Thus an input record has $2rw$ bits. Write V for the current logical bit volume, excluding temporary work streams. We first count the work on $C^{\otimes D}$ assuming one common exact coefficient format of width $O(p)$; the guard calculation below then supplies that format.

An individual selected-bit kernel. Write each input coefficient once in this common exact format and retain it until the final truncation. The input width is $\Theta(p)$, so the initial copy costs $O(V)$ and the guarded stream has volume $O(V)$.

To apply one C kernel, use the split from Lemma 3 at the selected address bit. Call the two streams 0 and 1 according to that bit. Because that bit has its complete two-element range,

the two streams have the same remaining address set in the same order. Their aligned record addresses therefore differ only in the selected bit, and their coefficients are still ordered by the same polynomial index. Scan each pair of records in that order. For corresponding complex coefficients u from stream 0 and v from stream 1, compute

$$C(u, v) = \left(\frac{u + v + i(u - v)}{2}, \frac{u + v - i(u - v)}{2} \right). \tag{25}$$

Addition, subtraction and halving use the fixed-point conventions of Section 2; multiplication by i swaps the real and imaginary components and changes one sign. The common guarded width makes these operations exact. The coefficient scan costs $O(rp)$ per pair of records, linear in their guarded bit length and including any reversal of bits inside a coefficient. Write the first result to stream 0 and the second to stream 1, both in coefficient order, then merge according to the selected bit. The merge restores the original lexicographic address order. The elementary split and merge include their counters and cleanup and cost $O(V)$; the paired arithmetic scan also costs $O(V)$. Thus e individual kernels cost $O(Ve)$ on any stream with these complete ranges and record format.

First consider an internal node containing $e = m^k$ consecutive active chunks, with $k \geq 1$. Split it into m consecutive slots, each containing e/m chunks. The phase construction supplies one recursive call on a slot for each vector in an edge residual basis, hence exactly s calls in one invocation of the network. If the logical volume entering this node is V, binary changes of basis, their inverses, scalar gates, endpoint corrections and reassembly of the physical banks cost

$$O(V((eK)^\tau + 1))$$

at this node. Each slot has $(e/m)K$ address bits, so Lemma 13 gives the stated bound for a row addition. The network and all of its m-dimensional residual bases are fixed, so the total number of row additions is fixed as well. At the same guarded width, the scalar gates cost a constant number of scans of the current volume. Address and descriptor work bounded by any fixed polynomial in p per record is also absorbed: its ratio to the record's $\Theta(rp)$ bits tends to zero.

Where the rows come from. Put

$$q_0 = \lceil \log_2 W \rceil \, \lceil \log_m (2d) \rceil, \qquad k_0 = \lceil \log_m d \rceil.$$

If $D \leq q_0$, use individual C kernels, at cost $O(V \log d)$. Otherwise apply C individually on the selected bits of the first q_0 chunks, in $O(V \log d)$ time. These chunks remain a complete consecutive address block before all the remaining chunks; regard their concatenated bits as one row index. Its range has length $R_{\text{row}} = 2^{q_0 K} \geq W^{k_0}$, since $K \geq 1$ for sufficiently large p and $q_0 K \geq k_0 \log_2 W$. Within each preceding prefix, add the same number of zero rows to reach the next multiple of W^{k_0}; the volume grows by a factor at most two. At any recursive depth $j \leq k_0$, the row count is divisible by $W^{k_0 - j}$.

At a node, write a row index as $u = Wg + w$, with $0 \leq w < W$, and send that entire row to physical wire role w, indexed there by g. Every role stream has the same remaining address shape. Thus a pointwise gate reads the records with the same g and the same remaining address in its touched roles. The scalar network allows these role values to be arbitrary, so the split supplies its scratch roles from existing rows. Each recursive edge call acts on one role stream with volume exactly $1/W$ of its parent's logical volume.

Splitting the row index removes no address within a row. Every row in every role stream, including a padded row, retains the complete suffix $[2^K]^{D - q_0} \times [S]$ in the same field order and with the same lengths; inactive chunks remain complete spectator fields. The row counts also remain

equal across preceding prefixes, decreasing by the same factor W at each split. The network is invoked only on these equal rows: their common suffix gives the aligned role addresses and exact V/W child volumes above. For each packed call, collapse the preceding prefix and row index into one field, and collapse the spectator intervals around its three slots. This gives the rectangle of Lemma 13, whose guard count and deterministic repair apply to every child.

The W role streams use a fixed number of tapes. Before entering a child, push the other role streams onto a LIFO work tape, copy the active stream to a reserved child input area, and clear or reuse the role areas. On return, copy the result back and pop the parked streams, reversing each popped stream when necessary to restore its order. The stack head is at its top at every call boundary. Pushing, popping, copying, resetting and clearing at this node cost its own volume times a fixed constant; no step traverses an ancestor's parked data. The call descriptor and program counter use a separate stack. The number of child calls is the fixed integer s, so these costs are included in the node overhead above. The fixed-tape chunk-swap subroutine has its own fixed work areas, reused on every call.

For each packed call, padding has increased the row count by at most a factor two and row splitting can only decrease it. Its active address still has $O(p)$ bits. The four collapsed spectator lengths therefore have $O(p)$-bit descriptions. Its record-width parameter describes a polynomial of $\Theta(rp)$ bits and has $O(\log(rp)) = O(p)$ bits as well. These lengths, together with K, f, ρ and the finite chunk-name permutation, are precisely the local descriptor required by Lemma 13. Both the record count and record width have $O(p)$-bit lengths, so its bound uses $A_{\mathrm{rect}} = \lceil \log_2(2V) \rceil = O(p)$. A selected-position mask has at most $DK = O(p)$ bits.

The binary descriptions of all growing shape parameters have polynomial length in p. Each nonempty child stream still contains a complete length-r polynomial record. Its volume therefore absorbs polynomial descriptor setup at that node, as well as polynomial address work per record. Even if every node is counted separately, there are only $d^{O(1)}$ nodes: the branching factor s is fixed and the depth is at most $\lceil \log_m d \rceil$. Any additional fixed power of p from this count is still dominated by r.

Every completed network invocation applies $C^{\otimes e}$ separately to each input row. Its selected bits lie in the remaining chunks, disjoint from the bits used by the earlier preprocessing. On the genuine rows, those two completed operations therefore commute. Each padded zero row is again zero when an invocation finishes, so the added rows can be deleted after the invocations on the remaining chunks have completed. Intermediate scratch values may be arbitrary.

Unrolling the recurrence. Fix $\beta = 1/2$, and choose real exponents satisfying

$$0 < \tau < 1, \qquad \max\{0, \log_m(s/W)\} \le \sigma < 1, \qquad \max\{\tau, \sigma\} < \lambda < 1, \qquad \tau(1 + c/\beta) < \lambda.$$

The exponents can all be chosen rational, with a strict upper bound in the condition on σ. Stop recursion when $e < d^\beta$ and apply the e C kernels individually. If $F(e)$ denotes time divided by the current logical stream volume, then

$$F(e) \le (s/W)F(e/m) + O((eK)^\tau + 1), \qquad F(e) = O(e) \quad (e < d^\beta).$$

Indeed, the s child calls are executed on streams of volume V/W, so their total cost is $s(V/W)F(e/m)$. Dividing by the current volume V gives the factor s/W. At an internal node $e \ge d^\beta$, hence $K \le d^c \le e^{c/\beta}$. The overhead is therefore $O(e^{\tau(1+c/\beta)} + 1) = O(e^\lambda)$. Put $a = s/W$. Since $a \le m^\sigma < m^\lambda$, the internal costs sum to at most

$$Ce^\lambda \sum_{j \ge 0} (a/m^\lambda)^j = O(e^\lambda).$$

If the leaves after j levels contain $u = e/m^j < d^\beta$ active chunks each, their combined normalized contribution is at most

$$O(a^j u) \le O(e^\sigma u^{1-\sigma}) \le O(e^\sigma d^{\beta(1-\sigma)}).$$

The first inequality uses $a^j \le m^{j\sigma} = (e/u)^\sigma$. It also covers $a < 1$ by taking $\sigma \ge 0$.

Write $D - q_0 = \sum_j a_j m^j$ in base m, with $0 \le a_j < m$. Partition the remaining chunks into a_j consecutive pieces containing m^j chunks for each j. There are at most $(m-1)(1+\lfloor \log_m d \rfloor)$ pieces. Adding their costs and the row preprocessing gives

$$O\Big(\log(2d)\big[d^\lambda + d^{\sigma+\beta(1-\sigma)} + 1\big]\Big) = O(d^{\lambda'})$$

for any fixed

$$\max\{\lambda, \sigma + \beta(1-\sigma)\} < \lambda' < 1.$$

The case $D \le q_0$, in which preprocessing would consume all active chunks, was handled by individual kernels. Thus all constants and all tape counts are independent of p, d, r and the input data.

5.4 An explicit guard-width bound

The accelerated network is evaluated exactly until an entire normalized butterfly layer is complete. Its intermediate operators need not be contractions. We now give explicit constants that bound denominator and magnitude growth within one layer.

For the complex network, let

$$E = 64(W + m + 1)^3, \qquad B = s + E, \qquad \nu = \min\{j \ge 1 : m^j \ge B\},$$

and set

$$C_1 = \max\{20, \nu + 3\}, \qquad C_0 = 128 m B^2, \qquad \Delta = \lceil C_0 d^{C_1} \rceil.$$

For the stated $h = 100$ construction, the complex network has

$$\begin{aligned} m &= 1{,}000{,}000, \\ W &= 1{,}873{,}807{,}244{,}643{,}542{,}670{,}000, \\ s &= 1{,}873{,}807{,}244{,}636{,}671{,}267{,}308{,}000{,}000. \end{aligned}$$

These values give $m^{10} < B < m^{11}$, hence $\nu = 11$ and $C_1 = 20$. In fact the bound below needs only exponent fourteen; the choice twenty leaves an explicit additional margin.

We measure arithmetic dependency depth using complex additions, subtractions, halvings, sign changes and multiplication by i. For a group of e axes, let $A(e)$ bound the number of these operations on any chain from an input coefficient to an intermediate coefficient in its recursive C-layer. A chain follows an operand into a later arithmetic result. Work on other coefficients or records that do not feed this value lies on other chains, even though the fixed-tape machine executes that work sequentially. To obtain a conservative bound, we concatenate all scalar gates and all recursive edge calls within one network invocation.

The scalar schedule (2) touches a data wire at most four times in each of the three stages. A scratch wire belongs to one invocation and is touched at most four times there. Thus every physical wire occurs in at most twelve scalar gates, and there are at most $12W$ gates. A gate has at most W inputs and outputs and coefficients in $\{0, \pm 1, \pm 1/2\}$. One output is a sum of at most W terms, each requiring at most one halving and one addition or subtraction. Evaluating all outputs from saved inputs therefore takes at most $2W^2$ elementary operations.

For an inverse child, each $Z^{\otimes f}$ is one sign chosen by the parity of the selected address bits, and the scalar fourth-root phase takes at most one sign change and one multiplication by i. This gives at most $4s$ operations for the child corrections. The source, sink and bank corrections take at most $4W + 4$ more. Calculating the address parities and phase exponents is address or descriptor work, not arithmetic on coefficients. Since $s < Wm$, the total $24W^3 + 4s + 4W + 4$ is less than $E = 64(W + m + 1)^3$. Binary address permutations add no coefficient arithmetic at their interfaces.

The paired evaluation in (25) uses at most eight elementary operations: form the sum and difference, multiply the difference by i, then form the two half-sums. We may therefore use

$$A(e) \le sA(e/m) + E \quad \text{at an internal node,} \qquad A(e) \le 8e \quad \text{at a leaf.}$$

The depth estimate concatenates the s calls, so it uses s. The factor $1/W$ in the earlier time recurrence accounted for their smaller stream volumes.

There are at most $k \le \lceil \log_m d \rceil$ internal levels. For any bound $x \ge 1$, $sx + E \le (s + E)x = Bx$. Starting with $8e \le 8d \le 8dB$ at a leaf and applying this inequality through the levels gives the first bound below. For the second, use $B \le m^\nu$ and

$$B^{k+1} \le B^{\log_m d + 2} = B^2 d^{\log_m B} \le B^2 d^\nu.$$

Thus

$$A(e) \le 8dB^{k+1} \le 8B^2 d^{\nu+1}.$$

For the sufficiently large sizes under consideration, the earlier piece count is at most md. Concatenating their depths contributes at most $8mB^2 d^{\nu+2}$. In either preprocessing branch, at most d individual kernels are used, contributing at most $8d$. Splitting rows, padding, copying and removing padding add no coefficient arithmetic.

Each of the two outer diagonals applies a fourth-root phase and costs at most two elementary phase operations. Put $t = \lfloor D/2 \rfloor$. Since $b^2 = -i/2$, the scalar b^D is $(-i)^t 2^{-t} b^{D \bmod 2}$. Its phase costs at most two operations, and the possible factor b is computed as $(z - iz)/2$ in three operations. The shift by t consumes t fractional bits, so we charge it t units of denominator growth even though one tape scan performs the shift. Charging these same units to magnitude growth gives a common upper bound for both:

$$\begin{aligned} 8mB^2 d^{\nu+2} + 8d + t + 9 &\le 8mB^2 d^{\nu+2} + 18d \\ &\le 26mB^2 d^{\nu+2} \\ &< 128mB^2 d^{\nu+3} \le C_0 d^{C_1}. \end{aligned}$$

Here $d \ge 1$, $mB^2 \ge 1$, and $C_1 \ge \nu + 3$.

To see how this count controls numerical size, start with disk-valued coefficients in $2^{-p}\mathbb{Z}[i]$. A value at depth ℓ lies in $2^{-(p+\ell)}\mathbb{Z}[i]$ and has modulus at most 2^ℓ. This follows by induction: addition or subtraction can at most double the maximum modulus of its operands; halving adds at most one denominator bit; and signs and multiplication by i preserve both bounds. The charge of t for the final shift is the same as t successive halvings. Thus reserving $p + \Delta$ fractional bits and $\Delta + 3$ integer and sign bits per real component represents every intermediate coefficient exactly and prevents overflow. Shifts consume the reserved fractional zeros and discard no bits.

An address-swap subroutine may temporarily hold XOR combinations of the fixed-width encoding bits. These strings are not used as coefficients during the swap. It returns an exact permutation of the encodings before coefficient arithmetic resumes, so these internal bit operations cause no numerical rounding or denominator growth.

Choose $\epsilon C_1 < 1$. Since $d = \Theta(p^\epsilon)$, $\Delta = o(p)$, and the full coefficient width is $O(p)$, as required in the time analysis. Addition, signed shifts, copying and phase changes consequently take $O(p)$ bit operations per coefficient.

Proposition 14 (Simultaneous normalized butterfly layer)**.** *Fix*

$$\tau = \sigma = 1 - 2^{-50}, \qquad \beta = \frac{1}{2}, \qquad C_1 = 20.$$

Let $c, \epsilon, \lambda, \lambda'$ be fixed positive rational numbers satisfying

$$\max\{\tau, \sigma\} < \lambda < 1, \qquad \tau(1 + c/\beta) < \lambda, \qquad \max\{\lambda, \sigma + \beta(1 - \sigma)\} < \lambda' < 1, \qquad \epsilon C_1 < 1.$$

Fix positive constants $a_d \le b_d$ and $a_r \le b_r$, and fix positive constants $C_M, C_{\mathrm{desc}}, C_w$ with $C_w > 1$. For these fixed choices there are a cutoff p_0, one fixed finite-alphabet multitape procedure, and an implicit time constant with the following guarantee. The cutoff and time constant may depend on the fixed rational parameters and these constants. For every choice of positive integers p, d, r with $p \ge p_0$ and

$$a_d p^\epsilon \le d \le b_d p^\epsilon, \qquad 2^{a_r p/d} \le r \le 2^{b_r p/d},$$

put $K = \lfloor d^c \rfloor$, and let $1 \le D \le d$, $P, S \ge 1$, and $0 \le \rho < K$ be integers. Let $w \ge 1$ be an integer component width. The input supplies on a descriptor tape the self-delimiting header

$$\mathcal{E} = \Gamma(p)\Gamma(d)\Gamma(r)\Gamma(D)\Gamma(P)\Gamma(S)\Gamma(K)\Gamma(\rho + 1)\Gamma(w),$$

using the integer code from (1). Assume that this header is valid with the stated values and that the accompanying array is in lexicographic order on the complete address set

$$[P] \times [2^K]^D \times [S], \qquad M = PS\, 2^{KD},$$

with the following record format. For every address $x \in [P] \times [2^K]^D \times [S]$, write its record as

$$z(x) = \sum_{j=0}^{r-1} z_{x,j} y^j,$$

stored in coefficient order. For every x and $0 \le j < r$, assume $|z_{x,j}| \le 1$ and $\Re z_{x,j}, \Im z_{x,j} \in 2^{-p}\mathbb{Z}$. Each component is 2^{-p} times a signed w-bit two's-complement integer, written most significant bit first; the real component precedes the imaginary component. Require the pointwise bounds

$$\log_2(2M) \le C_M p, \qquad |\mathcal{E}| \le C_{\mathrm{desc}} p, \qquad p + 2 \le w \le C_w p,$$

where $|\mathcal{E}|$ is the header's bit length. The condition $w \ge p + 2$ allows every permitted disk-grid component, including -1 and 1. The logical bit volume is $V = 2Mrw = \Theta(Mrp)$.

On each of the D consecutive K-bit address chunks, select bit ρ, numbered from the least significant bit, and let $\mathcal{H}$ be the tensor product of

$$H_0 = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

on those selected bits. It acts coefficientwise and fixes every unselected address bit and both spectator fields. Let Q_p truncate the real and imaginary parts of every coefficient toward zero to $2^{-p}\mathbb{Z}$.

The procedure returns $Q_p(\mathcal{H}z)$ in the same array, coefficient order, and component format in $O(Vd^{\lambda'})$ time, uniformly over all these choices of $p, d, r, D, P, S, \rho, w$ and z. Every returned coefficient belongs to the complex unit disk, and, in the maximum coefficient norm,

$$\|Q_p(\mathcal{H}z) - \mathcal{H}z\|_\infty < \sqrt{2}\,2^{-p}.$$

The procedure evaluates the entire layer exactly before this final truncation. Put

$$C_0 = 128m\big(s_{\mathrm{c}} + 64(W_{\mathrm{c}} + m + 1)^3\big)^2,$$

where $m, W_{\mathrm{c}}, s_{\mathrm{c}}$ are the fixed constants of Proposition 8. The procedure uses at most $p + \lceil C_0 d^{C_1}\rceil$ fractional bits and $\lceil C_0 d^{C_1}\rceil + 3$ integer and sign bits per real component internally; these widths are $O(p)$. The time includes address changes, deterministic repair, row padding and its removal, and fixed-tape workspace cleanup. Its tape count is fixed, and its time constant is independent of the allowed array lengths and coefficient values.

Proof. The procedure reads the current parameters and component width from $\mathcal{E}$. Parsing this header and forming the initial collapsed descriptors take polynomial time in p, already charged in the descriptor work above. The phase-frame construction reduces a group of e selected bits to s_{c} children on e/m selected bits. The row split gives each child $1/W_{\mathrm{c}}$ of the parent's logical volume. The packed selected-bit procedure supplies the binary basis changes. We verify that its cost is uniform over the displayed bands for d, r. Put $\Delta = \lceil C_0 d^{C_1}\rceil$. Since $d \le b_d p^\epsilon$ and $\epsilon C_1 < 1$, a common cutoff gives $\Delta \le p$. The guarded record width is then $R_{\mathrm{g}} = 2r(p + 2\Delta + 3) \le 12rp$; also $R_{\mathrm{g}} \ge 2rp$. Padding at most doubles the original record count M, and row splitting only decreases it. Thus a packed child with M_{child} records has the following value A_* of its rectangular width parameter:

$$\begin{aligned} A_* &= \lceil \log_2(2M_{\mathrm{child}}R_{\mathrm{g}})\rceil \\ &\le \lceil \log_2(4MR_{\mathrm{g}})\rceil \le C_A p, \qquad C_A = C_M + b_r + 7. \end{aligned}$$

The last inequality uses $\log_2(2M) \le C_M p$, $\log_2 r \le b_r p/d \le b_r p$, and $\log_2 p \le p$.

Uniformly for large p, the bands give $f \le d \le p$ and

$$K \ge \tfrac{1}{2} a_d^c p^{\epsilon c}, \qquad r \ge 2^{(a_r/b_d)p^{1-\epsilon}}.$$

The first inequality follows from $K = \lfloor d^c \rfloor$ once $d^c \ge 2$. In particular, $K \ge 6$ and $K/\log p \to \infty$, while r dominates every fixed polynomial. Substituting the common bound $A_* \le C_A p$ into the two additional normalized terms in (19) gives

$$\frac{A_*^3}{R_{\mathrm{g}}} \le \frac{C_A^3 p^2}{2r},$$

$$80(f-1)2^{-K}A_*\left(1 + \frac{A_*}{R_{\mathrm{g}}}\right) \le 80C_A p^2 2^{-K}\left(1 + \frac{C_A}{2r}\right).$$

Both bounds tend to zero uniformly over all current allowed d, r. Thus Lemma 13 has the required $O(M_{\mathrm{child}}R_{\mathrm{g}}(L^\tau + 1))$ cost for every packed call, with one common time constant. The row construction and the recurrence analysis above give total time $O(Vd^{\lambda'})$ on fixed tapes, including all padded and auxiliary streams. Equation (18) converts the resulting exact C-layer to $\mathcal{H}$.

The guard-width calculation above bounds both denominator and magnitude growth by $\lceil C_0 d^{C_1}\rceil$. Since $d \le b_d p^\epsilon$ and $\epsilon C_1 < 1$, this is $o(p)$ uniformly over the allowed d, so every intermediate

coefficient is represented exactly with the stated $O(p)$ width. The initial scan removes or adds only redundant leading sign bits and appends the reserved fractional zeros to put each supplied component in this guarded format. To truncate toward zero at the end, shift its absolute numerator right by the number of added fractional bits, restore its sign, and write the result in w bits. These scans cost $O(V)$. Finally, every row of $\mathcal{H}$ has absolute row sum one. Thus $\mathcal{H}$ maps disk arrays to disk arrays, and componentwise truncation preserves that disk and has error less than $\sqrt{2}\,2^{-p}$: truncation cannot increase the absolute value of either component, and changes each by less than 2^{-p}. The truncated numerators have magnitude at most 2^p, so the supplied component width can hold them. □

Making roots of unity into signed shifts

Added explanation Read before synthetic transforms and ring products.

A transform normally multiplies values by roots of unity. In the synthetic representation, a value is itself a short polynomial, stored as its coefficient list. Work in

$$\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1).$$

This means that y^r is replaced by -1. Multiplication by y shifts the coefficient list and negates the coefficient that wraps around.

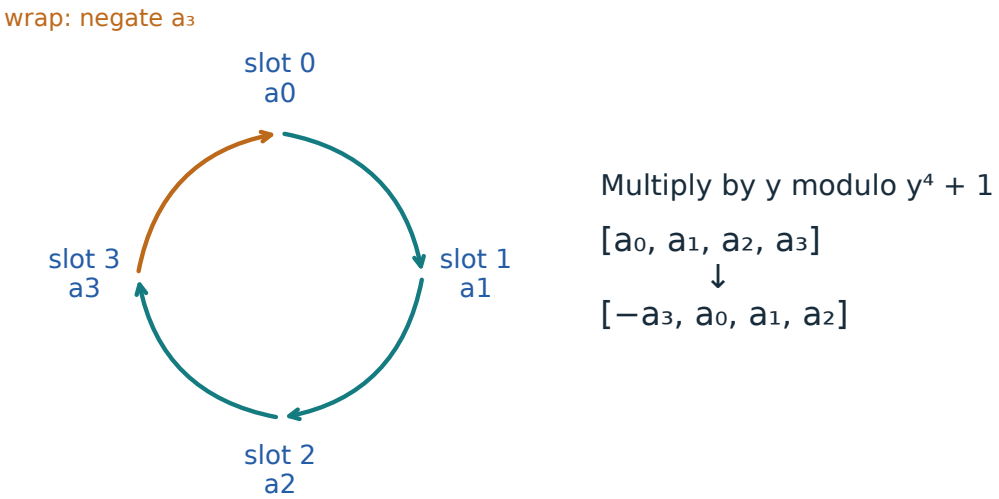


Figure E18. A root operation becomes exact data movement. For $r = 4$, $y(a_0 + a_1 y + a_2 y^2 + a_3 y^3)$ reduces to $-a_3 + a_0 y + a_1 y^2 + a_2 y^3$. No approximate trigonometric multiplication is needed for this operation.

For a power of two t dividing $2r$, the element $\omega_t = y^{2r/t}$ serves as a transform root. Its powers act as signed coefficient permutations. The source proves the needed character cancellation directly. This matters because the quotient ring has zero divisors: dividing by a nonzero-looking ring element is not automatically legal.

Keep the order the algorithm naturally produces. A recursive Fourier transform often produces frequencies in bit-reversed order. For length four, that order is $0, 2, 1, 3$. If both transformed inputs use the same order, corresponding entries can still be multiplied. Sorting each transform into a prettier order would add work without changing the mathematical product. The opposite transform is written to consume the order that is actually present.

Normalization is part of the interface. Both transforms in this manuscript contain a factor $1/t$. Consequently, $F_t^+ F_t^- = I/t$, rather than I. With this convention, for a length-t cyclic convolution,

$$F_t^+\left((F_t^- u) \odot (F_t^- v)\right) = \frac{u * v}{t^2}.$$

One factor of t comes from the forward product normalization, and the other from the normalized opposite transform. Losing either scale would destroy exact recovery. Multidimensional versions replace t by the product of axis lengths, subject to the source's retained permutations.

Why polynomial multiplication does not make the proof circular. Pointwise entries are now polynomials, so their products still need computation. The source packs bounded signed coefficients into larger integers with enough separation to recover each coefficient. It handles centered extraction, signs, and negacyclic wrapping explicitly. For these shorter integers it invokes the already established $O(N \log N)$ multiplier, not the improved algorithm it is trying to establish. The input length N of those calls differs from the original n, and all calls are charged in the final cost table.

What to carry forward. A representation change can replace expensive scalar root multiplications by exact signed shifts, but layout and scaling still cost work.

Original source anchors: `06-transforms.tex, lem:synthetic-characters; lem:signed-ring-product`.

Original manuscript continues below.

6 Synthetic transforms and their tape layout

We now implement the normalized Fourier transforms needed for convolution. Their entries will be short polynomials in a quotient where the required roots of unity act by exact signed permutations of coefficient lists. We also specify where each frequency entry is stored, because the opposite transform consumes that same order after a pointwise product.

The polynomial roots and radix-two butterflies belong to the fast polynomial-transform method of Nussbaumer and Quandalle [21, 20]; we use the form developed in [15, Section 2.4 and Lemma 3.2]. The work here is to apply the simultaneous layers on a suitable tape layout and retain the resulting frequency order through convolution.

For an integer $r \geq 2$ that is a power of two, put

$$\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1), \qquad \left\| \sum_{k=0}^{r-1} z_k y^k \right\|_\infty = \max_{0 \leq k < r} |z_k|.$$

We always use representatives of degree less than r. Multiplication by y^a, for any integer a, permutes the coefficients and changes some signs; consequently it is an isometry for this norm. For arrays over $\mathcal{R}_r$, the same notation denotes the maximum over all coefficients in all entries. A disk array

has norm at most one. A p-bit grid array has real and imaginary coefficient parts in $2^{-p}\mathbb{Z}$. For any array, let Q_p truncate each real and imaginary coefficient part toward zero to this grid. Then

$$\|Q_p z - z\|_\infty < \sqrt{2}\,2^{-p}, \qquad \|Q_p z\|_\infty \le \|z\|_\infty. \tag{26}$$

6.1 Roots, butterflies, and stored frequency order

For a power of two t dividing $2r$, define $\omega_t = y^{2r/t}$. For $z \in \mathcal{R}_r^t$ and $0 \le j < t$, set

$$(F_t^- z)_j = \frac{1}{t}\sum_{k=0}^{t-1} \omega_t^{-jk} z_k, \qquad (F_t^+ z)_j = \frac{1}{t}\sum_{k=0}^{t-1} \omega_t^{jk} z_k.$$

Every power of ω_t is an isometry, and each transform entry is an average of t such images. Thus both transforms are contractions. The ring has zero divisors, so we verify character cancellation directly instead of dividing by $\omega_t^j - 1$.

Lemma 15 (Synthetic character cancellation)**.** *If j is an integer, then*

$$\sum_{k=0}^{t-1} \omega_t^{jk} = \begin{cases} t, & t \mid j, \\ 0, & t \nmid j. \end{cases} \qquad F_t^+ F_t^- = t^{-1} I.$$

Proof. Only the second case needs proof. Write $j = 2^v j_0$ with j_0 odd and $0 \le v < \log_2 t$. Replacing k by $k + t/2^{v+1}$ modulo t permutes the summands and multiplies their sum by $y^{rj_0} = -1$. Hence the sum is zero, since the additive group of $\mathcal{R}_r$ has no two-torsion. Expanding the composite transforms and applying this identity proves the final formula. □

Write

$$H_0 = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

The length-one procedure is the identity. For $t \ge 2$, the decimation-in-frequency step pairs z_k with $z_{k+t/2}$ for $0 \le k < t/2$. The two outputs of H_0 occupy these same slots, whose high address bit is $b = 0$ or $b = 1$, respectively. Multiply the $b = 1$ output by ω_t^{-k}, and then apply the length-$t/2$ procedure separately within the two branches. This computes F_t^- because, for $b \in \{0, 1\}$,

$$(F_t^- z)_{2h+b} = \frac{1}{t/2}\sum_{k=0}^{t/2-1} (\omega_t^2)^{-hk} \omega_t^{-bk} \frac{z_k + (-1)^b z_{k+t/2}}{2}. \tag{27}$$

To follow the recursion, name the original address-bit slots of an axis of length $t = 2^a$ by $a-1, \dots, 0$, from most to least significant. For $0 \le s \le a$, after s levels the values of the first s named slots are branch bits $b_0, \dots, b_{s-1}$. Equation (27) identifies these as frequency bits $0, \dots, s-1$, numbered from the least significant bit. The remaining slots encode an index in $[0, u)$, where $u = t/2^s$. When $s < a$, the next level uses slot $h = a - 1 - s$ to distinguish the paired inputs and then to hold b_s. Writing x_v for the bit in a still unprocessed slot v, the index used by its twiddle is

$$k = \sum_{v=0}^{h-1} 2^v x_v, \qquad 0 \le k < u/2.$$

Previously written branch bits do not enter k. The same description holds in each shorter branch by (27), which proves the assertion inductively.

After all a rounds, the named slots from high to low therefore contain the frequency bits from low to high. If $\mathrm{rev}_a(j)$ reverses the a binary digits of j, the entry $(F_t^- z)_j$ occupies slot $\mathrm{rev}_a(j)$. Let B_t denote this record permutation: it moves the entry at j to $\mathrm{rev}_a(j)$. The recursion writes its outputs directly in this order; it does not move whole frequency indices.

The procedure consuming this stored order traverses the recursion in reverse: first process the two smaller branches, then apply the inverse monomial to the second branch, and finally apply H_0. Each forward level is invertible, since both its butterfly and its monomial are invertible. As $H_0^2 = I/2$, the reversed level is half the inverse of the corresponding forward level. There are $a = \log_2 t$ levels, so the reverse procedure composed with $B_t F_t^-$ is $2^{-a} I = I/t$. Lemma 15 gives the same composite for $F_t^+ B_t^{-1}$ with $B_t F_t^-$. The forward procedure is invertible, so these two reverse operators agree. Thus the two exact procedures implement

$$B_t F_t^-, \qquad F_t^+ B_t^{-1}, \tag{28}$$

respectively.

For D axes of lengths $t_1, \ldots, t_D$, put

$$M = \prod_{i=1}^{D} t_i, \qquad F_{\boldsymbol{t}}^{\pm} = \bigotimes_{i=1}^{D} F_{t_i}^{\pm}, \qquad B = \bigotimes_{i=1}^{D} B_{t_i}.$$

A round applies one current H_0 on each participating axis. The twiddle for axis i depends only on that axis's address bits. The two entries paired by another axis's butterfly have the same axis-i address, so that twiddle commutes with the other butterfly. Consequently the individual axis steps may be grouped as a parallel butterfly $\mathcal{H}$, followed by one recordwise monomial operation J_E. At each address, J_E multiplies the record by

$$y^E, \qquad E = -\sum_{i \text{ participating}} \frac{2r}{u_i} b_i k_i \pmod{2r}. \tag{29}$$

Here u_i is that axis's current recursive length, b_i its newly written branch bit, and $0 \le k_i < u_i/2$ is encoded by its still unprocessed named slots. The exponent depends on the address, not on coefficient values. In the opposite direction the grouped round is $\mathcal{H} J_E^{-1}$: apply y^{-E} first and the parallel butterfly second. Each $\mathcal{H}$ is a contraction and each J_E is an isometry, so every completed round is a contraction.

6.2 A layout on which the parallel layers apply

To apply Proposition 14 in a transform round, we must place the selected bits of the participating axes at a common offset in consecutive K-bit chunks. We now construct that layout, keeping each polynomial's coefficients in a contiguous suffix. The layout uses the single-bit moves of Lemma 3 and the chunk interchanges of Lemma 12; each movement is included in the transform cost. The layer then supplies one truncated contraction per round, and its temporary rows and work streams are removed before the next round begins.

Lemma 16 (Transform layout and cost)**.** *Fix the rational parameters and positive comparison constants $a_d \le b_d$, $a_r \le b_r$ of Proposition 14. Fix constants $C_\ell > 0$ and $C_w > 1$, and choose the layer constants once as*

$$C_M = C_\ell + 1, \qquad C_{\mathrm{desc}} = 32(C_\ell + C_w + 1).$$

Use these constants and C_w in Proposition 14. These fixed choices determine a cutoff p_1, one fixed-tape procedure, and an implicit time constant with the following guarantee. For every choice of positive integers p, d, r with $p \ge p_1$ and

$$a_d p^{\epsilon} \le d \le b_d p^{\epsilon}, \qquad 2^{a_r p/d} \le r \le 2^{b_r p/d},$$

suppose $d \ge 2$ and $r = 2^{\ell}$ with an integer $2 \le \ell < p$, and require

$$d\ell \le C_\ell p, \qquad 1 \le K = \lfloor d^c \rfloor \le \ell - 1.$$

Let $t_i \in \{2^{\ell-1}, 2^{\ell}\}$ for $1 \le i \le d-1$, put $M = \prod_{i=1}^{d-1} t_i$ and $T = rM$, and let w be an integer with $p + 2 \le w \le C_w p$. The cutoff and time constant are independent of p, d, r, these axis choices, the round headers constructed from them, and w within the displayed bounds.

The input supplies the self-delimiting header

$$\mathcal{T} = \Gamma(p)\Gamma(d)\Gamma(r)\Gamma(K)\Gamma(w) \prod_{i=1}^{d-1} \Gamma(t_i),$$

where the product is concatenation in axis order and Γ is the integer code of (1). The supplied d determines the number of axis entries, and $\ell = \log_2 r$ is obtained from the supplied power of two r. Assume that the header is valid with the stated values and that the input records use the two's-complement component format of Proposition 14 with width w.

The procedure constructs an explicit positional layout permutation L of record addresses, preserving their polynomial suffixes, and transforms disk grid arrays of M contiguous degree-less-than-r polynomial records. Its exact counterpart is $LBF_{\boldsymbol{t}}^{-}$. Its opposite procedure consumes that order and has exact counterpart $F_{\boldsymbol{t}}^{+} B^{-1} L^{-1}$. Each procedure has error less than $\sqrt{2}\ell\, 2^{-p}$, returns disk grid coefficients in the same component format, and costs

$$O\left(Tp\left[dK + pK^{\tau-1} + \ell d^{\lambda'}\right]\right). \tag{30}$$

Both layout construction and final restoration are charged in this bound. The forward procedure may retain its output layout for a pointwise product and the following opposite procedure.

Proof. Put $D = d - 1$ and write $a_i = \log_2 t_i$. The supplied encoding has $2w = \Theta(p)$ bits per coefficient, so the logical bit volume is $\Theta(Tp)$. Call an axis long if $a_i = \ell$ and short if $a_i = \ell - 1$. Name the original bit slots of axis i by (i, h), $a_i - 1 \ge h \ge 0$. Initially these slots are in axis order and, within each axis, in decreasing h. The coefficients of each polynomial form a contiguous suffix. Put

$$b = (\ell - 1) \bmod K, \qquad q = \lfloor (\ell - 1)/K \rfloor.$$

Form a list of the slots to move to a prefix. First list $(i, \ell - 1)$ for the length-2^{ℓ} axes in increasing i. Then list (i, h) for $h = b - 1, \dots, 0$, and within each h in increasing i; this second list is empty if $b = 0$. Move each listed slot immediately after the prefix already formed, leaving all other slots in their relative order. There are at most $D(b+1) = O(dK)$ listed slots, so the moves cost $O(TpdK)$. A descriptor records the physical position of every named slot. The part of each axis left outside the prefix now consists of q consecutive K-bit chunks.

Number the axes by i and their chunks, from high to low, by j. The remaining physical order changes from

$$(1,1), (1,2), \dots, (1,q), (2,1), \dots, (D,q) \quad \text{to} \quad (1,1), (2,1), \dots, (D,1), (1,2), \dots, (D,q).$$

To obtain this order, scan its target positions from left to right. At each position except the last, exchange the chunk currently there with the required chunk if they differ. This uses at most $Dq-1$ exchanges of width-K address chunks, even when the chunks are nonadjacent. Each costs $O(TpK^\tau)$, so all these exchanges cost

$$O(Tp\,DqK^\tau) = O(Tp\,pK^{\tau-1}).$$

These prefix moves and chunk exchanges define L. We use the same letter for its induced record permutation: a record at address x moves to address Lx, while its polynomial suffix stays intact. Recording the inverse operations in reverse order gives L^{-1} with the same cost.

For a small illustration of this positional permutation, take $\ell = 6$, $K = 2$, and one long axis followed by one short axis. Then $b = 1$ and $q = 2$. Write $[i : h, h']$ for the chunk containing the two named slots $(i, h), (i, h')$, in that order. After L the slots are

$$\underbrace{(1,5),(1,0),(2,0);}_{\text{prefix}} \qquad [1:4,3]\ [2:4,3]\ [1:2,1]\ [2:2,1].$$

For example, the round at named position $h = 4$ uses offset one in each of the first two chunks. It writes frequency bit one on the long axis and frequency bit zero on the short axis. The named position specifies the same physical pairing rule on both axes even though the frequency significance differs. This example illustrates only the slot permutation; the parameter bounds of the lemma govern its asymptotic use.

The header has $O(p)$ bits: its axis entries use $O(\sum_i a_i + d) = O(p)$ bits, and its remaining entries have $O(p)$ bits in total. The layout lists have binary length polynomial in p. Parsing the header and generating these lists by elementary integer arithmetic take $\mathrm{poly}(p) = O(Tp)$ time. Indeed the fixed comparison bands give $T \geq r \geq 2^{(a_r/b_d)p^{1-\epsilon}}$, uniformly over the permitted d, r, so T dominates every fixed power of p.

First process the round at named position $h = \ell - 1$ on the long axes, if there are any. Every axis then has current recursive length $2^{\ell-1}$. Next process $h = \ell - 2, \ldots, b$ in the qK rounds on the remaining chunks, and finally the bottom positions $h = b-1, \ldots, 0$, if $b > 0$. Thus all participating axes in a common round have current length $u_i = 2^{h+1}$. The branch bit written at position h is frequency bit $a_i - 1 - h$; its frequency significance can differ between long and short axes. The descriptor locates that named slot after L, so the physical move has not changed which pair the current butterfly uses. The opposite procedure executes this schedule in reverse order and, when present, ends with the extra leading round on the long axes.

For a position h in the remaining chunks, the selected bit has offset $(h - b) \bmod K$, counted from the least significant bit, in each of the D consecutive chunks with the same chunk index j. Let n_P and n_S be the numbers of record-address bits before and after that group. Every combination of those bits occurs, so the layer sees the complete address set

$$[P] \times [2^K]^D \times [S], \qquad P = 2^{n_P}, \quad S = 2^{n_S}, \quad n_P + DK + n_S = \sum_i a_i \leq D\ell = O(p).$$

Thus its record count is $PS2^{DK} = M$ and $\log_2(2M) \leq (C_\ell + 1)p = C_M p$. The current transform parameters supply p, d, r, K, w, and the round determines D, P, S and the selected offset ρ. Encode these values as the header $\mathcal{E}$ of Proposition 14. The same fixed descriptor bound covers every axis choice and round. Indeed $|\Gamma(v)| \leq 2\log_2 v + 1$, and $\ell + n_P + n_S \leq d\ell \leq C_\ell p$. Also $d, D \leq \max\{C_\ell, 1\}p$, $K, \rho + 1 \leq p$, and $w \leq C_w p$. Adding the nine code lengths therefore gives

$$|\mathcal{E}| \leq 2C_\ell p + 12\log_2 p + 4\log_2 \max\{C_\ell, 1\} + 2\log_2 C_w + 9 \leq C_{\mathrm{desc}} p.$$

The last inequality uses $p \geq 1$, $\log_2 p \leq p$, and $\log_2 x \leq x$ for $x \geq 1$. This is the descriptor passed to the layer; the longer table locating named slots remains separate for the transform's address calculations. The simultaneous-layer procedure therefore applies at cost $O(Tpd^{\lambda'})$ per round and $O(Tp\ell d^{\lambda'})$ in total.

In a round on prefix slots, process the participating bits individually. Temporarily move one such bit immediately before the polynomial suffix. In each now adjacent pair, buffer the first record and scan it with the second, writing the half-sums and half-differences to two work tapes. Append the half-sum record and then the half-difference record, and reverse the bit move after all pairs are processed. Including head returns, the work is $O(rp)$ per pair and $O(Tp)$ for the moves and scans at coefficient width $O(p)$. That width suffices without truncating between axes: after $s \leq D$ individual H_0 factors, the coefficients are exact dyadics on the $2^{-(p+s)}$ grid and still lie in the disk. A temporary sum has modulus at most two and needs only a constant number of extra integer bits. Hence $p + D + O(1) = O(p)$ bits per component suffice, since $D \leq d$ and $d\ell = O(p)$. Complete the combined monomial exactly and apply Q_p once at the end of the round. Write the resulting disk-grid numerators in the supplied w-bit component format. Their magnitudes are at most 2^p, so this takes one linear scan within the same bound. In an opposite round on prefix slots the inverse monomial comes first and preserves the grid and disk, so the same width bound holds for its individual factors. There are at most $b + 1 \leq K$ rounds on prefix slots, giving cost $O(TpdK)$.

For each output record of a forward butterfly round, the descriptor locates the new branch bit b_i and the lower named slots that encode k_i. Read their current values to compute (29). Before an opposite round at position h, all lower positions have already been processed in reverse. Those slots again index the lower inputs of that level, while slot h still gives b_i. They therefore encode the same k_i needed by the inverse monomial before its butterfly. The binary descriptions of the addresses, exponent, and layout have polynomial length in p. Ordinary fixed-tape arithmetic therefore computes the exponent in $\mathrm{poly}(p)$ time per polynomial record. Each record contains $\Theta(rp)$ bits, and r dominates every fixed power of p, so this work fits within its scan cost. The same bound covers scans of the polynomially many round and layout descriptors. Growing lists reside on tapes; a fixed number of counter and work tapes suffices for every number of axes.

To implement the monomial, write $E = er + s$ with $0 \leq s < r$. Split the coefficient list before position $r - s$, put its tail before its head, negate the moved tail, and apply the common sign $(-1)^e$. This is multiplication by y^E modulo $y^r + 1$. There are always two pieces. Splitting them to two tapes and merging in that order costs $O(rp)$ per record, including head returns; $s = 0$ is an ordinary signed copy. The inverse twiddle uses the same split and merge with exponent $-E$, and no coefficient sort is required.

For a forward round on a group of D chunks, the layer call returns $Q_p(\mathcal{H}z)$ and the next operation is J_E. Truncation toward zero is odd and acts separately on every real and imaginary part. Since J_E is a signed coefficient permutation, it follows that

$$J_E Q_p(\mathcal{H}z) = Q_p(J_E \mathcal{H}z).$$

Thus the actual forward call sequence returns exactly the result of truncating once after the exact contraction round. In the opposite direction, J_E^{-1} first preserves the disk grid, and the layer call returns $Q_p(\mathcal{H}J_E^{-1}z)$ directly. The rounds on prefix slots have the same forms, with one truncation, by their exact evaluation above. A contraction does not increase the norm of the difference between the computed and exact arrays; the single truncation then adds less than $\sqrt{2}\,2^{-p}$ by (26). Both the contraction and truncation preserve the disk. Starting with zero error, at most ℓ rounds therefore give the claimed error and disk bounds. The layout permutations are isometries and add no numerical error.

It remains to identify the order of the full procedures. Before relocating slots, write $\mathcal{D}_- = BF_{\boldsymbol{t}}^-$ and $\mathcal{D}_+ = F_{\boldsymbol{t}}^+ B^{-1}$ for the exact logical forward and opposite procedures. Executing each operation on named slots after L conjugates it by L. The forward procedure first moves the records by L, whereas the opposite procedure restores them by L^{-1} after its reverse arithmetic. Their full exact operators are

$$(L\mathcal{D}_-L^{-1})L = LBF_{\boldsymbol{t}}^-, \qquad L^{-1}(L\mathcal{D}_+L^{-1}) = F_{\boldsymbol{t}}^+ B^{-1}L^{-1}.$$

In particular, the forward entry with frequency address j is stored at $L(Bj)$. The reverse arithmetic consumes the frequency branch coordinates and outputs the normalized opposite transform indexed by the original axis coordinates, which are still in layout L. The factor B^{-1} is realized by that arithmetic; the final charged movement L^{-1} restores axis-major physical order. □

Pointwise multiplication commutes with applying the same record permutation LB to both operands. We may therefore multiply the forward outputs in their retained order and pass that order directly to the opposite procedure. The following polynomial product implements this operation with an established integer multiplier.

7 Exact normalized polynomial products

To multiply the polynomial records, we use the fixed-tape integer multiplier of Harvey and van der Hoeven [15, Theorem 1.1], with cost $O(N\log(2N))$ on N-bit inputs. This is an independent subroutine. Its inputs below have $O(rp)$ bits, and we include the conversions between coefficient streams and signed integers in the cost. The reduction is a signed form of Kronecker substitution; compare [15, Lemma 2.5]. We specify the packing and recovery of coefficients to include negative values and the required tape format.

Lemma 17 (Signed packing and normalized ring multiplication)**.** *Let $r = 2^\ell < 2^p$ with integers $\ell \geq 1$ and $p \geq 2$, and fix $C_w > 1$. Let w be an integer with $p + 2 \leq w \leq C_w p$. The input supplies $\Gamma(p)\Gamma(r)\Gamma(w)$ and two disk grid polynomials $f, g \in \mathcal{R}_r$ in the component format of Proposition 14 with width w. One can compute the exact Gaussian integer numerators of fg, and then return $Q_p(fg/r)$ in that same format, in time $O(rp\log(rp))$ on a fixed number of tapes. The output is a disk grid polynomial with error less than $\sqrt{2}\,2^{-p}$ from fg/r.*

Proof. The supplied power of two r determines $\ell = \log_2 r$. Write

$$f = 2^{-p}(a + ib), \qquad g = 2^{-p}(c + ie),$$

where $a, b, c, e \in \mathbb{Z}[y]$ have degree less than r and coefficients of absolute value at most 2^p. Set $B_0 = 2^{4p}$ and, for $u \in \{a, b, c, e\}$, define the signed evaluation integer

$$A_u = u(B_0) = \sum_{j=0}^{r-1} u_j B_0^j.$$

The bound

$$|A_u| \leq 2^p \frac{B_0^r - 1}{B_0 - 1} < B_0^r/2$$

shows that each magnitude fits in $4pr$ bits. Multiply the four pairs of absolute values using the established multiplier and restore each product's sign. The resulting integers are $A_aA_c, A_bA_e, A_aA_e, A_bA_c$. For example, if $h = ac$ is the ordinary product in $\mathbb{Z}[y]$, then

$$A_aA_c = (ac)(B_0) = \sum_{j=0}^{2r-2} h_j B_0^j.$$

The other three evaluation products have the same form. Every coefficient in any of these ordinary polynomial products satisfies

$$|h_j| \le r2^{2p} < 2^{3p} < B_0/2. \tag{31}$$

After the lower coefficients have been removed from an evaluation, this strict bound identifies the next coefficient by its centered residue modulo B_0.

To form A_u, start with carry $\gamma_0 = 0$ and write

$$d_j \equiv u_j + \gamma_j \pmod{B_0}, \quad 0 \le d_j < B_0, \qquad \gamma_{j+1} = \frac{u_j + \gamma_j - d_j}{B_0}.$$

Since $-B_0 < u_j + \gamma_j < B_0$ whenever $\gamma_j \in \{-1, 0\}$, the carry stays in $\{-1, 0\}$. Telescoping the recurrence gives

$$A_u = \sum_{j=0}^{r-1} d_j B_0^j + \gamma_r B_0^r.$$

The r blocks are therefore a fixed-width two's-complement encoding of A_u, with $\gamma_r = -1$ exactly when $A_u < 0$. In that case complement and increment the blocks to obtain $|A_u|$; otherwise retain them. Every block has $4p$ bits and every carry has constant size, so a constant number of scans costs $O(rp)$. Endianness changes required by the multiplier are linear-time reversals on a work tape.

Now let Z be one of the four signed product integers, and let h be its corresponding ordinary polynomial product. Put $J = 2r$ and pad that product by $h_{J-1} = 0$. The bound on the evaluation integers gives $|Z| < B_0^J/4$, so Z fits in J blocks of width $4p$. Convert its sign and magnitude to sign-extended two's complement:

$$Z = \sum_{j=0}^{J-1} q_j B_0^j + \varepsilon B_0^J, \qquad 0 \le q_j < B_0, \quad \varepsilon \in \{-1, 0\}.$$

Zero-fill the unused high blocks; for a negative sign, complement and increment within this fixed width. These conversions take a constant number of scans. Put $\rho_0 = 0$. At block j, let $\widehat{h}_j$ be the unique integer in $[-B_0/2, B_0/2)$ congruent to $q_j + \rho_j$ modulo B_0, and put

$$\rho_{j+1} = \frac{q_j + \rho_j - \widehat{h}_j}{B_0}.$$

The carry belongs to $\{0, 1\}$: if ρ_j has this property, then $0 \le q_j + \rho_j \le B_0$, and centering either leaves this value unchanged or subtracts B_0.

The meaning of the carry is the following residual identity, after coefficients below j have been correctly extracted:

$$\sum_{k=j}^{J-1} h_k B_0^{k-j} = \sum_{k=j}^{J-1} q_k B_0^{k-j} + \rho_j + \varepsilon B_0^{J-j}.$$

At $j = 0$ this is the equality between the two representations of $Z = h(B_0)$, with $\rho_0 = 0$. For $j < J$, reduction modulo B_0 gives $h_j \equiv q_j + \rho_j \pmod{B_0}$. By (31), including the padded value $h_{J-1} = 0$, the unique centered representative is h_j itself. Thus $\widehat{h}_j = h_j$. Subtracting this coefficient and dividing the residual identity by B_0 gives the next identity, with exactly the displayed update for ρ_{j+1}. At $j = J$ the identity is $0 = \rho_J + \varepsilon$. Hence a negative product has final carry 1, canceled by its sign extension $\varepsilon = -1$; a nonnegative product has final carry zero. The machine implements

the induction by reading one block and updating this constant-size carry at each step, so extraction costs $O(rp)$.

The real and imaginary numerator streams are respectively $ac - be$ and $ae + bc$. To reduce them modulo $y^r + 1$, subtract the coefficient at $j + r$ from the one at j for $0 \le j < r$, treating a missing coefficient as zero. Splitting the two halves to tapes aligns these terms in one further scan. For each first input index $j_1 \in \{0, \dots, r-1\}$, exactly one second index j_2 in that range satisfies $j_1 + j_2 \equiv j \pmod r$. Its sign is positive when $j_1 + j_2 = j$ and negative when $j_1 + j_2 = j + r$. Thus each resulting Gaussian integer coefficient H_j is a sum of exactly r signed products of Gaussian integer input coefficients of modulus at most 2^p. Consequently

$$|H_j| \le r2^{2p}.$$

The same triangle inequality before imposing the disk bound gives $\|fg/r\|_\infty \le \|f\|_\infty \|g\|_\infty$; in particular the normalized product of disk polynomials lies in the disk. All intermediate coefficient additions fit in $O(p)$ bits. The width-$4p$ packing fields and the exact multiplier output prevent overflow or ambiguity between adjacent fields.

Finally the p-bit grid numerator of the normalized product is

$$\operatorname{trunc}_0\left(\frac{\Re H_j}{2^{p+\ell}}\right) + i \operatorname{trunc}_0\left(\frac{\Im H_j}{2^{p+\ell}}\right).$$

To implement it, shift the absolute magnitude right by $p + \ell$ bits and restore its sign, at cost $O(p)$ per coefficient. The returned coefficient is 2^{-p} times this numerator, namely $Q_p(2^{-2p}H_j/r)$; (26) proves its error and disk bounds. Each returned component numerator has magnitude at most 2^p, so write it as a signed w-bit two's-complement word, most significant bit first, with the real word before the imaginary word and coefficient indices in increasing order. This costs $O(rw) = O(rp)$ and supplies the format used by the opposite transform. Four exact integer products dominate the linear packing, extraction, combination, wrapping, and truncation costs. □

7.1 The normalized convolution interface

For arrays on $\prod_i \mathbb{Z}/t_i\mathbb{Z}$ with values in $\mathcal{R}_r$, let $*$ denote cyclic convolution in the displayed axes and define

$$\mathcal{M}_r(f, g) = \frac{f * g}{rM} = \frac{f * g}{T}.$$

For a fixed output coefficient, each of the M first cyclic addresses determines one second address, and each of the r first polynomial indices determines one second index modulo r. The negacyclic wrap supplies the sign. Thus that coefficient averages rM signed scalar products, and $\mathcal{M}_r$ is a bilinear contraction:

$$\|\mathcal{M}_r(f, g)\|_\infty \le \|f\|_\infty \|g\|_\infty.$$

If the other operand is a disk array, replacing one operand changes the output by at most the norm distance between its two values.

Proposition 18 (Convolution from the retained transform layout)**.** *Under the hypotheses of Lemma 16, $\mathcal{M}_r$ on disk grid operands has an approximation with error less than*

$$\sqrt{2}\, M(3\ell + 1)2^{-p} \tag{32}$$

and cost

$$O\left(Tp\left[dK + pK^{\tau-1} + \ell d^{\lambda'} + \log(rp)\right]\right).$$

Put $m_M = \log_2 M$ and $w_M = w + m_M$. The returned grid array is in the original array and coefficient order, with the same two's-complement component format at width w_M. The procedure supplies $\Gamma(w_M)$ on the output descriptor tape, so the caller knows every component boundary from this width and the retained shape parameters. If one operand has norm at most $\rho < 1$ and the bound in (32) is at most $1 - \rho$, the returned grid array is a disk array.

Proof. Perform both forward transforms and retain their common LB order. Pass the supplied p, r, w to Lemma 17 for corresponding polynomial records. Its output keeps that record order and uses the component format required by the opposite procedure. Apply the opposite procedure to that order, and multiply its result exactly by $M = T/r$.

To check normalization, expansion of the finite sums gives

$$F_{\boldsymbol{t}}^{-}(f * g) = M(F_{\boldsymbol{t}}^{-} f)(F_{\boldsymbol{t}}^{-} g).$$

As $F_{\boldsymbol{t}}^{+} F_{\boldsymbol{t}}^{-} = I/M$, it follows that

$$M F_{\boldsymbol{t}}^{+}\left(\frac{(F_{\boldsymbol{t}}^{-} f)(F_{\boldsymbol{t}}^{-} g)}{r}\right) = \frac{f * g}{rM}.$$

Let $P(u, v)$ denote the exact pointwise product uv/r, and put $\mathcal{T}_+ = F_{\boldsymbol{t}}^{+} B^{-1} L^{-1}$. Since LB permutes records and leaves their polynomial suffixes intact, $P(LBu, LBv) = LBP(u, v)$. Thus $\mathcal{T}_+$ applied to the pointwise product of the exact stored forward outputs equals $F_{\boldsymbol{t}}^{+} P(F_{\boldsymbol{t}}^{-} f, F_{\boldsymbol{t}}^{-} g)$. The displayed normalization identity identifies its exact scaling by M with $\mathcal{M}_r(f, g)$.

For the error estimate, put $\delta = \sqrt{2}\ell\, 2^{-p}$ and $\eta = \sqrt{2}\, 2^{-p}$. Let u, v be the exact stored forward outputs and $\widetilde{u}, \widetilde{v}$ the returned ones. All four are disk arrays, and each forward error is less than δ. The exact map P is bilinear and satisfies $\|P(x, y)\|_\infty \le \|x\|_\infty \|y\|_\infty$ by the coefficient bound in Lemma 17. Hence

$$\begin{aligned} P(\widetilde{u}, \widetilde{v}) - P(u, v) &= P(\widetilde{u} - u, \widetilde{v}) + P(u, \widetilde{v} - v), \\ \|P(\widetilde{u}, \widetilde{v}) - P(u, v)\|_\infty &< 2\delta, \end{aligned}$$

where the first term uses the disk bound on $\widetilde{v}$ and the second uses it on u. Write $\widetilde{z} = Q_p(P(\widetilde{u}, \widetilde{v}))$ and $z_* = P(u, v)$. The product lemma makes $\widetilde{z}$ a disk grid array and gives $\|\widetilde{z} - z_*\|_\infty < 2\delta + \eta$. If $\widetilde{h}$ is the returned opposite output, its approximation bound applies to $\widetilde{z}$. Since $\mathcal{T}_+$ is a contraction,

$$\|\widetilde{h} - \mathcal{T}_+ z_*\|_\infty \le \|\widetilde{h} - \mathcal{T}_+ \widetilde{z}\|_\infty + \|\mathcal{T}_+(\widetilde{z} - z_*)\|_\infty < 3\delta + \eta.$$

The final exact multiplication by M therefore gives error less than $M(3\delta + \eta) = \sqrt{2}\, M(3\ell + 1)2^{-p}$, as claimed. If one original operand has norm at most ρ, then $\|\mathcal{M}_r(f, g)\|_\infty \le \rho$. Under the stated error margin, the returned norm is less than $\rho + (1 - \rho) = 1$.

Every computed output before the final scaling is a disk grid array. The layer's internal dyadic values and the integer product numerators use the larger exact widths stated in their respective procedures. The scale $M = 2^{m_M}$ is a power of two, so it preserves the p-bit grid. Compute $m_M = \sum_i \log_2 t_i$ from the supplied axis lengths. For each signed w-bit numerator returned by the opposite procedure, append exactly m_M zero bits at its least significant end. This is the exact two's-complement encoding of the scaled numerator in $w_M = w + m_M$ bits, including for negative values. The real word still precedes the imaginary word and the coefficient order is unchanged. Since $w \le C_w p$ and $m_M \le C_\ell p$, this known output width is $O(p)$. Writing $\Gamma(w_M)$ takes $O(p)$ time.

There are $M = T/r$ pointwise polynomial products. Their total cost is $O(Tp \log(rp))$; the three transform costs are as stated, and the last exact scaling is linear in Tp. All data movement uses the previously specified scans, swaps, and layout restoration. □

Changing the grid without sorting everything

Added explanation Read before Gaussian resampling.

The natural convolution layout and the efficient synthetic-transform layout do not have the same axis lengths. The multiplication reduction uses pairwise coprime source axes; synthetic transforms prefer powers of two. Gaussian resampling connects them through carefully controlled linear maps. This is an analytic bridge, not ordinary rounding of an index to its nearest neighbor.

In one dimension the source gives an identity of the form

$$P_s F_s = 2^{2\alpha^2} B_0 P_t F_t A.$$

Here A maps from the source grid of length s to a longer grid of length t; B_0 maps back. The P operators are coordinate permutations. The maps are implemented approximately at a stated precision, with explicit error and time bounds. The large known scalar is accounted for in recovery. The inequalities relating s, t, α, p are assumptions of this interface, not decoration that can be dropped when changing parameters.

Why leaving a permutation in a formula can save time. Let P reorder coordinates. If two arrays use the same reordering, then

$$(PA) \odot (PB) = P(A \odot B).$$

Corresponding frequencies are still paired correctly. A consumer that knows the order can use them directly. The simple identity below explains the opportunity; the source proves the more specific identities needed to make all its source and target permutations compatible with the opposite transform.

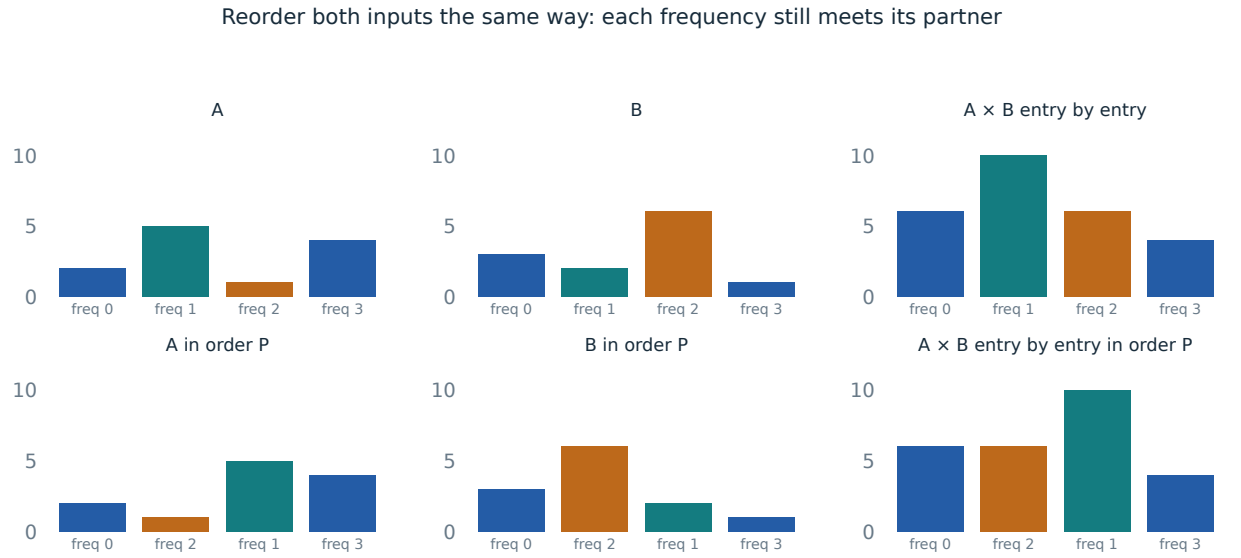


Figure E19. Consistency can remove a needless reorder. The bars show the same permutation on both concrete factors and their pointwise product $[6, 10, 6, 4]$. The diagram alone does not prove that an arbitrary inverse-transform implementation accepts that order. Its interface must establish this.

Where the chirp identity fits. Bluestein's identity rewrites the bilinear exponent using $2jk = j^2 + k^2 - (j-k)^2$. Quadratic phase factors on the inputs and outputs turn a transform into a

convolution with a known chirp. The original proof keeps the relevant permutation inside that calculation. A further phase twist reconciles cyclic convolution with the negacyclic polynomial relation $y^r = -1$. Each twist and its removal has both an algebraic and a precision cost.

The error budget has two different kinds of growth. For a composition of exact contractions, existing error is not amplified, so local approximation errors can be added. However, undoing a known large normalization multiplies absolute error. It is therefore wrong to conclude "all transforms are contractive, so the final answer is accurate" without tracking the rescaling. The source chooses sufficient working precision to survive both accumulated local error and the final amplification.

What to carry forward. A layout is part of an algorithm's input and output contract. Preserving a useful order is valuable only when every later step accepts it.

Original source anchors: `07-resampling.tex, lem:no-sort-resampling; 08-assembly.tex`.

Original manuscript continues below.

8 Resampling with the permutations left in the transform

The transform lengths needed for the integer product are pairwise coprime, whereas the synthetic transforms use powers of two. Gaussian resampling connects these two shapes. Its usual implementation sorts twice on each coordinate. We retain both permutations in the transform formula and compute that formula directly. This section proves the resulting interface, including its tape cost when the number of coordinates grows.

For an integer $q \geq 1$, write

$$(F_q u)_k = \frac{1}{q} \sum_{j=0}^{q-1} u_j e^{-2\pi i j k/q}, \qquad (F_q^+ u)_k = \frac{1}{q} \sum_{j=0}^{q-1} u_j e^{2\pi i j k/q}.$$

All subscripts on a vector of length q are taken modulo q. The vector norm is the maximum modulus of its coordinates. All operator norms are induced by this norm: $\|L\| = \sup_{\|u\| \leq 1} \|Lu\|$. Each normalized Fourier transform above is a contraction, since each output averages inputs multiplied by complex numbers of modulus one. Coordinate permutations preserve the norm. For an integer precision $p \geq 1$, put

$$\mathbb{D}_p = \{2^{-p}(a + ib) : a, b \in \mathbb{Z},\ a^2 + b^2 \leq 2^{2p}\}.$$

For storage between completed scalar routines, put $w_c = p + 2$. Represent $2^{-p}(a + ib) \in \mathbb{D}_p$ by a signed w_c-bit two's-complement word for a, followed by one for b, with each word written most significant bit first. A coordinate list is the concatenation of these fixed-length records in its stated order. Since $|a|, |b| \leq 2^p < 2^{w_c - 1}$, this format includes both signs of every permitted numerator, including -2^p and 2^p. An approximation $\widetilde{L} : \mathbb{D}_p^a \to \mathbb{D}_p^b$ to a linear contraction $L : \mathbb{C}^a \to \mathbb{C}^b$ has error E if $2^p \|\widetilde{L}u - Lu\| < E$ for every input in $\mathbb{D}_p^a$. The approximation itself need not be linear. Composing such algorithms adds their errors: at each step, insert the exact map at the computed input and use its contraction bound on the accumulated input error.

We use the established $O(N \log N)$ integer multiplier of Harvey and van der Hoeven [15, Theorem 1.1] for the shorter integer records that occur below. We also use their Lemma 2.12: for integer precision $p > 100$ and integers $k > 1$, $0 \leq j < k$, one can compute $\widetilde{\zeta} \in \mathbb{D}_p$ with $2^p |\widetilde{\zeta} - e^{2\pi i j/k}| < 2$ in time $O(\max(p, \log k)^{1+\delta})$, for every fixed $0 < \delta < 1/8$. Throughout this section, numbered citations to that paper refer to its author-hosted 45-page manuscript.

8.1 The one-dimensional interface

Lemma 19 (Resampling without executing the two permutations)**.** *Fix $0 < \delta < 1/8$. Let $p > 100$, $2 \le s < t < 2^p$ and $2 \le \alpha < \sqrt{p}$ be integers, with $\gcd(s,t) = 1$ and*

$$\theta := t/s - 1 > p/\alpha^4.$$

Define coordinate permutations by

$$(P_s u)_j = u_{tj}, \qquad (P_t v)_k = v_{-sk}.$$

There are linear contractions $A : \mathbb{C}^s \to \mathbb{C}^t$ and $B_0 : \mathbb{C}^t \to \mathbb{C}^s$ satisfying

$$P_s F_s = 2^{2\alpha^2} B_0 P_t F_t A. \tag{33}$$

There are fixed multitape algorithms for approximations $\widetilde{A} : \mathbb{D}_p^s \to \mathbb{D}_p^t$ and $\widetilde{B}_0 : \mathbb{D}_p^t \to \mathbb{D}_p^s$, each with error less than p^2 and time

$$O(tp^{3/2+\delta}\alpha).$$

The constant is uniform in p, s, t, α. Inputs and outputs use the scalar component format above and occur in increasing coordinate order.

Proof. The Gaussian resampling argument uses two maps $\mathsf{S}, \mathsf{T} : \mathbb{C}^s \to \mathbb{C}^t$. For $0 \le k < t$, they are

$$(\mathsf{S}u)_k = \frac{1}{\alpha} \sum_{j \in \mathbb{Z}} e^{-\pi \alpha^{-2}(j - sk/t)^2} u_j, \qquad (\mathsf{T}u)_k = \sum_{j \in \mathbb{Z}} e^{-\pi \alpha^2 (tj/s - k)^2} u_j.$$

Both sums converge absolutely. For $0 \le j < s$, define

$$q_j = \left\lfloor \frac{tj}{s} + \frac{1}{2} \right\rfloor, \qquad \beta_j = \frac{tj}{s} - q_j.$$

Thus q_j is an integer nearest to tj/s, with ties rounded upward. Let $C : \mathbb{C}^t \to \mathbb{C}^s$ select these coordinates, and let $\mathsf{D} : \mathbb{C}^s \to \mathbb{C}^s$ be the diagonal map

$$(Cv)_j = v_{q_j}, \qquad (\mathsf{D}u)_j = e^{\pi \alpha^2 \beta_j^2} u_j.$$

Set $N = C\mathsf{T}\mathsf{D}$. In $(Nu)_j$, the summand with index j is exactly u_j: the factor in D cancels the Gaussian factor at the selected coordinate q_j. The other summands form $((N - I)u)_j$. The exact Gaussian identity and inverse estimate proved in [15, Theorem 4.2 and Lemma 4.6] give

$$\mathsf{T} P_s F_s = P_t F_t \mathsf{S}, \qquad \|N - I\| < 2.01 e^{-\pi \alpha^2 \theta/2} < 2^{-\alpha^2 \theta}. \tag{34}$$

The first identity uses the negative Fourier sign in our definition of F_q; its right-hand permutation is consequently multiplication of the frequency by $-s$. Since $\alpha^2 < p$ and $\theta > p/\alpha^4$, we have $\alpha^2 \theta > p/\alpha^2 > 1$. Thus $\|N - I\| < 1/2$, so N is invertible by the Neumann series. Put

$$\mathsf{S}' = \mathsf{S}/2, \qquad \mathsf{J}' = N^{-1}/2, \qquad \mathsf{D}' = 2^{-(2\alpha^2 - 2)} \mathsf{D}.$$

The proof of Proposition 4.7(i) in the cited paper gives

$$\|\mathsf{S}'\| < \frac{3}{4}, \qquad \|\mathsf{J}'\| < \frac{7}{8}, \qquad \|\mathsf{D}'\| < 1.$$

Now set $A = \mathsf{S}'$ and $B_0 = \mathsf{D}'\mathsf{J}'C$. Because $\|C\| = 1$, both are contractions. The diagonal map D is invertible, and $C\mathsf{T} = N\mathsf{D}^{-1}$, so

$$(\mathsf{D}N^{-1}C)\mathsf{T} = \mathsf{D}N^{-1}(N\mathsf{D}^{-1}) = I.$$

Applying this left inverse to the first identity in (34) gives $\mathsf{D}N^{-1}CP_tF_t\mathsf{S} = P_sF_s$. The normalizations in D', J', and S' have product $2^{-(2\alpha^2-2)}2^{-1}2^{-1} = 2^{-2\alpha^2}$. Hence

$$B_0P_tF_tA = 2^{-2\alpha^2}\mathsf{D}N^{-1}CP_tF_t\mathsf{S} = 2^{-2\alpha^2}P_sF_s,$$

which is (33).

Ordered numerical maps. Lemmas 4.8, 4.9 and 4.12 of [15] supply the following algorithms. Their inputs and outputs are in increasing coordinate order, and every output is in the disk grid. The record conversions and tape movement needed to use these routines in our format are accounted for below. Their numerical guarantees are

map	error bound	time bound
$\widetilde{\mathsf{S}}'$	$16p$	$O(tp^{3/2+\delta}\alpha)$
$\widetilde{\mathsf{J}}'$	$3p^2/4$	$O(tp^{3/2+\delta}\alpha)$
$\widetilde{\mathsf{D}}'$	4	$O(tp^{1+\delta})$.

For C, observe that $q_0 = 0$, $q_{s-1} < t$, and $q_{j+1} > q_j$, because $t/s > 1$. Thus C selects an ordered subsequence. A forward scan, with the current input coordinate stored on a work tape, copies a record exactly when that coordinate equals the next q_j.

The selected indices can themselves be generated in order using only $O(p)$ work per index. Precompute $t = hs + \rho$ with $0 \le \rho < s$. Start with $f_0 = e_0 = 0$, and maintain $tj = sf_j + e_j$ with $0 \le e_j < s$. At index j, first use $q_j = f_j + \mathbf{1}_{2e_j \ge s}$. If $j < s - 1$, after copying that record form the values for index $j + 1$ by

$$c_j = \mathbf{1}_{e_j+\rho\ge s}, \qquad e_{j+1} = e_j + \rho - c_j s, \qquad f_{j+1} = f_j + h + c_j.$$

Here $e_j + \rho < 2s$, so c_j is the only possible carry; the displayed update gives $t(j+1) = sf_{j+1} + e_{j+1}$. All these counters have $O(p)$ bits. Elementary division computes h, ρ in $O((\log t)^2) = O(tp)$ time. The record scan and its coordinate counter also cost $O(tp)$, so C takes $O(tp)$ time with zero numerical error.

Consequently we may take

$$\widetilde{A} = \widetilde{\mathsf{S}}', \qquad \widetilde{B}_0 = \widetilde{\mathsf{D}}'\,\widetilde{\mathsf{J}}'\,C.$$

Contraction of the exact maps yields errors less than $16p < p^2$ and $4 + 3p^2/4 < p^2$, respectively. The costs add to the claimed bound.

The original proof of Proposition 4.7(ii) implements $B = P_s^{-1}\mathsf{D}'\mathsf{J}'CP_t$. Its two label-and-sort operations implement P_t and P_s^{-1}, each costing $O(tp\log t)$. In (33) these permutations remain in the transform identity. The algorithms for A and B_0 use C and the three quoted arithmetic maps, with all their original truncations.

Record conversion and tape movement. We specify how these scalar records meet the cited numerical interfaces. Sections 2.1–2.2 of the cited paper use standard binary integers, represent $2^{-p}(a+ib) \in \mathbb{D}_p$ by the exact integer pair (a, b) with p known, and store vectors as ordered coordinate lists. The cited exponential and Gaussian routines return grid values in this representation. Their

computed numerator pairs are exact representations of the approximations; the numerical errors remain those stated for the approximated analytic maps. The source leaves the sign code, digit order, padding and field boundaries open, so we make a concrete choice when implementing its constructive algorithms.

At each scalar input or output boundary of a cited routine, use one sign symbol, the ordinary binary magnitude in most-significant-bit-first order, and a fixed separator for each integer. Use one zero digit and the nonnegative sign for zero, put the real field before the imaginary field, and concatenate pairs in coordinate order. Each grid numerator magnitude has at most $p+1$ digits. To convert one field to our w_{c}-bit word, buffer its magnitude, right-align it in a zero-filled block of that width, and, for a negative sign, complement the block and add one from right to left. Copy the block from left to right to the output. The inverse conversion reads one complete w_{c}-bit word, remembers its leading sign, complements and adds one if negative, and emits the signed magnitude after removing leading zeroes. In both directions the represented integer is unchanged.

A unary ruler of w_{c} cells is prepared once per call by starting with an empty ruler and reading the binary digits of p from most to least significant, successively doubling the current unary prefix and appending one cell when the next digit is one, then adding two cells. This takes $O(p)$ time. Reusing the ruler, a constant number of sweeps of the current field and work blocks performs each conversion, including head returns and cleanup, in $O(p)$ time on fixed tapes. Separators identify the next source field, and the ruler identifies the next fixed-width word; the input head finishes after the consumed field and the output head after the appended field. The inverse adapter supplies the Gaussian input vectors, and the forward adapter formats their outputs. Lemma 2.12 instead receives the ordinary binary integers j, k and needs only the output adapter. These value-preserving conversions add no error. On the Gaussian line lengths $s < t$, their $O((s+t)p) = O(tp)$ work, and their $O(p)$ work per exponential output, are included in the stated bounds.

The remaining tape movement inside the quoted algorithms also has the claimed bound. The S' procedure uses a consecutive periodic input window of radius $\lceil\sqrt{p}\rceil\alpha$ for each output. Visiting these $O(p)$-bit records costs $O(tp\sqrt{p}\alpha)$ tape steps. Remark 4.10 in the cited paper charges the extra jumps at the cyclic boundary by the same bound.

The Neumann evaluation also keeps its computed records bounded. Put $E = N - I$ and $k = \lceil p/(\alpha^2\theta)\rceil$. Under the present hypotheses, Lemma 4.11 gives, for every $z \in \mathbb{D}_p^s$,

$$\widetilde{E}z \in \mathbb{D}_p^s, \qquad 2^p\|\widetilde{E}z - Ez\| < p/3.$$

Its proof establishes the rounded output bound

$$\|\widetilde{E}z\| < 0.42\|z\| + (p/3)2^{-p} < 1.$$

For $u \in \mathbb{D}_p^s$, Lemma 4.12 rounds the real and imaginary parts of each coordinate of $u/2$ toward zero at precision p, obtaining $\widetilde{v}^{(0)} \in \mathbb{D}_p^s$. It then computes $\widetilde{v}^{(j)} = \widetilde{E}\widetilde{v}^{(j-1)}$ for $1 \le j < k$. Induction therefore keeps every iterate in $\mathbb{D}_p^s$. The approximation is $\widetilde{\mathsf{J}}'u = \sum_{j=0}^{k-1}(-1)^j\widetilde{v}^{(j)}$. Since $k \le \alpha^2+1 < p+1$, each component of any signed partial numerator sum has magnitude at most $k2^p$ and needs $p+\lceil\log_2(k+1)\rceil+O(1) = O(p)$ bits. Inside one $\widetilde{E}$ row, the $2m$ disk-grid summands of Lemma 4.11 likewise need $p+\lceil\log_2(2m+1)\rceil+O(1) = O(p)$ accumulator bits, where $m = \lceil\sqrt{p}/(2\alpha)\rceil$. These partial sums are accumulated exactly on separate tapes; only completed iterates are fed into $\widetilde{E}$.

The window of radius m in one $\widetilde{E}$ call has tape movement cost $O(tp(1+\sqrt{p}/\alpha))$, including the analogous boundary accounting. The exact row additions cost $O(smp)$, within this bound. The $k-1$ calls therefore spend

$$O((\alpha^2+1)tp(1+\sqrt{p}/\alpha)) = O(tp\sqrt{p}\alpha)$$

on these scans and additions, using $2 \le \alpha < \sqrt{p}$. For either Gaussian procedure, if a window radius w satisfies $s \le 2w$, then $O(1 + w/s)$ traversals of the period cost $O((s + w)p) = O(wp)$ per output. This also covers windows longer than one period. The current iterate, next iterate, and running sum use a fixed number of tapes holding $O(p)$-bit records. Outer accumulation costs $O(ksp) = O(tp\alpha^2)$, within $O(tp\sqrt{p}\alpha)$. All truncations are those of the cited algorithms, and the tape count and constants remain uniform in the parameters. □

8.2 Growing dimension and the padded array

For the tensor product, apply the one-dimensional maps to one coordinate line at a time. The chunk-swap algorithm supplies the movement of the physical axis fields, including its cost when the number of axes grows.

Lemma 20 (Tensor interface)**.** *Suppose every pair (s_i, t_i), $1 \le i \le d$, satisfies Lemma 19 with the same p, α, δ. Put $T = \prod_i t_i$, $\gamma = 2d\alpha^2$, and*

$$R = \bigotimes_i P_{s_i}, \quad Q = \bigotimes_i P_{t_i}, \quad \mathcal{A} = \bigotimes_i A_i, \quad \mathcal{B}_0 = \bigotimes_i B_{0,i}.$$

Then

$$RF_{\boldsymbol{s}} = 2^{\gamma}\mathcal{B}_0 Q F_{\boldsymbol{t}}\mathcal{A}, \qquad F_{\boldsymbol{q}} := \bigotimes_i F_{q_i}. \tag{35}$$

Also write $F^+_{\boldsymbol{q}} = \bigotimes_i F^+_{q_i}$. Both $\mathcal{A}$ and $\mathcal{B}_0$ are contractions. The linewise implementations of $\mathcal{A}$ and $\mathcal{B}_0$ each have error less than dp^2. The total time spent in the one-dimensional algorithms for both maps is $O(dTp^{3/2+\delta}\alpha)$, with a constant independent of d.

If each $t_i \in \{r/2, r\}$ with $r = 2^{\ell}$, the array is kept in the full padded box of T entries, and each entry occupies $O(p)$ bits, then exposing and restoring all the lines costs

$$O\big(Tp\, d^2(1 + \ell^{\tau})\big)$$

using the equal-width chunk-swap bound $O(Vu^{\tau})$ and the fixed-digit movement primitive. Any fixed polynomial in p of setup per line is negligible compared with Tp when r grows faster than every such polynomial and d is polynomially bounded in p.

Proof. Tensoring (33) proves (35). To implement $\mathcal{A}$, first embed its input in the padded box by writing zero outside $\prod_i [0, s_i)$. Expand the axes one at a time. After the axes in a set I have been expanded, the valid entries have coordinate ranges $[0, t_i)$ for $i \in I$ and $[0, s_i)$ otherwise; every other entry is zero. Expose the next axis i as the innermost address field. On a line whose other coordinates are valid, read its first s_i values and apply $\widetilde{A}_i$, then write the resulting t_i values. On every other line write t_i zeros. The new valid set is the one for $I \cup \{i\}$.

For compression, start with all coordinates in $[0, t_i)$ valid. After a set J of axes has been compressed, validity means $j_h < s_h$ for $h \in J$ and $j_h < t_h$ otherwise. For the next axis i, apply $\widetilde{B}_{0,i}$ to each t_i-line whose other coordinates are valid. Write its s_i outputs followed by $t_i - s_i$ zeros. A line invalid on another coordinate is already zero; consume it and write a zero line without calling $\widetilde{B}_{0,i}$. The box therefore keeps its full size, and the new valid set is the one for $J \cup \{i\}$.

For a fixed axis, the exact map acts independently on its valid lines and is zero on the other lines. Taking the maximum over the line norms shows that this axis map is a contraction; taking the maximum of the line errors gives an error less than p^2 for the whole axis. Composing the d axis

maps gives error less than dp^2. At most T/t_i lines invoke the one-dimensional algorithm for axis i, so their cost is bounded by

$$\sum_{i=1}^{d} \frac{T}{t_i} O(t_i p^{3/2+\delta} \alpha) = O(dT p^{3/2+\delta} \alpha).$$

Initial padding costs $O(Tp)$, and scanning and zeroing the remaining lines costs $O(Tp)$ per axis. Both are within the same bound. All calls reuse one fixed collection of work tapes. Loop indices and the growing shape descriptor reside on tapes; they are not extra heads.

The descriptor records which coordinate each physical address field holds. Thus moving fields changes their positions, but not the validity conditions above. To move an axis past its neighbor, interchange their ℓ-bit or $(\ell-1)$-bit address fields. Equal widths use Lemma 12. For unequal widths, write the longer field as a leading bit e followed by $\ell - 1$ bits. If it comes first, the order is $e\,x\,y$, with x, y of equal width. Interchange x, y, then move e between them, obtaining $y\,e\,x$. If the longer field comes second, first move its leading bit left of the shorter field, then interchange the two equal-width pieces. The bit move uses Lemma 3. Each adjacent interchange therefore costs $O(Tp(1 + \ell^\tau))$. Exposing and restoring one axis needs $O(d)$ adjacent interchanges, hence all axes need $O(d^2)$. These are exact permutations of the padded box, so they change neither norms nor errors.

Since $t_i \geq r/2$, each tensor map invokes at most $\sum_i T/t_i \leq 2dT/r$ lines. For any fixed setup degree c, work $O(p^c)$ per line totals $O(dTp^c/r) = o(Tp)$: after division by Tp, the ratio is $O(dp^{c-1}/r)$, which tends to zero under the stated growth assumptions. This estimate includes local descriptor copying, validity tests per line, and preparing line boundaries. □

8.3 The middle transform and the final permutation

The factor $QF_{\boldsymbol{t}}$ in (35) can be computed from a cyclic convolution and two pointwise phase multiplications. The chirp identity below performs the frequency change in Q without applying Q as a separate permutation of the array. The second identity shows how the retained source permutation R cancels when computing a convolution. Here $*$ denotes cyclic convolution on the product of the coordinate groups and a dot denotes coordinatewise multiplication.

Lemma 21 (Chirp and permutation cancellation)**.** *With R, Q, s_i, t_i as in Lemma 20, assume every t_i is even. For $\boldsymbol{j} \in \mathbb{Z}^d$, define*

$$a_{\boldsymbol{j}} = \exp\left(-\pi i \sum_{i=1}^{d} \frac{s_i j_i^2}{t_i}\right).$$

Then a has period t_i in coordinate i, and

$$QF_{\boldsymbol{t}} u = \overline{a} \cdot \frac{a * (\overline{a} \cdot u)}{T}. \tag{36}$$

If $S = \prod_i s_i$, then

$$(RF_{\boldsymbol{s}}^{+})[(RF_{\boldsymbol{s}} u) \cdot (RF_{\boldsymbol{s}} v)] = \frac{u * v}{S^2}. \tag{37}$$

Complex conjugation of the input and output of an algorithm for $RF_{\boldsymbol{s}}$ gives an algorithm for $RF_{\boldsymbol{s}}^{+}$.

Proof. Replacing j_i by $j_i + t_i$ changes the exponent defining a by $-\pi i s_i(2j_i + t_i)$, an integral multiple of $2\pi i$. The coefficient of $u_{\boldsymbol{k}}$ in the right side of (36) is

$$\frac{1}{T} \overline{a}_{\boldsymbol{j}} a_{\boldsymbol{j}-\boldsymbol{k}} \overline{a}_{\boldsymbol{k}} = \frac{1}{T} \exp\left(2\pi i \sum_i \frac{s_i j_i k_i}{t_i}\right).$$

This is precisely the coefficient in $QF_{\boldsymbol{t}}$, because Q reads frequency $(-s_i j_i)_i$ from the negative-sign transform.

For a unit c modulo q, let $(R_c z)_j = z_{cj}$. Substituting $h = ck$ in the Fourier sum gives

$$(F_q^+ R_c z)_j = \frac{1}{q} \sum_{h=0}^{q-1} z_h e^{2\pi i h c^{-1} j/q} = (R_c^{-1} F_q^+ z)_j.$$

Each t_i is a unit modulo s_i, so tensoring this identity gives $F_{\boldsymbol{s}}^+ R = R^{-1} F_{\boldsymbol{s}}^+$ and hence $R F_{\boldsymbol{s}}^+ R = F_{\boldsymbol{s}}^+$. Also $(R F_{\boldsymbol{s}} u) \cdot (R F_{\boldsymbol{s}} v) = R(F_{\boldsymbol{s}} u \cdot F_{\boldsymbol{s}} v)$, because a coordinate permutation commutes with pointwise products. Therefore

$$R F_{\boldsymbol{s}}^+ \big[R(F_{\boldsymbol{s}} u \cdot F_{\boldsymbol{s}} v)\big] = F_{\boldsymbol{s}}^+ (F_{\boldsymbol{s}} u \cdot F_{\boldsymbol{s}} v).$$

The normalized convolution rule is $F_{\boldsymbol{s}}(u * v) = S(F_{\boldsymbol{s}} u) \cdot (F_{\boldsymbol{s}} v)$, while $F_{\boldsymbol{s}}^+ F_{\boldsymbol{s}} = I/S$. The two normalizations give

$$F_{\boldsymbol{s}}^+ (F_{\boldsymbol{s}} u \cdot F_{\boldsymbol{s}} v) = \frac{1}{S} F_{\boldsymbol{s}}^+ F_{\boldsymbol{s}} (u * v) = \frac{u * v}{S^2},$$

proving (37). Finally, conjugation changes the negative Fourier sign to the positive one and commutes with the real permutation R. It preserves both the disk grid and the norm, so conjugating the input and output also preserves an algorithm's error bound. □

Small error must stay small after rescaling

Added explanation A numerical budget you can audit by hand.

Imagine an approximate computation whose current absolute error is at most 0.01. Passing it through an exact contraction does not enlarge that error. If the next approximation adds at most 0.02, the new error bound is at most 0.03. This is a worst-case sum: we do not assume errors cancel or behave randomly.

Now restore a scale factor of eight. The error bound becomes $8 \cdot 0.03 = 0.24$, which is safely below one half. If the scale factor were 32 instead, the bound would become 0.96. That weaker guarantee does not justify rounding to the right integer.

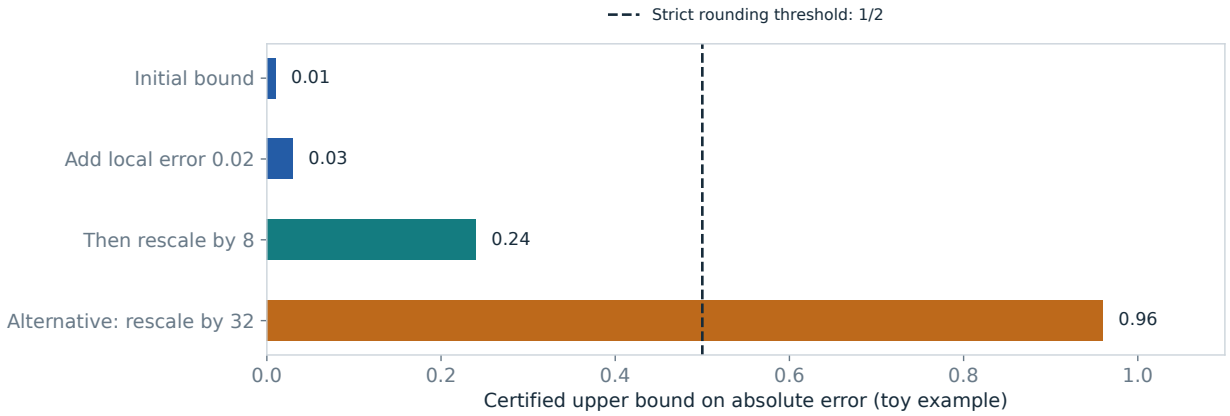


Figure E20. A small internal error is not the final error. The last two bars are alternative rescalings of the same 0.03 bound, not successive operations. Crossing 1/2 means this bound is insufficient to guarantee exact rounding; it does not prove the actual answer is wrong.

The source proof has larger and more elaborate normalizations, but the principle is the same. It budgets local errors, bounds intermediate sizes, and tracks the multiplication of error when scales are undone. It also needs enough bits for intermediate exact dyadic values, even when the completed transform is contractive.

Why strict inequality matters. If the true integer is 12, a result strictly between 11.5 and 12.5 rounds uniquely to 12. At exactly 12.5, the distance to 12 and 13 is the same, so an error allowance of "at most one half" cannot determine which integer was intended. The paper proves a strict final bound for every coefficient and then performs exact carry propagation.

What to carry forward. Approximation is compatible with an exact answer when the fully rescaled error is proved smaller than half the gap between integers.

Why improving one part may change nothing

Added explanation The final saving is limited by the weakest margin.

Suppose three operations have normalized costs $p^{0.80}$, $p^{0.97}$, and $p^{0.90}$. Their margins below a full power of p are 0.20, 0.03, and 0.10. For large p, the term $p^{0.97}$ grows fastest and controls the exponent of the total.

Make the first operation much cheaper, changing $p^{0.80}$ to $p^{0.60}$. Its margin improves to 0.40, yet $p^{0.97}$ still controls the total. Alternatively, improve the second operation to $p^{0.92}$. Now the minimum margin is 0.08, so the combined exponent can improve as well, subject to its remaining overhead and strict-slack requirements.

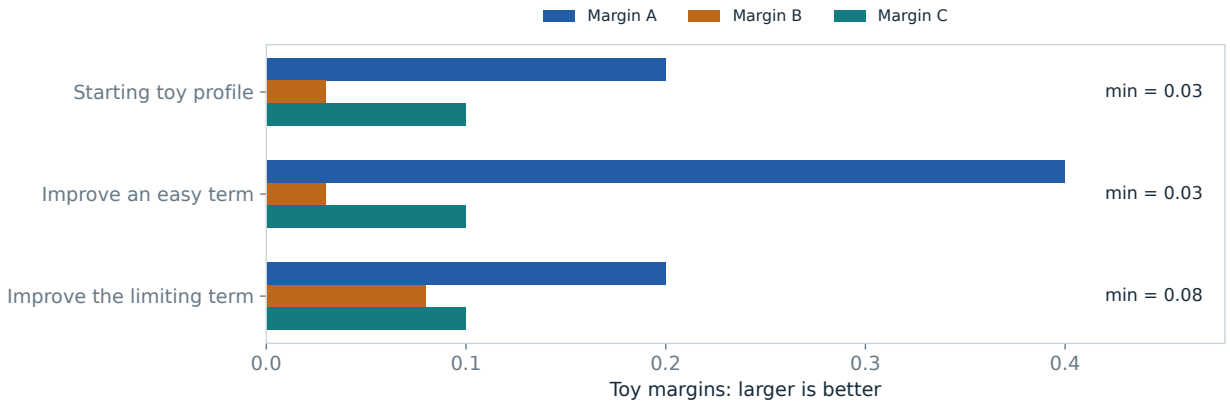


Figure E21. A local gain need not improve the overall exponent. Each row is a separate toy profile, not a historical result. Larger margins are better. The printed minimum stays at 0.03 when an already easy term improves, and rises to 0.08 when the limiting term improves.

The source's final accounting has seven dominant terms instead of three. The figure in the next technical explanation computes their actual original margins. Its logarithmic scale makes extremely small positive values visible. That plot reverses the visual convention: farther right means a smaller margin because the axis is minus the logarithm of the margin. Always read the axis before deciding which bar is better.

Why a little extra room is needed. A cost may also carry factors such as $(\log p)^2$. Such a factor grows more slowly than any fixed positive power p^ρ, but it is not a constant. Choosing a final saving strictly below the available margin leaves room to absorb these factors for sufficiently large p. Equality at the boundary would not justify the same argument.

What to carry forward. Track the full cost profile. An improvement is useful at the headline level only when all the limiting terms and interfaces permit it.

Putting exactness and the running time together

Added explanation Read before the final assembly proof.

The paper has two finish lines. It must return the exact product, and every part of the computation must fit the claimed time bound. These obligations are related through precision and storage volume, but neither replaces the other.

Exactness from a strict error interval. The desired convolution coefficients c_j are integers. If the computation produces an approximation $\widetilde{c}_j$ with $|\widetilde{c}_j - c_j| < 1/2$, nearest-integer rounding recovers c_j uniquely. The bound must hold *after* restoring all transform normalizations and digit scales. A tie at exactly $1/2$ is not enough. Once coefficients are recovered, carries and binary output are exact integer operations.

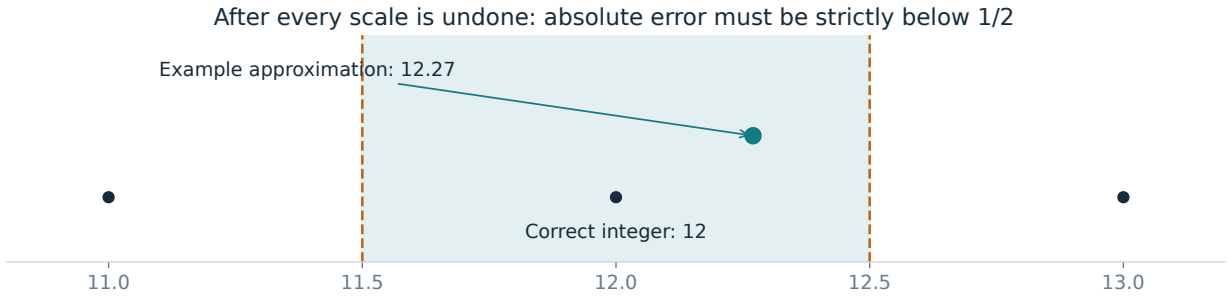


Figure E22. Approximate internal arithmetic, exact final output. The displayed 12.27 is a teaching example. The source's proof supplies a uniform bound on every recovered coefficient; this picture does not measure its actual numerical error.

The source uses radix width $b = \lceil \log_2 n \rceil$ and working precision $p = 6b$ in its original assembly. Roughly, there are n/b digits and $\Theta(p)$ stored bits per coefficient, making the padded volume $V = Tp = \Theta(n)$. This relation is what turns costs per stored bit into a bound in terms of the original input size.

Seven competing bills. The final table separates operations that scale differently with p. Write a normalized term as p^{1-g_i}; then g_i is its margin below a full power of p. With $d = \Theta(p^\epsilon)$, $K = \Theta(p^{\epsilon c})$, and $\ell = \Theta(p^{1-\epsilon})$, the margins are

$$g_1 = 1 - \epsilon(1 + c), \qquad g_2 = \epsilon c(1 - \tau),$$
$$g_3 = \epsilon(1 - \lambda'), \qquad g_4 = 1 - \tau - \epsilon(2 - \tau),$$
$$g_5 = \tfrac{1}{4} - \delta - \tfrac{3}{2}\epsilon, \qquad g_6 = 1 - \delta - \epsilon,$$
$$g_7 = \epsilon.$$

The total is controlled by the *smallest* margin. Making an already large margin larger may have no effect on the headline exponent.

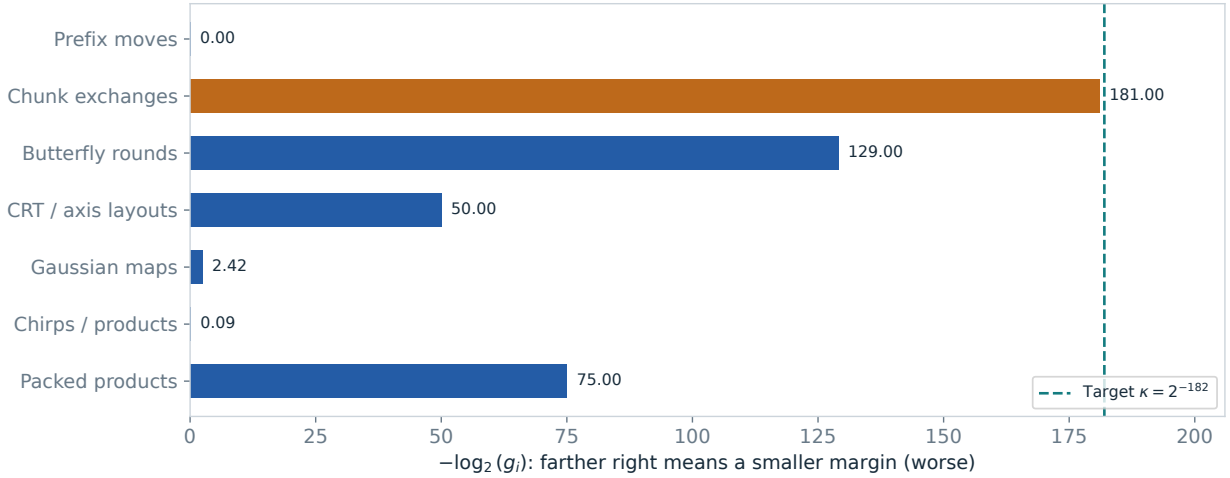


Figure E23. The original parameter choice has a clear bottleneck. These values are computed from exact rational parameters in the original source, not estimated from timings. The horizontal scale is $-\log_2 g_i$: longer bars mean smaller savings. Chunk exchanges have $g_2 = 2^{-181}$; the chosen final saving is $\kappa = 2^{-182}$.

Read one row all the way through. For transform-layout exchanges, there are $O(p/K)$ swaps of width K. Each costs $O(VK^{\tau})$. Thus the total is

$$O(VpK^{\tau-1}) = O\big(Vp^{1-\epsilon c(1-\tau)}\big).$$

This is exactly the second margin g_2. The small quantities multiply: a small width-growth exponent ϵc and a small swap saving $1-\tau$ yield an even smaller final margin. This explains why an impressive local idea may produce a very small asymptotic exponent in the whole algorithm.

Why strict slack is reserved. Let $G = \min_i g_i$. Fixed powers of $\log p$ can be absorbed if $\kappa < G$, since $(\log p)^C = o(p^{G-\kappa})$ for fixed C. The original conservative choice gives $G = 2\kappa$. The final cost then fits

$$O(Vp^{1-\kappa}) = O\big(n(\log n)^{1-\kappa}\big).$$

This argument is eventual. A single fixed cutoff covers all the required inequalities; below it, the machine uses schoolbook multiplication. Enlarging the big-O constant covers the finite set of smaller lengths. Establishing the complete uniform implementation remains part of the proof obligation.

What later community refinements are trying to improve. Some refinements change a finite network or the price of its frame transitions; others reuse workspace, change how stages are combined, or improve an outer parameter choice. These operate at different layers. A new finite profile must still be bound to its actual circuit and then passed through the applicable recurrence and cost analysis. The original seven-margin diagram is a reading aid for this frozen manuscript, not a claim that every later construction uses the same accounting or that it displays the latest community witness.

What to carry forward. Exact recovery needs a strict error bound; the exponent needs a strict margin in every dominant cost. Both must refer to the same algorithm.

Original source anchors: `08-assembly.tex, eq:fixed-parameters; eq:margin-list; eq:final-error`.

Original manuscript continues below.

9 Exact integer multiplication and its cost

We now use the preceding algorithms to multiply two n-bit integers. An equal-width address swap on logical bit volume V costs $O(Vu^{\tau})$ for width u. A completed parallel normalized butterfly layer costs $O(Vd^{\lambda'})$ and has coefficient error less than $\sqrt{2}\,2^{-p}$. The ordered Gaussian resampling maps are contractions; their disk-grid approximations have error less than $p^2 2^{-p}$ and cost $O(tp^{3/2+\delta}\alpha)$ on a line of length t. Address permutations and intermediate dyadic arithmetic are exact. We choose compatible parameters for these procedures, recover the integer coefficients from their approximations, and account for the remaining tape operations.

9.1 A fixed rational choice

The two motif exponents were fixed in (10). Take the following fixed rational numbers:

$$\begin{gathered} \tau = \sigma = 1 - 2^{-50}, \qquad \beta = \frac{1}{2}, \qquad \delta = \frac{1}{16}, \\ \lambda = 1 - 2^{-52}, \qquad c = 2^{-56}, \qquad \lambda' = 1 - 2^{-54}, \\ C_1 = 20, \qquad \epsilon = 2^{-75}, \qquad \kappa = 2^{-182}. \end{gathered} \tag{38}$$

These constants are deliberately conservative; no optimization is claimed.

The layer conditions hold with room to spare:

$$\max\{\tau, \sigma\} < \lambda < 1, \qquad \tau(1 + c/\beta) < 1 - 31 \cdot 2^{-55} < \lambda,$$

and

$$\sigma + \beta(1 - \sigma) = 1 - 2^{-51} < \lambda < \lambda' < 1.$$

The strict final inequality absorbs the logarithmic number of groups in a base-m partition. The dimension exponent also satisfies

$$\epsilon < \frac{1}{12}, \quad \epsilon C_1 < 1, \quad \epsilon(2 - \tau) < 1 - \tau, \quad \frac{3}{4} + \delta + \frac{3}{2}\epsilon < 1,$$
$$\epsilon(1 + c) < 1, \qquad \epsilon + \delta < 1.$$

9.2 Input and transform sizes

The input is the promised string $x\#y$, with both operands written most significant bit first. In one left-to-right pass, copy x and y to two fixed work tapes and count the bits of x with a binary counter. The promise gives both operands this counted length n. The counter bound in Section 2 and the two sequential copies give cost $O(n)$, including all leading zeroes. The work-copy heads finish just after the least significant bits, ready to move leftward when the digits are formed.

9.2.1 The padded box and working precision

After obtaining n, compute the size parameters below with the separate setup costs stated in this subsection. For sufficiently large n, put

$$b = \lceil \log_2 n \rceil, \quad p = 6b, \quad d = \lfloor b^\epsilon \rfloor, \quad K = \lfloor d^c \rfloor.$$

Take the power of two T in $[4n/b, 8n/b)$, and let $r = 2^{\lceil (\log_2 T)/d \rceil}$ and $\ell = \log_2 r$. Choose $d\ell - \log_2 T$ of the d axis lengths to be $r/2$ and the others to be r. This gives

$$t_i \in \{r/2, r\}, \qquad \prod_i t_i = T, \qquad t_d = r.$$

Indeed $d\ell - \log_2 T$ is an integer in $[0, d)$, so at least one axis has length r; choose it as the last axis. These definitions give

$$Tp = \Theta(n), \quad d = \Theta(p^\epsilon), \quad K = \Theta(p^{\epsilon c}), \quad \ell = \Theta(p^{1-\epsilon}), \quad r = 2^{\Theta(p^{1-\epsilon})}. \tag{39}$$

The comparison constants can be fixed for every input length at once. For sufficiently large b, the bounds on n, T give $b/2 \le \log_2 T \le b$. Since $p = 6b$, $d = \lfloor b^\epsilon \rfloor$ and $d \le b$, they imply

$$\frac{1}{12} p^\epsilon \le d \le p^\epsilon, \qquad \frac{p}{12d} \le \ell \le \frac{p}{3d}.$$

Thus we fix $a_d = a_r = 1/12$, $b_d = 1$, $b_r = 1/3$, and $C_\ell = 1$ in the layer and transform interfaces. Their pointwise bounds cover every computed pair (d, r), including all choices arising at the same p. These bounds also give that r exceeds every fixed power of p, $K/\log p \to \infty$, and $\ell/K \to \infty$. There are eventually enough axes for the $O(\log d)$ reserved row axes and enough bits for all work blocks.

Set

$$\alpha = \left\lceil (12d^2 b)^{1/4} \right\rceil, \qquad \gamma = 2d\alpha^2, \qquad \eta = \frac{1}{4d}.$$

The ceiling does not affect the needed bounds: for $A \ge 1$, $\lceil A \rceil \le 2A$, and hence

$$\alpha \le 2(12d^2 b)^{1/4}, \qquad \gamma < 28 d^2 \sqrt{b} \le 28 b^{1/2+2\epsilon} \le 28 b^{2/3}. \tag{40}$$

It follows that $2 \le \alpha < \sqrt{p}$ eventually and $\gamma = o(b)$. For example $b \ge 2^{24}$ implies $\gamma \le b/4$.

9.2.2 Prime source lengths

The distinct prime lengths s_i must be close to the powers of two t_i: their product must occupy a fixed fraction of the padded box, while each ratio t_i/s_i must leave enough separation for resampling. We verify both requirements for the parameters just chosen.

The prime-interval result [15, Lemma 5.1], based on explicit estimates of Rosser and Schoenfeld [23], gives the following bound: for $0 < \eta < 1/4$ and $x > e^{2/\eta}$, the interval

$$(1 - 2\eta)x < q \le (1 - \eta)x$$

contains at least $\eta x/(2 \log x)$ primes. Here log is the natural logarithm. The size bounds in (39) give $d = \Theta(p^\epsilon)$ and $\log r = \Theta(p/d)$. For large n we have $d \ge 2$, so $0 < \eta < 1/4$, and

$$\frac{\log(r/2)}{d} = \Theta(p^{1-2\epsilon}) \longrightarrow \infty.$$

Thus both $x = r/2$ and $x = r$ satisfy $\log x > 8d = 2/\eta$. For either choice, the stated lower bound for the number of primes is $x/(8d \log x)$ and exceeds d: indeed, $\log x = \Theta(p^{1-\epsilon})$, whereas $\log(8d^2 \log x) = O(\log p) = o(\log x)$, so $x > 8d^2 \log x$ eventually. The two intervals are disjoint because $(1-\eta)r/2 < (1-2\eta)r$ for $\eta < 1/4$, and both intervals lie above 2. We may therefore choose distinct odd primes satisfying

$$(1-2\eta)t_i < s_i \le (1-\eta)t_i.$$

Every $\gcd(s_i, t_i)$ equals one because t_i is a power of two. Put $S = \prod_i s_i$. Multiplying the lower bounds on s_i and applying strict Bernoulli inequality gives

$$\frac{S}{T} > \left(1 - \frac{1}{2d}\right)^d > \frac{1}{2} \qquad (d \ge 2).$$

Together with $s_i < t_i$, this gives $T/2 < S < T$. Writing $\theta_i = t_i/s_i - 1$, the upper bound on s_i gives $\theta_i \ge \eta/(1-\eta) > \eta$. Therefore

$$\alpha^4 \theta_i > \frac{12d^2 b}{4d} = 3db \ge 6b = p.$$

The choice of α gives $\alpha = \Theta(p^{1/4+\epsilon/2})$, and (40) already ensures $2 \le \alpha < \sqrt{p}$ for large n. We also have $t_i \le r \le T < 8n/b < 2^{6b} = 2^p$. Thus every hypothesis of Lemma 19 holds for all sufficiently large input lengths. The superpolynomial growth of r established above also supplies the setup condition in Lemma 20.

The machine finds these primes by scanning the two intervals deterministically and testing each candidate by trial division through its square root. Testing every candidate would visit $O(r)$ candidates and at most $O(\sqrt{r})$ trial divisors per candidate. Elementary division on $O(\log r)$-bit integers and management of a list of at most d selected primes give the conservative bound $O(dr^{3/2} \operatorname{poly}(p))$. Its logarithm is

$$O(\log p + \log r) = O(\log p + p/d) = o(p).$$

Since $p = 6\lceil \log_2 n \rceil = \Theta(\log n)$, the search takes $n^{o(1)} = o(n)$ time. All remaining scalar setup, including comparisons for fixed rational powers, modular inverses, prime products and shape descriptors, has cost polynomial in p. For example, $d = \lfloor b^{2^{-75}} \rfloor$ is found by binary search using exact comparisons $d^{2^{75}} \le b$; the exponent is fixed. The same procedure computes K and all other fixed rational powers. All primes and length parameters are therefore generated by the machine; they are not advice.

9.2.3 The radix-digit polynomials

Write the two inputs as polynomials in 2^b, each with $q = \lceil n/b \rceil$ nonnegative digits. Their product has degree at most

$$2q - 2 < 2n/b \le T/2 < S.$$

Thus convolution modulo $x^S - 1$ has no nonzero wraparound. Every product coefficient obeys

$$0 \le c_j \le q(2^b - 1)^2 < 2^{3b}, \tag{41}$$

since $q \le n \le 2^b$.

9.3 An explicit cyclic-convolution layout

To use the source tensor, we first place the ordinary coefficient stream in its tensor order. Agarwal and Cooley used the Chinese remainder theorem to convert one cyclic convolution into multidimensional cyclic convolution [1]. The following map uses that algebraic split, with coordinate powers that make the change of order a sequence of controlled cyclic shifts.

Let $s_1, \dots, s_d$ be the pairwise coprime source lengths, let $S = \prod_i s_i$, and put

$$P_i = \prod_{j<i} s_j, \qquad \mu_i = P_i^{-1} \pmod{s_i}.$$

Here P_i denotes an integer prefix product, not a Fourier-coordinate permutation. The assignment

$$\Phi : \mathbb{C}[x]/(x^S - 1) \longrightarrow \mathbb{C}[x_1, \dots, x_d]/(x_1^{s_1} - 1, \dots, x_d^{s_d} - 1), \qquad x \longmapsto \prod_i x_i^{\mu_i}$$

respects $x^S = 1$, so it defines an algebra map. The target monomial group has size S, and the image of x has order S, because each μ_i is a unit and the s_i are pairwise coprime. Its powers therefore run through all target basis monomials exactly once. Hence Φ is an isomorphism and preserves cyclic convolution.

Every ordinary coefficient index has a unique mixed-radix expression

$$k = \sum_{j=1}^{d} a_j P_j, \qquad 0 \le a_j < s_j.$$

The coordinate of $\Phi(x^k)$ in axis i is

$$b_i = a_i + \mu_i \sum_{j<i} a_j P_j \pmod{s_i}. \tag{42}$$

Indeed terms with $j > i$ are divisible by s_i, and $\mu_i P_i = 1$ modulo s_i. Numerical order of k is lexicographic order of $(a_d, \dots, a_1)$. Insert the padding in one nested scan of the full box $\prod_i [0, t_i)$ in this lexicographic order, with a_d the outermost counter and a_1 the innermost. At an address satisfying $a_i < s_i$ for every i, copy the next original coefficient record; at every other address, write a zero record. The valid addresses occur in the original coefficient order, so each record is copied to its intended mixed-radix address. To delete the padding after the inverse axis reorder, use the same enumeration, copying the record at each valid address and skipping the record at each invalid address.

Store the nested binary counters consecutively on a fixed work tape. Their total width is $\sum_i \log_2 t_i = \log_2 T = O(p)$, and the stored bounds s_i also have total width $O(p)$. At each of the T addresses, updating the counters, testing validity, and handling the $O(p)$-bit record therefore cost $O(p)$ steps, including counter-head returns. Each insertion or deletion scan costs $O(Tp)$.

Now reorder the complete padded axes to $(a_1, \dots, a_d)$. In this physical order, the coordinates with indices below i precede coordinate i. Update the targets in the order $i = d, d-1, \dots, 1$ using (42). When target i is updated, none of its lower-index controls has yet changed, so the offset is computed from the original digits a_j required by the formula.

To specify the action on padded addresses, let

$$\mathcal{P} = \prod_i [0, t_i), \qquad \mathcal{V} = \prod_i [0, s_i)$$

be sets of integer addresses. On a fixed control prefix, rotate the valid target interval $[0, s_i)$ by the offset in (42) and fix $[s_i, t_i)$. If any lower-index control is invalid, use the identity. This defines a bijection of $\mathcal{P}$: for fixed controls and spectators it is a cyclic permutation of the valid target interval and the identity elsewhere. Each step preserves $\mathcal{V}$ and its complement. Consequently zero padding remains zero after every step. An inverse is explicit: process $i = 1, \dots, d$, subtracting the same expression formed from the already recovered lower coordinates; then undo the axis reorder and perform the deletion scan just described.

Each triangular update is a prefix-controlled cyclic shift. The controls precede its target in the current physical order. For each prefix, prepare the offset, split the valid target interval into its two consecutive pieces, and merge those pieces in the opposite order. Every target position carries its full spectator suffix as a block; the invalid interval is copied unchanged. There are at most T/t_i control prefixes for target i. For any fixed setup degree C, all offsets therefore cost $O(dTp^C/r) = o(Tp)$ to prepare, since $t_i \geq r/2$ and r exceeds every fixed power of p.

The axis widths are ℓ or $\ell - 1$. To exchange two adjacent axes of unequal width, move the longer axis's leading bit outside their equal-width pair, swap that pair by the proved chunk-swap procedure, and put the leading bit at its new position. This adds a fixed number of linear passes to the equal-width swap cost. Reversing all axes uses $O(d^2)$ such moves. Therefore the map Φ, its inverse, and their padding operations cost

$$O(Tp\, d^2(1 + \ell^\tau)).$$

This construction describes payload movement from address a to its new address. It is distinct from the operator convention $(R_c z)_j = z_{cj}$ used for Fourier dilations, which moves a payload from address j to $c^{-1}j$.

9.4 Complex convolution by a coefficient twist

The power-of-two complex convolution used in the chirp identity is obtained from the synthetic convolution by a coefficient twist. Put $\mathcal{R} = \mathcal{R}_r = \mathbb{C}[y]/(y^r + 1)$ and $\zeta = e^{\pi i/r}$. The map

$$\Theta : \mathbb{C}[x_d]/(x_d^r - 1) \longrightarrow \mathcal{R}, \qquad x_d \longmapsto \zeta y$$

is well defined, since $(\zeta y)^r = (-1)(-1) = 1$. On the bases $(x_d^k)_{0 \leq k < r}$ and $(y^k)_{0 \leq k < r}$ it is diagonal, multiplying coefficient k by the nonzero number ζ^k. It is therefore an isomorphism, with inverse multiplying that coefficient by ζ^{-k}. Both exact maps preserve the coefficient sup norm. For an explicit check on the wrap, write $k + l = j + hr$, where $h \in \{0, 1\}$. A product term after twisting and negacyclic reduction has factor

$$\zeta^{k+l}(-1)^h = \zeta^j(\zeta^r)^h(-1)^h = \zeta^j.$$

Untwisting therefore recovers the cyclic coefficient with the correct sign. We next specify how this linewise map meets the ring-convolution interface. Put $M = \prod_{i<d} t_i = T/r$. For a complex array f on $\prod_{i=1}^d \mathbb{Z}/t_i\mathbb{Z}$, with $t_d = r$, define

$$(\iota f)_{\boldsymbol{j}} = \sum_{k=0}^{r-1} f_{\boldsymbol{j},k} y^k, \qquad \boldsymbol{j} \in \prod_{i<d} \mathbb{Z}/t_i\mathbb{Z}.$$

This identifies the scalar tensor with an array of M records in $\mathcal{R}$. As maps of values, ι and ι^{-1} preserve norms and grids. Write $(D_\pm f)_{\boldsymbol{j},k} = \zeta^{\pm k} f_{\boldsymbol{j},k}$. The linewise extension of Θ is ιD_+, with inverse $D_-\iota^{-1}$. Consequently, with $\mathcal{M}_r$ from Section 7, normalized complex cyclic convolution is

$$\mathcal{M}_{\mathbb{C}}(f, g) := D_-\iota^{-1}\mathcal{M}_r(\iota D_+ f, \iota D_+ g) = \frac{f * g}{T}.$$

Here $\mathcal{M}_r$ convolves the first $d-1$ axes over $\mathcal{R}$, whereas $*$ on the right convolves all d scalar axes. The division by T is already part of $\mathcal{M}_r$. The maps $D_\pm$ and $\iota^{\pm 1}$ are isometries, so $\mathcal{M}_{\mathbb{C}}$ is also a bilinear contraction.

9.4.1 Computed complex convolution

For the tape representation, use the scalar format of Section 8, with $w_{\mathrm{c}} = p+2$ bits per component. For $p \geq 2$, this satisfies $p+2 \leq w_{\mathrm{c}} \leq 2p$, so fix $C_w = 2$ and instantiate the synthetic component width as $w = w_{\mathrm{c}}$. Both representations then concatenate exactly the same real and imaginary words in the same coefficient order: the last coordinate is already the contiguous polynomial suffix. Hence ι and ι^{-1} change only the shape descriptor and require no payload movement when these component formats agree.

For the synthetic transform calls, the machine writes the header

$$\mathcal{T} = \Gamma(p)\Gamma(d)\Gamma(r)\Gamma(K)\Gamma(w_{\mathrm{c}}) \prod_{i=1}^{d-1} \Gamma(t_i).$$

For each normalized polynomial product it supplies $\Gamma(p)\Gamma(r)\Gamma(w_{\mathrm{c}})$. These are precisely the headers of Lemmas 16 and 17, formed from the computed parameters. Their lengths are $O(p)$; forming the transform header costs $\mathrm{poly}(p) = o(Tp)$, and copying a product header for each of the $M = T/r$ polynomial records costs $O(Mp) = O(Tp)$.

For $0 \leq k < r$, obtain disk-grid approximations $\widetilde{\theta}_k^+$ and $\widetilde{\theta}_k^-$ to ζ^k and ζ^{-k}, each with error less than $2\,2^{-p}$. The exponential routine uses denominator $2r$ and costs $O(p^{1+\delta})$ per factor. Write $\widetilde{\mathcal{M}}_r$ for the procedure in Proposition 18. For disk-grid inputs f, g, one of norm at most .7, the computed complex convolution is the following sequence, with the twist arrays repeated on every last-coordinate line:

$$\begin{aligned} f' &= Q_p(\widetilde{\theta}^+ \cdot f), \\ g' &= Q_p(\widetilde{\theta}^+ \cdot g), \\ h' &= \widetilde{\mathcal{M}}_r(\iota f', \iota g'), \\ \widetilde{\mathcal{M}}_{\mathbb{C}}(f, g) &= Q_p\left(\widetilde{\theta}^- \cdot \iota^{-1} h'\right). \end{aligned}$$

Each phase product is computed exactly as a dyadic product before Q_p truncates its two components. The factors and disk operands have numerators of magnitude at most 2^p; four signed integer products and their sums therefore use $O(p)$ bits. Truncation shifts the absolute component numerators right by p places and restores their signs, then writes the resulting disk-grid value in the w_{c}-bit component format. It costs $O(p^{1+\delta})$ per coefficient. Generating every factor anew therefore still gives total twist cost $O(Tp^{1+\delta})$. The precision calculation below proves that these calls return disk-grid arrays.

At the h' step, Proposition 18 returns its computed grid numerators after the exact scale M, in the known width $w_M = w_{\mathrm{c}} + \log_2 M$. The disk margin proved below gives $\|h'\| < .701$ in every use here, so each component numerator has magnitude at most 2^p. Before applying ι^{-1}, scan each w_M-bit word and remove only leading sign-extension bits, writing the same integer in w_{c} bits. The retained sign bit agrees with every removed bit by that magnitude bound. This exact normalization costs $O(Tw_M) = O(Tp)$ and supplies the matching scalar format for the output untwist; it introduces no numerical truncation.

9.5 Chirp and source-transform procedures

We next use the complex convolution to evaluate the middle factor $QF_{\boldsymbol{t}}$ in (35), and then compose it with the Gaussian maps. For the chirp array a in (36), the phase numerator at tensor address j is computed modulo $2r$ as

$$-\sum_i s_i j_i^2 (r/t_i) \pmod{2r}.$$

All integers have $O(p)$ bits and $r/t_i \in \{1, 2\}$. Scan the stored axis descriptors and compute the d squares and products with the previously established integer multiplier. Additions, reductions modulo a power of two, and restarting local counters cost $O(p)$ per term; multiplication costs $O(p \log p) = O(p^{1+\delta})$. Exponential evaluation then has the same $O(p^{1+\delta})$ bound. Thus chirp construction costs $O(Tdp^{1+\delta})$ in total. In particular, this cost includes the scan of the axis descriptors at every coefficient.

Let $\widetilde{a}$ be the resulting disk-grid array, so $\|\widetilde{a} - a\|_\infty < 2\,2^{-p}$. For a disk-grid target tensor x of norm at most .6, the computed chirp transform forms the arrays

$$\begin{aligned} x_0 &= Q_p(\overline{\widetilde{a}} \cdot x), \\ y_0 &= \widetilde{\mathcal{M}}_{\mathbb{C}}(\widetilde{a}, x_0), \\ \widetilde{\mathscr{C}}(x) &= Q_p(\overline{\widetilde{a}} \cdot y_0). \end{aligned}$$

Thus $\widetilde{a}$ is the convolution kernel and x_0 is the data operand after the first endpoint phase. Replacing the approximations by exact phases and exact convolution gives $\overline{a} \cdot \mathcal{M}_{\mathbb{C}}(a, \overline{a} \cdot x) = QF_{\boldsymbol{t}}x$ by (36). Below we show that $\|x\|_\infty \le .6$ implies $\|x_0\|_\infty \le .6$, which supplies the smaller operand required by the computed convolution.

9.5.1 The source transform

For a source disk-grid tensor of norm at most .5, write $\widetilde{\mathscr{F}}$ for the following procedure: apply the linewise approximation to $\mathcal{A}$, apply the computed chirp transform $\widetilde{\mathscr{C}}$, apply the linewise approximation to $\mathcal{B}_0$, and multiply the grid numerators exactly by 2^γ. Its exact counterpart is

$$\mathscr{F} = 2^\gamma \mathcal{B}_0 Q F_{\boldsymbol{t}} \mathcal{A} = R F_{\boldsymbol{s}}.$$

Here an exact counterpart replaces every numerical subroutine by the exact map it approximates and omits truncations. The source arrays are stored in the padded power-of-two box, with zeros outside the valid source coordinates. The compressed scalar output uses w_{c}-bit components. Appending γ zero bits to each word gives the exact scaled numerator in the known signed width $w_{\mathrm{c}} + \gamma$. For the stated input norm, the disk estimate below proves that every scaled coordinate still belongs to $\mathbb{D}_p$. Its component numerators therefore have magnitude at most 2^p, so a scan removes only leading sign-extension bits and returns the same integers in w_{c} bits. This scan costs $O(T(w_{\mathrm{c}}+\gamma)) = O(Tp)$ and leaves the padded zero records zero. Define the opposite-sign procedure by

$$\widetilde{\mathscr{F}}^+(z) = \overline{\widetilde{\mathscr{F}}(\overline{z})};$$

its exact counterpart is $RF_{\boldsymbol{s}}^+$ by Lemma 21.

9.6 The multiplication algorithm

The source-transform procedures now supply the three transforms needed for the radix-digit convolution. Read each stored operand backward from its least significant end in groups of b bits. This

produces the radix-2^b digits $a_0, \dots, a_{q-1}$ in increasing coefficient order. Buffer each group on a fixed work tape and read it back most significant bit first, reversing its least-significant-first input order; pad the final short group with leading zeroes. For digit a_j, write the exact grid numerator $a_j 2^{p-b-2}$ in a signed w_c-bit real word, followed by a zero w_c-bit imaginary word. This exponent is nonnegative because $p = 6b$, and the represented value is exactly $a_j 2^{-b-2}$. Append $S - q$ zero coefficient records. Buffering and ordering the groups costs $O(n)$, including the one partial group's padding, and writing the S records costs $O(Sp)$. Together with the initial count and copies, this input conversion costs $O(n + Sp) = O(Tp)$; all n input bits, including leading zeroes, are retained before the final group is padded.

With these streams prepared, perform the following operations.

1. Apply the coefficient-address map Φ, with its padded layout, to obtain source arrays u, v. Their coefficients are exact p-bit grid values of norm less than $1/4$.
2. Compute $\widetilde{\mathscr{F}}(u)$ and $\widetilde{\mathscr{F}}(v)$.
3. Multiply corresponding complex coefficients exactly and truncate each real and imaginary part toward zero to p fractional bits, writing the result in the common w_c-bit component format. Denote the resulting source array by z.
4. Compute $\widetilde{\mathscr{F}}^{+}(z)$ and multiply its grid numerators exactly by S, retaining the denominator 2^p and writing each component in signed width $w_c + b$. Denote the represented dyadic array by $\widetilde{w}$.
5. Multiply the grid numerators of $\widetilde{w}$ exactly by $2^{2b+4}S$, again retaining the denominator 2^p, and write each component in signed width $w_c + 4b + 4$. Round the represented real values to the nearest integers. At each valid address, write the rounded coefficient in one signed $(3b+1)$-bit word, most significant bit first; at every padded address, write a zero word of that width. Supply this local payload width to Φ^{-1} and apply it, including removal of the address padding, to restore the ordinary coefficient order.
6. Propagate radix-2^b carries in increasing coefficient order. Write the resulting bits from least to most significant on a work tape, then reverse that stream and output exactly $2n$ bits in most-significant-first order.

The two occurrences of S in this algorithm undo different normalizations. By (37), the exact counterpart after the three transforms is $(u * v)/S^2$, so after the first scale it is $w = (u * v)/S$. The second scale gives $2^{2b+4}(u * v)$, whose inverse image under Φ is the original integer coefficient convolution. The next subsection verifies the disk bounds needed by every call and proves that the final error is less than $1/2$; it also justifies the rounding, carries, and output length.

9.7 Precision and exact recovery

Proposition 14 uses $C_1 = 20$, so $\epsilon C_1 < 1$ makes its coefficient width $O(p)$. Lemma 16 therefore applies to every synthetic transform here, with error less than $\sqrt{2}\ell\, 2^{-p} < L2^{-p}$, where $L = \log_2 T$, for all sufficiently large n.

9.7.1 Scales and disk margins

Put $h_p = 2^{-p}$, and use the coefficient sup norm throughout this subsection. An error of a will mean absolute error less than ah_p. A linear contraction propagates an input error without increasing it.

For a bilinear contraction $\mathcal{M}$, replacing its operands gives

$$\|\mathcal{M}(f',g') - \mathcal{M}(f,g)\| \le \|f'-f\|\,\|g'\| + \|f\|\,\|g'-g\|.$$

When all four operands are in the disk, their two errors therefore add.

We also need the norm as well as the error of a computed phase product. Let e be a unit phase, let $\widetilde{e}$ be a disk-grid approximation with $|\widetilde{e}-e| < 2h_p$, and let x be a disk operand. The truncation bound in (26) gives

$$\|Q_p(\widetilde{e}x) - ex\| < (2\|x\| + \sqrt{2})h_p < 4h_p, \qquad \|Q_p(\widetilde{e}x)\| \le \|x\|.$$

The second inequality follows from $|\widetilde{e}| \le 1$ and the fact that Q_p does not increase the norm. It preserves every smaller operand norm through a twist or an endpoint phase multiplication. The same bounds hold coordinatewise for arrays of phases.

For disk-grid ring arrays f, g, put $M = T/r$. Two normalized synthetic forwards, pointwise polynomial multiplication divided by r, and the normalized opposite transform give $(f*g)/(rM^2)$. The exact multiplication by M in Proposition 18 then gives $\mathcal{M}_r(f,g) = (f*g)/T$. Before that multiplication, the three transform errors and the polynomial-product truncation total less than $3L+2$. Thus the error of the returned normalized convolution is bounded by

$$E_{\text{ring}} := M(3L+2) < 5TL.$$

If one operand has norm at most .7, bilinear contraction gives $\|\mathcal{M}_r(f,g)\| \le .7$. Its computed grid output consequently has norm less than $.7 + 5TLh_p < .701$ for large n. This proves the disk margin after the scale M, without clipping the output.

Consider now $\widetilde{\mathcal{M}}_{\mathbb{C}}(f,g)$ for disk grid operands, one of norm at most .7. Its two computed input twists f', g' are disk grid arrays, and the smaller one still has norm at most .7 by the phase inequality. The identification ι preserves these bounds. The preceding ring margin therefore gives $\|h'\| < .701$. Neither ι^{-1} nor the computed output untwist increases this norm. In particular, $\widetilde{\mathcal{M}}_{\mathbb{C}}(f,g)$ is a disk grid array.

To compare it with $\mathcal{M}_{\mathbb{C}}(f,g)$, insert the exact normalized $\mathcal{M}_r$ at $\iota f'$ and $\iota g'$. Its bilinear contraction bound propagates the two input twist errors, each less than 4. Its approximation contributes E_{ring}, and the output untwist contributes less than 4. All these comparisons are at the normalized $\mathcal{M}_r$ interface, after its factor M. They give

$$\|\widetilde{\mathcal{M}}_{\mathbb{C}}(f,g) - \mathcal{M}_{\mathbb{C}}(f,g)\| < (E_{\text{ring}} + 4 + 4 + 4)h_p < 6TLh_p.$$

For a disk-grid chirp input x with $\|x\| \le .6$, the previously defined data operand $x_0 = Q_p(\overline{\widetilde{a}} \cdot x)$ has norm at most .6. The kernel $\widetilde{a}$ is a disk grid array. Thus this pair satisfies the smaller-operand condition just proved: $\|y_0\| < .701$, and the last endpoint multiplication gives $\|\widetilde{\mathscr{C}}(x)\| < .701$. In particular, the chirp output is a legal disk-grid input to the compression maps.

For its error, the exact operands a and $\overline{a} \cdot x$ are in the disk because every entry of a has modulus one. The kernel approximation contributes less than 2 and the first endpoint multiplication contributes less than 4 when inserted into the exact bilinear contraction $\mathcal{M}_{\mathbb{C}}$. The computed convolution contributes less than $6TL$, and the last endpoint multiplication less than 4. Equation (36) identifies the exact result, so

$$\|\widetilde{\mathscr{C}}(x) - QF_{\boldsymbol{t}}x\| < (6TL + 2 + 4 + 4)h_p < 8TLh_p.$$

Finally let the input to a source transform have norm at most .5. The computed expansion has error less than dp^2 and norm less than $.5 + dp^2h_p < .51$. It is therefore a permitted chirp input.

The chirp output is in the disk by the preceding argument, so each compression call is defined and returns another disk-grid array. Insert the exact contractions $\mathcal{A}$, $QF_{\boldsymbol{t}}$, and $\mathcal{B}_0$ at the computed inputs. The error before the source scale is less than $2dp^2 + 8TL$. Multiplication by 2^γ therefore gives error less than $2^\gamma(2dp^2 + 8TL)h_p$, which is bounded by $E_s h_p$ with

$$E_s = 9\,2^\gamma TL, \tag{43}$$

because $2dp^2 < TL$ eventually. The exact source transform $RF_{\boldsymbol{s}}$ is a contraction, so the scaled approximation has norm at most $.5 + E_s h_p < .501$. Its exact numerator scaling therefore also leaves a disk-grid output without clipping. Conjugation preserves the grid and norm, so the opposite procedure has the same error and disk guarantee. All these margins may be enforced by one fixed cutoff. For example, $b \geq 2^{24}$, together with the other eventual construction conditions, makes $dp^2 h_p$, $8TLh_p$, $E_s h_p$ and $28\,2^\gamma T^2 L h_p$ smaller than 10^{-3}.

9.7.2 The final integer coefficients

The scaled radix digits in u, v are exact p-bit grid values of norm strictly less than $1/4$. Put

$$\begin{aligned} U &= RF_{\boldsymbol{s}}u, & V_1 &= RF_{\boldsymbol{s}}v, \\ \widetilde{U} &= \widetilde{\mathscr{F}}(u), & \widetilde{V}_1 &= \widetilde{\mathscr{F}}(v). \end{aligned}$$

The exact arrays have norm less than $1/4$, and the computed arrays have norm less than $1/4 + E_s h_p < .26$. Hence $z = Q_p(\widetilde{U} \cdot \widetilde{V}_1)$ has norm at most $.26^2 < .5$. To estimate its error, write

$$\widetilde{U} \cdot \widetilde{V}_1 - U \cdot V_1 = (\widetilde{U} - U) \cdot \widetilde{V}_1 + U \cdot (\widetilde{V}_1 - V_1).$$

The factors $\widetilde{V}_1$ and U are in the disk. The two terms have total norm less than $2E_s h_p$, and truncation adds less than $2h_p$. Thus $\|z - U \cdot V_1\| < (2E_s + 2)h_p$.

Let $G := RF_{\boldsymbol{s}}^+$ be the exact opposite contraction. Lemma 21, applied to $U = RF_{\boldsymbol{s}}u$ and $V_1 = RF_{\boldsymbol{s}}v$, gives

$$G(U \cdot V_1) = (u * v)/S^2.$$

The computed opposite transform may be applied to z because $\|z\| < .5$. Insert G at that computed input:

$$\begin{aligned} \|\widetilde{\mathscr{F}}^+(z) - G(U \cdot V_1)\| &\leq \|\widetilde{\mathscr{F}}^+(z) - Gz\| \\ &\quad + \|G(z - U \cdot V_1)\| \\ &< (3E_s + 2)h_p. \end{aligned}$$

The first exact numerator multiplication by S gives $\widetilde{w}$, approximating $w = (u * v)/S$ with error less than $E_w h_p$, where

$$E_w := S(3E_s + 2) < 28\,2^\gamma T^2 L.$$

Each coefficient of w is an average of S products of coefficients of u and v, so $\|w\| < 1/16$. The earlier cutoff therefore gives

$$\|\widetilde{w}\| < 1/16 + E_w h_p < 1/16 + 10^{-3} < 1.$$

This proves the disk bound for the computed first scaled result.

In tensor order, the unscaled integer coefficient array is $2^{2b+4} Sw$. This second occurrence of S reverses the normalization still present in w, and 2^{2b+4} reverses the two input scales. Both

multiplications act exactly on grid numerators. Since $S \le T < 2^b$ and $L < b$ eventually, the final absolute error is less than

$$2^{2b+4} S \cdot 28\, 2^{\gamma} T^2 L \cdot 2^{-6b} < 448b\, 2^{-b+\gamma} \le 448b\, 2^{-3b/4} < \frac{1}{2}. \tag{44}$$

The true coefficients are real integers. If H_j is the real numerator after the second scaling, its represented value is $H_j/2^p$. The strict error below $1/2$ makes the nearest integer to that value the unique correct coefficient.

These exact operations use known fixed record widths. If N is a signed w_0-bit integer, then $-2^{w_0-1} \le N < 2^{w_0-1}$. Since $0 < S < 2^b$, its product satisfies $-2^{w_0+b-1} < SN < 2^{w_0+b-1}$ and therefore fits in signed width $w_0 + b$. The first product by S starts at width w_c and is written in width $w_c + b$. The next product by S is written in width $w_c + 2b$; appending $2b + 4$ least significant zero bits then gives the exact second scale in width $w_c + 4b + 4$. Each complex record still places its most-significant-bit-first real word before its imaginary word. The known widths determine every boundary in the next scan and are $O(p)$ because $p = 6b$.

The established integer multiplier performs the two products by S in $O(p \log p)$ time per entry. To serialize each exact signed product at its stated width, zero-pad its magnitude and complement and increment for a negative sign. The proved bounds ensure that no significant bit is discarded. These conversions and the power-of-two shift use linear scans. To round $H_j/2^p$, recover the real magnitude, inspect its low p bits, and apply the rounding carry with the sign restored. The strict error bound makes the result the correct nonnegative coefficient, and (41) puts it in $[0, 2^{3b})$. It therefore fits in one signed $(3b + 1)$-bit two's-complement word, written most significant bit first. Scan the padded box in the physical axis order recorded by its shape descriptor, using the same $O(p)$-bit nested counters and validity tests as above. Consume the imaginary word and write this real coefficient word at each valid address, with an all-zero word at each padded address. This serialization costs $O(Tp)$ in total. It keeps the tensor address order and supplies payload width $3b+1$ to the local descriptors for Φ^{-1}, so the address routine receives a stream of known fixed-length records.

Lemma 17 proves exact signed packing, centered extraction and negacyclic reduction. Its only numerical error is the stated truncation after division by r, already included above. Undo the coefficient-address isomorphism Φ before propagating radix carries.

To propagate carries, let $Q_0 = 2^b$. The recovered ordinary coefficients are the c_j in (41); they are zero for $2q - 2 < j < S$. With $q = \lceil n/b \rceil$ as above, use the recurrence for $0 \le j < S$:

$$k_0 = 0, \qquad d_j = (c_j + k_j) \bmod Q_0, \qquad k_{j+1} = \left\lfloor \frac{c_j + k_j}{Q_0} \right\rfloor.$$

The carries satisfy $0 \le k_j \le q(Q_0 - 1) < Q_0^2$. If this holds at j, then

$$c_j + k_j \le q(Q_0 - 1)^2 + q(Q_0 - 1) = qQ_0(Q_0 - 1),$$

which proves the next carry bound. The defining relation $c_j + k_j = d_j + Q_0 k_{j+1}$ also gives the exact invariant

$$\sum_{h<j} c_h Q_0^h = \sum_{h<j} d_h Q_0^h + k_j Q_0^j.$$

It holds for $j = 0$, and the defining relation proves its next instance. The unprocessed terms are multiples of Q_0^j, so the identity fixes the first j radix digits of the full product. The value k_j is the carry from the processed coefficients into the remaining positions.

After Φ^{-1} the coefficients occur in increasing ordinary index, so this carry scan proceeds from least to most significant. Carry storage and work per coefficient are $O(b)$ bits and steps. Write each digit's bits from low to high on a work tape, and drain the final $O(b)$-bit carry in the same order. Copy the completed stream backward once to obtain most-significant-first order. The stream has length $O(Sb) = O(Tp)$, including chunk padding and the final carry, so this reversal is a charged linear tape pass. The exact product is less than 2^{2n}; all bits above position $2n-1$ are therefore zero. Remove only those zero padding bits and retain leading zeros as needed to produce exactly $2n$ bits. Zero inputs obey the same invariant and output rule.

9.8 The complete time bound

For the cost calculation put $V = Tp = \Theta(n)$. The fixed-width coefficient formats use $\Theta(p)$ bits per coefficient, so a full padded array has $\Theta(V)$ stored bits. Table 1 lists costs after division by V; a fixed number of repeated transforms and convolutions is included in its constants.

Operation	Normalized cost	Power of p
Prefix-slot moves and individual butterfly rounds	dK	$\epsilon(1+c)$
Chunk exchanges for transform layout	$pK^{\tau-1}$	$1-\epsilon c(1-\tau)$
Simultaneous butterfly rounds	$\ell d^{\lambda'}$	$1-\epsilon(1-\lambda')$
CRT and axis layouts	$d^2(1+\ell^\tau)$	$\tau+\epsilon(2-\tau)$
Gaussian line maps	$dp^{1/2+\delta}\alpha$	$3/4+\delta+3\epsilon/2$
Chirps, twists and scalar products	dp^δ	$\epsilon+\delta$
Packed polynomial products	$\log(rp)$	$1-\epsilon$
Final scaling and rounding	p^δ	absorbed above
Input, padding, carries and output	1	absorbed above

Table 1: Costs divided by Tp; each coefficient representation has $O(p)$ bits.

The seven dominant powers in Table 1 have the following margins below one:

$$\begin{aligned} &g_1 = 1-\epsilon(1+c), \qquad g_2 = \epsilon c(1-\tau), \qquad g_3 = \epsilon(1-\lambda'),\\ &g_4 = 1-\tau-\epsilon(2-\tau), \qquad g_5 = 1/4-\delta-3\epsilon/2,\\ &g_6 = 1-\delta-\epsilon, \qquad g_7 = \epsilon. \end{aligned} \tag{45}$$

Direct substitution gives

$$g_1 > \frac{1}{2}, \quad g_2 = 2^{-181}, \quad g_3 = 2^{-129}, \quad g_4 > 2^{-51}, \quad g_5 > \frac{1}{8}, \quad g_6 > \frac{1}{2}, \quad g_7 = 2^{-75}.$$

Thus $\min_i g_i = 2^{-181} = 2\kappa$. This explicit slack will cover any remaining fixed power of $\log p$.

Here d^2 is absorbed by $d^2\ell^\tau$, and $\log(rp) = O(p^{1-\epsilon} + \log p) = O(p^{1-\epsilon})$. The first three rows are the terms in (30). The prefix-slot moves and individual butterfly rounds in Lemma 16 use $O(dK)$ linear scans. Its positional layout uses $O(p/K)$ exchanges of width-K chunks, because $d\ell = O(p)$. Each exchange costs $O(VK^\tau)$ by Lemma 12, so the normalized total is

$$O(p/K)\,K^\tau = O(pK^{\tau-1}).$$

For the at most ℓ simultaneous rounds, Proposition 14 supplies $O(Vd^{\lambda'})$ per round. Its bound already includes the binary basis changes, deterministic repair, row padding and removal, all base-m groups, and fixed-tape cleanup.

Axis exposure and restoration require $O(d^2)$ adjacent axis moves. Each costs $O(V(1+\ell^\tau))$: peel a single excess bit when widths differ by one, exchange their equal-width parts, and restore the bit. The d CRT rotations copy valid and invalid intervals in $O(dV)$ time. Their offset setup was bounded above by $O(dTp^C/r) = o(Tp)$, using at most T/t_i control prefixes for target i.

For Gaussian resampling, axis i has at most T/t_i processed lines. The line-machine costs therefore sum to

$$\sum_{i=1}^{d} \frac{T}{t_i} O(t_i p^{3/2+\delta}\alpha) = O(dTp^{3/2+\delta}\alpha).$$

Dividing by $V = Tp$ gives the fifth row. The factor $\alpha = \Theta(p^{1/4+\epsilon/2})$ explains its displayed power of p.

Chirp numerators have $O(p)$ bits and can be computed modulo $2r$ as sums of d terms $s_i j_i^2 (r/t_i)$. Computing their squares and products with the previously established $O(N\log(2N))$ multiplier on N-bit inputs, then generating the exponentials, costs $O(dp^{1+\delta})$ per entry. The smaller twist and scalar pointwise-product costs fit the same row: they use a fixed number of exact products on $O(p)$-bit component numerators, followed by linear truncation scans. The two final products by S also cost $O(p\log p)$ per entry, so their normalized cost is $O(\log p) = O(p^\delta)$ as listed separately. There are T/r polynomial records in each pointwise ring product. Four integer products on $O(rp)$ bits per record therefore cost $O(Tp\log(rp))$ in total. Signed packing, centered extraction, negacyclic wrapping, and copying are linear stream passes. These calls use the previously established multiplier on records of length $O(rp) = o(n)$; they do not call the improved algorithm being constructed. The exact source scaling by 2^γ is a shift of $O(p)$-bit numerators by $\gamma = O(p)$ places, so it is a linear scan per entry and fits the final-scaling row. The scalar-format adapters and the exact removal of sign-extension bits after the ring and source scales also use linear scans of their $O(p)$-bit components, within the listed scan costs.

Setup and exceptional streams across all calls. We next separate setup at recursion nodes from work repeated for every polynomial record or every Gaussian line. Here a visit means a node of the layer or address-layout recursions, or one of their round and group invocations. The internal recursion of the established integer multiplier and the iterations of the quoted Gaussian algorithms are already included in their separately charged running times. The layer and address-layout recursions have constant branching and depth $O(\log p)$, and the number of their rounds and base-m groups is polynomial in p. Choose a fixed exponent A so that their total number of visits is at most p^A, and a fixed C bounding descriptor setup performed once per visit by p^C steps. This setup then costs at most $p^{A+C} = o(Tp)$.

At a layer or synthetic-transform visit that processes polynomial records, there are at most $O(T/r)$ such records, including the bounded row padding. Every relevant child keeps the whole length-r polynomial suffix. Consequently $O(p^C)$ descriptor or address work per record, over these visits, costs at most $O(Tp^{A+C}/r) = o(Tp)$. Setup before each Gaussian line call has a different count: there are at most $2dT/r$ lines per tensor map and only a fixed number of such maps, so $O(p^C)$ setup per line costs $O(dTp^C/r) = o(Tp)$. These estimates all use that r exceeds every fixed power of p.

The general address-swap subroutine also covers one-bit payloads, where no polynomial suffix is available. For root volume V_0 and current child volume V_j, the descriptor estimate in the proof of Proposition 11 gives $(\log(2V_0))^{O(1)} = O(V_j)$. It uses the invariant that both original chunk ranges remain present in every child, and therefore includes polynomial descriptor work even for a one-bit payload. Elementary counter updates use local length copies, so they do not rescan a

growing descriptor for each data bit. The chirp computation does scan the axis descriptors at every coefficient; its $O(Tdp^{1+\delta})$ cost was included in the chirp row.

For one packed change of basis, let V_{call} be its current logical volume. The bad-address fraction is $O(d2^{-K})$, independently of the payload. Each polynomial record has $\Theta(rp)$ payload bits, and an exceptional record adds an $O(p)$-bit destination key. Thus the compact exceptional stream has length

$$O\left(\frac{V_{\mathrm{call}}}{rp}\, d2^{-K}(rp+p)\right) = O(V_{\mathrm{call}} d2^{-K}).$$

Stable bit sorting makes $O(p)$ scans of this stream, for one-call cost $O(V_{\mathrm{call}} dp2^{-K})$. Extraction and reinsertion use a constant number of scans of the full current volume; those scans are already in the node overhead. Since there are at most p^A visits, each of volume $O(V)$, the compact sorting costs sum to at most

$$O\left(dp2^{-K}\sum_{\mathrm{calls}} V_{\mathrm{call}}\right) = O(Vp^{A'}2^{-K}) = o(V)$$

for another fixed exponent A'. Here $d = O(p^{\epsilon})$ and $K = \Theta(p^{\epsilon c})$ with $\epsilon c > 0$. Predicate and key arithmetic is the polynomial work per record counted above. This is a deterministic worst-case bound.

Finally, all role, arithmetic, and stack tapes form fixed finite families. A child parks and restores only its parent's streams, with the stack head at the active top; copying and cleanup are charged to that logical volume. No operation repeatedly scans parked ancestors. Growing axis lists and call states are stored data, not extra tapes or alphabet symbols. Global setup and prime search cost $o(V)$, as proved earlier.

Each of the seven displayed powers is at most $1 - 2\kappa$ by (45). Any fixed power of $\log p$ is at most p^{κ} eventually. The complete large-input bound is consequently

$$O(Tp\, p^{1-\kappa}) = O(n(\log n)^{1-\kappa}).$$

Fix one cutoff beyond all the size, precision, margin and layout conditions, and use schoolbook multiplication below it. The finitely many smaller lengths are included by enlarging the implicit constant. With $\lg n = \max\{\lceil \log_2 n \rceil, 1\}$, the resulting single fixed-alphabet, fixed-tape deterministic machine has worst-case time $O(n(\lg n)^{1-\kappa})$ for every input length and produces precisely the required $2n$-bit product.

What a successful check actually tells us

Added explanation Three claims that should not be confused.

There are different meanings of "it works." A circuit may return the right values on a specified finite set of inputs. A physical program may execute the circuit with a proved resource bound. A theorem may cover every input length, including sizes no experiment can run. Each is valuable, but they are different statements.

Three different claims require three different kinds of evidence

Algebra example	Concrete tape program	All-size proof
Outputs match, and scratch returns to its starting value	Head movement, copies, routing, and cleanup are charged	Every input size satisfies the cost and error bounds
Finite replay checks a concrete instance	A scalar identity alone does not price movement	A finite run alone does not prove every size

Figure E24. Read the claim at the level its evidence supports. The columns separate finite algebra, physical implementation, and all-size analysis. They are obligations to connect, not competing ways of proving the same statement.

A concrete example. The register plot shows that the four updates work when the scratch value starts at seven. The symbolic calculation proves restoration for arbitrary values in its stated ring. Neither statement tells us how much tape movement an enormous array implementation needs. That requires another argument. Similarly, verifying the seven rational margins checks arithmetic within a cost model; it does not establish every algorithmic premise of that model.

Why the guide labels toy examples. Large, visible exponent gaps are useful for learning but would misrepresent the paper if presented as its measured speedup. Small matrices and arrays can demonstrate an invariant without being the actual finite construction. The captions say which numbers come from the source and which are teaching examples. The new figure generators check the displayed arithmetic so that the illustrations explain real identities rather than suggestive pictures.

After this guide. You should be able to describe why convolution helps, why moving data costs time, why common frames preserve addition, how a weighted recursion saves an exponent, and why recovery requires a strict error budget. Following every detail of the source theorem still requires further study of linear algebra, Fourier analysis, and complexity proofs. The expanded edition puts the original arguments immediately after their explanatory material.

What to carry forward. Understanding the architecture is a substantial step. It is not the same claim as having independently verified every part of the theorem.

What follows, and what still needs checking

Added explanation Read before the final corollaries and appendix.

Division and square root. Fast multiplication is a building block for other arithmetic. Newton-style iterations improve reciprocal or square-root approximations at geometrically increasing precisions. If the cost at precision n is $M(n)$ and the smaller stages cost $M(n/2), M(n/4), \ldots$, the sum is controlled by the largest stage for the regular bound in this manuscript. Approximate answers must still be corrected: division verifies quotient and remainder, and integer square root uses exact square comparisons. An approximation routine alone is not an exact division or floor-square-root algorithm.

Matrix transposition. The paper invokes an external reduction from multiplication to rearranging a row-major matrix into column-major order. It accounts for the total number of matrix bits, not just its side length. This is a result about data permutation. It is not a claim about the exponent of matrix multiplication, which is a different problem. The reduction and its input-format conversions retain their own assumptions and costs.

The grouped-rectangle appendix. The appendix extends packed address manipulations to a collection of complete rectangular blocks. "Rectangle" refers to a Cartesian product of address ranges, not a geometric approximation to arbitrary data. Full payloads and destination information must be charged when exceptional records are extracted and repaired. This material supports layout details rather than introducing a separate headline improvement.

A practical reading checklist. When evaluating a proposed improvement, ask these questions in order:

1. What exact scalar operation does the emitted circuit perform, with arbitrary initial scratch values? Does it restore the promised workspace?
2. Do all incidences of each gate have the required common frame? Are all changed operators, signs, source contributions, and endpoints included?
3. Does the paid rank profile describe that circuit, rather than a relaxed or schematic construction? Are finite-field specializations justified?
4. Does the recursive implementation truly give each child volume V/W, with routing, padding, copying, and restoration charged?
5. Do the precision and resampling contracts still apply to the new layouts?
6. Are every final cost margin and the recovery error strictly within their required limits for the same parameter choice?

These questions separate algebra, implementation, and asymptotic analysis. A success at one level is useful evidence at that level; it does not silently discharge the others.

Verification performed for this edition. The frozen original files were checked against their supplied SHA-256 manifest. The digit-convolution example, the four-update restoration identity on 343 integer triples, the two-axis butterfly example, and all seven original rational margins were recomputed. The second edition also checks interpolation, frequency-pattern reconstruction, vector and array frame identities, intersection cancellation, permutation compatibility, and the illustrative error budgets. The figures are generated from those examples or from explicitly labeled schematic identities. Compilation and rendered-page inspection check document integrity and readability. None of these actions constitutes a new audit of the full multiplication theorem.

What to carry forward. The guide explains the proof's architecture and its interfaces. The following source material supplies the detailed arguments; independent validation of the complete theorem is a separate research task.

Original manuscript continues below.

10 Exact division and integer square root

Let n be the supplied input width, let a be the nonnegative dividend or radicand, and let $b > 0$ be the divisor for the division task, with the values and output conventions of Corollary 2. Here $p \geq 1$ denotes an integer target precision for a Newton routine.

Proof of Corollary 2. We use the classical Newton reductions as presented by Brent [5, Lemmas 2.1–2.4], with their tape costs made explicit. Set $B(p) = p(\lg p)^{1-\kappa}$. A linear scan handles a zero dividend or radicand and finds the significant bits of a positive input. For division write $b = 2^h c$, where $0 \leq h < n$ and $1 \leq c < 2$. For square root write $a = 2^{2e} c$, where $0 \leq e < n/2$ and $1 \leq c < 4$. Copy the normalized digits most significant first to a tape with a marked beginning. This initial scan also records the relevant exponent.

For an integer $t \geq 1$, let $R_t(z) = 2^{-t}\lfloor 2^t z \rfloor$ and put $\delta = 2^{-t}$. The prefix $c_t = R_t(c)$ has $0 \leq c - c_t < \delta$. Each product below is computed exactly on integer significands and then rounded by the indicated R_t; additions and shifts are exact until a stated rounding. When precision increases, the stored dyadic iterate is extended by zero fractional bits.

For the reciprocal iteration $x \leftarrow x(2 - cx)$, compute

$$u = R_t(c_t x), \qquad x^+ = R_t\big(x(2 - u)\big).$$

Put $\varepsilon = 1 - cx$. If $|\varepsilon| \leq 1/2$, then $0 < x \leq 3/2$. Writing $\eta_1 = u - c_t x$ and $\eta_2 = x^+ - x(2 - u)$ gives $|\eta_i| < \delta$ and

$$1 - cx^+ = \varepsilon^2 - c(c - c_t)x^2 + cx\eta_1 - c\eta_2.$$

Since $cx^2 = (1 - \varepsilon)^2/c \leq 9/4$, $cx \leq 3/2$, and $c < 2$,

$$|1 - cx^+| \leq \varepsilon^2 + \frac{23}{4}\delta < \varepsilon^2 + 6\delta. \tag{$*$}$$

The fixed seed $x = 5/8$ has $|\varepsilon| \leq 3/8$ throughout $1 \leq c < 2$. One step with $t = 6$ therefore gives residual less than $9/64 + 6/64 = 15/64 < 1/4$.

For a requested target $p \geq 1$, begin with $m = 1$ after this warm-up. While $m < p$, set $m' = \min\{2m, p\}$ and work at $t = m' + 5$ fractional bits. If the residual is at most 2^{-m-1}, then $(*)$ gives

$$|1 - cx^+| \leq 2^{-2m-2} + 6\,2^{-m'-5} \leq \frac{7}{16}\,2^{-m'} < 2^{-m'-1}.$$

Set $m = m'$ and continue. The reciprocal routine at target p thus returns a dyadic x satisfying $|1 - cx| \le 2^{-p-1}$ and $|x - 1/c| \le 2^{-p-1}$. These errors are measured against the true c at every stage, although the prefix c_t changes.

For the root iteration $x \leftarrow (x + c/x)/2$, first suppose $1/2 \le x \le 5/2$. Normalize this stored dyadic exactly as $x = 2^j d$, where $1 \le d < 2$ and $j \in \{-1, 0, 1\}$. The reciprocal routine at target t returns z with $|1 - dz| \le 2^{-t-1}$. Put $v = 2^{-j} z$. Because $d \ge 1$ and $2^{-j} \le 2$, we have $|v - 1/x| \le 2^{-t} = \delta$. Compute the quotient and root update by

$$q_t = R_t(c_t v), \qquad x^+ = R_t\big((x + q_t)/2\big).$$

Here $c_t < 4$ and $1/x \le 2$, so the quotient has the uniform error

$$|q_t - c/x| \le \delta + c_t|v - 1/x| + \frac{|c_t - c|}{x} \le 7\delta.$$

The exact Newton identity gives

$$|x^+ - \sqrt{c}| \le \frac{(x - \sqrt{c})^2}{2x} + \delta + \frac{1}{2}|q_t - c/x| \le \frac{(x - \sqrt{c})^2}{2x} + \frac{9}{2}\delta.$$

The fixed seed $x = 3/2$ has $|x - \sqrt{c}| \le 1/2$. At this seed the first term is at most $1/12$, so one step with $t = 6$ has error less than $1/12 + 5/64 < 1/4$. Thereafter the invariant $|x - \sqrt{c}| \le 2^{-m-1}$, with $m \ge 1$, gives $3/4 \le x \le 9/4$ and hence supplies the denominator bound for the next step. Since $1/(2x) \le 1$ and $9/2 < 6$, the same capped doubling schedule with five guard bits gives $|x - \sqrt{c}| \le 2^{-p-1}$ at target p.

These operations use a fixed number of scalar registers. In a reciprocal step, $x \le 3/2$ and $u \le c_t x \le 3/2$ whenever $(*)$ is used. In a root step, $x \le 5/2$, $v \le 2 + \delta < 3$, and $q_t < 12$. A fixed number of integer guard bits therefore suffices. At working precision t, each retained significand has $t + O(1)$ bits, including the fixed additional precision of the inner reciprocal, and each exact product has $2t + O(1)$ bits. Theorem 1 applies to the padded significands. The streaming conventions give $O(t)$ cost for additions, shifts, copying, truncation and head returns.

Each stage starts at the marked beginning of its normalized input, reads only the required prefix, and returns over that prefix. If the finite input ends first, the remaining fractional bits are zero. A root stage forms d once in $O(t)$ time; each stage of its inner reciprocal routine likewise reads only its own shorter prefix of d. Thus no early stage rescans an entire longer input.

For the capped doubling targets m_j ending at p, $\sum_j (m_j + 5) = O(p)$, and monotonicity of lg gives

$$\sum_j B(m_j + 5) \le (\lg(p + 5))^{1-\kappa} \sum_j (m_j + 5) = O(B(p)).$$

The fixed warm-up adds constant cost. A reciprocal stage uses two products, so the complete reciprocal routine at target p costs $O(B(p))$. A root stage of working precision t uses a reciprocal at target t and one further product, and hence costs $O(B(t))$. The outer precision sum is again $O(B(p))$. The scalar scan costs sum to $O(p)$ at both levels.

The outer registers and the reused reciprocal and multiplier tapes form one fixed finite tape set with fixed subroutine depth. Marking the ends of each call's visited intervals on fixed tracks permits those intervals to be cleared and their heads returned in a constant multiple of the call's running time. No random-access simulation is used.

Take $P = 2n + 4$. At this target there are $O((\lg n)^2)$ outer and inner stage calls. Their $O(\lg n)$-bit length and precision descriptors incur a fixed polynomial in $\lg n$ of total work, hence $O(n)$. Together with the initial normalization scan, this accounts for the fixed-tape implementation. For

division, the reciprocal routine gives $|x - 1/c| \le 2^{-P}$. Form $a2^{-h}x$ by an exact significand product and an exact shift of the binary point, retaining its fractional bits. Since $a < 2^n$,

$$\left| a2^{-h}x - \frac{a}{b} \right| \le a2^{-h}2^{-P} < 2^{n-P} < \frac{1}{4}.$$

Its floor q_0 differs from $q = \lfloor a/b \rfloor$ by at most one. Compute $r_0 = a - bq_0$ exactly. If $r_0 < 0$, decrement q_0 and add b; if $r_0 \ge b$, increment q_0 and subtract b. For square root, an approximation to $\sqrt{c}$ within 2^{-P} becomes an approximation to $\sqrt{a}$ within $2^{e-P} < 1/4$ after the exact shift by 2^e. Its floor s_0 differs from $\lfloor \sqrt{a} \rfloor$ by at most one. Decrement it if $s_0^2 > a$, and increment it if $(s_0 + 1)^2 \le a$; otherwise retain it. The remainder and square tests cover exact quotients and perfect squares. All final significands, trial answers and exact test products have $O(n)$ bits; their computation costs $O(B(n))$, and output padding is linear. Leading zeroes affect neither normalization bound, since n is the supplied input length. □

Original manuscript continues below.

11 A consequence for matrix transposition

For positive integers n_1, n_2, b, a row-major $n_1 \times n_2$ array of b-bit strings stores entry (i, j), where $0 \le i < n_1$ and $0 \le j < n_2$, at position $n_2 i + j$; transposition places this string at position $n_1 j + i$. Harvey and van der Hoeven prove that any multiplication machine with worst-case cost $M(m)$ yields a fixed multitape transposition machine with cost [16, Corollary 6.2]

$$O(M(m) + m \lg \lg \max(n_1, n_2)), \qquad n_1 n_2 b \le m.$$

The cited transposition interface receives m as well as n_1, n_2, b and the matrix stream [16, Section 1.1]. For the matrix problem defined here, compute $m = n_1 n_2 b$ from the three binary descriptors and supply all four parameters. The four descriptors have total length $O(\log(2m))$, and the two schoolbook products that compute m cost $O(\log^2(2m)) = O(m)$ steps.

The cited multiplication interface likewise receives m explicitly and two nonnegative operands smaller than 2^m; it uses standard binary representations of the operands and product. On each such call, read the supplied m and form two words of exactly m bits, most significant bit first, by adding leading zeroes. Feed their $x\#y$ concatenation to Theorem 1. Its final output pass writes the $2m$-bit product consecutively. Scan this block to remove leading zeroes, emitting one 0 bit if every bit is zero. This fixes a canonical binary representation of the product. If a tape orientation requires reversal, read the stored word backward while writing it forward on a spare tape. Padding, normalization, and copying therefore cost $O(m)$ steps on a fixed number of tapes.

Suppose the multiplication bound is $O(m(\lg m)^{1-\kappa})$ for a fixed $0 < \kappa < 1$. Positivity of the dimensions and entry width gives $\max(n_1, n_2) \le n_1 n_2 b \le m$. Hence

$$\lg \lg \max(n_1, n_2) \le \lg \lg m = o((\lg m)^{1-\kappa}).$$

The final relation uses $1 - \kappa > 0$; the bounded cases are absorbed using the stated convention for lg. Thus the cited transposition cost is $O(m(\lg m)^{1-\kappa})$. Taking $m = n_1 n_2 b$ expresses this bound in matrix bits. In particular, an $n \times n$ binary matrix can be transposed in $O(n^2(\lg n)^{1-\kappa})$ time. Dimensions equal to one cause no exception, since our convention gives $\lg 1 = 1$.

Original manuscript continues below.

A Packed additions on grouped rectangles

The simultaneous-layer construction uses the rectangular layout of Lemma 13. We record here a more general version in which a supplied descriptor lists groups with different spectator lengths. Each group is a complete rectangle, so the full rectangular algorithm, including its deterministic repair, can be applied to one group at a time. The only additional work is reading the group descriptors and passing consecutive groups through the same local tapes.

Definition 22 (Grouped packed stream)**.** Let $M, R \geq 1$, $K \geq 6$, $f \geq 1$, $L = fK$, and $0 \leq \rho < K$. Fix an ordered template of $t \geq 3$ two-bit tags: `00` denotes a spectator field, and `01`, `10`, `11` denote x, y, z, respectively. Each chunk tag occurs exactly once. The template is common to all $J \geq 1$ groups of this input, but its length may vary between inputs. Group ν has a positive row count n_ν and a positive length $a_{\nu,h}$ at each spectator tag h; at a chunk tag put $a_{\nu,h} = 2^L$. Define

$$Q_\nu = \prod_{h=1}^{t} a_{\nu,h}, \qquad M_\nu = n_\nu Q_\nu, \qquad C_\nu = \sum_{\mu<\nu} M_\mu, \qquad V_\nu = M_\nu R,$$

and require $M = \sum_{\nu=1}^{J} M_\nu$.

The stream consists of M records of R bits. In group order, their addresses form the consecutive lexicographic rectangles

$$[n_\nu] \times \prod_{h=1}^{t} [a_{\nu,h}].$$

The first coordinate is a row ordinal. Every row contains the full Cartesian suffix displayed above: no suffix length depends on a coordinate within that row. The suffix lengths may differ between groups. Every finite sequence of per-row spectator length vectors in the fixed template is representable, using groups of count one when necessary.

Use the integer code Γ from (1). The supplied descriptor is exactly

$$\mathcal{D} = \Gamma(M)\Gamma(R)\Gamma(K)\Gamma(f)\Gamma(\rho+1)\Gamma(t)\,T_1 \cdots T_t\,\Gamma(J) \cdot \prod_{\nu=1}^{J} \left(\Gamma(n_\nu) \prod_{h:T_h=\mathtt{00}} \Gamma(a_{\nu,h}) \right),$$

where T_h is the tag at position h, and juxtaposition and products mean bitstring concatenation in increasing index order. The header, tag count, and group count delimit the entire list. A grouped packed stream is valid when this descriptor and its accompanying records have exactly the parameters, shapes, and orders just specified.

Lemma 23 (Packed selected-bit addition for grouped rectangles)**.** *Let a valid grouped packed stream of Definition 22 be supplied, and let $A \geq 2$ bound the actual total bit length of its descriptor $\mathcal{D}$, including the header and the entry for R. At the positions $j_i = \rho + iK$, numbered from the least significant bit, one fixed multitape procedure performs*

$$y_{j_i} \longleftarrow y_{j_i} \oplus x_{j_i} \quad (0 \leq i < f)$$

in every group. It fixes the group, row ordinal, spectator coordinates, all other chunk bits, and every record payload; in particular, the chunk z is restored. For $V = MR$, its time is

$$O\Big(V(L^\tau + 1) + MA^3 + M\min\{1, 80(f-1)2^{-K}\}\, A(R + A)\Big). \tag{46}$$

When $A = O(p)$, R exceeds every fixed polynomial in p, $f \le d \le p$, and $K/\log p \to \infty$, this is $O(V(L^\tau + 1))$, uniformly over the group counts, shapes, and payloads whose supplied descriptors satisfy that bound.

Proof. Write $s = t - 3$ for the number of spectator tags. Each group explicitly stores $s + 1$ positive integer codes, so $J(s + 1) \le A$ and $Jt = J(s + 3) \le 3A$. Also $J \le M$, because every group is nonempty. The header includes $\Gamma(M)$ and $\Gamma(R)$, so $\log_2 M + \log_2 R = O(A)$. Every field length, partial suffix product, row count and group record count is at most M. These integers, as well as $V_\nu = M_\nu R$, therefore have $O(A)$ bits. In particular,

$$A_\nu := \lceil \log_2(2V_\nu) \rceil = O(A)$$

with a constant independent of the number and shapes of the groups.

Parse the descriptor in forward group order, keeping the common template on a separate tape and saving the current group's length vector locally. Collapse the row ordinal and the spectator fields before the first chunk into one prefix field. Collapse each spectator interval between the chunks and after the last chunk in the same way; an empty interval has length one. The resulting four positive lengths N_0, N_1, N_2, N_3, together with the current order of the names x, y, z, give exactly the rectangle in Lemma 13, with M_ν records and volume V_ν. Fixing a collapsed coordinate fixes every one of its mixed-radix factors. Hence the rectangular algorithm preserves the row ordinal and all original spectator coordinates, as the grouped claim requires.

For each group, reading the template and length vector, forming the four products, and writing the local descriptor with its fixed number of entries take $O(A^3)$ time: there are $O(A)$ field visits and schoolbook operations, each on $O(A)$-bit integers. The descriptor supplied to the rectangular routine contains only its four collapsed lengths and R, K, f, ρ, in canonical binary, with the finite permutation of the three chunk names. The routine does not receive or rescan the outer group table.

Use separate outer input and output tapes with forward cursors. Copy the V_ν bits of group ν into a local area, apply the complete rectangular algorithm there, append its output to the outer output tape, and clear the visited local area. Keep the outer cursors at their group boundaries during the local call. Reuse the same local and primitive work tapes for every group. The copy, append, positioning and cleanup cost $O(V_\nu)$; one final outer rewind costs $O(V)$. The inner routine restores its local work tapes as part of its stated bound. Thus the tape count is fixed, and no local call scans an earlier or later group.

Put $\delta_{\mathrm{bad}} = \min\{1, 80(f-1)2^{-K}\}$. Within group ν, Lemma 13 performs all the required XORs, restores z for every initial value, and preserves each payload. Its exceptional records are extracted, sorted and reinserted entirely within that group. Their destination ranks therefore require only the local A_ν-bit address bound; no ordering across groups is needed. The local time is

$$O\Big(V_\nu(L^\tau + 1) + M_\nu A_\nu^3 + \delta_{\mathrm{bad}} M_\nu A_\nu (R + A_\nu)\Big).$$

Summing these bounds and the descriptor work, using $\sum_\nu M_\nu = M$, $\sum_\nu V_\nu = V$, $A_\nu = O(A)$ and $J \le M$, gives

$$O\Big(V(L^\tau + 1) + MA^3 + \delta_{\mathrm{bad}} MA(R + A)\Big),$$

which is (46). This sum includes the whole record payload on every local exceptional-sort pass. The rectangular lemma also covers $f = 1$, when its rotations and repair are empty.

After division by $V = MR$, the two additional terms are bounded by A^3/R and $80(f - 1)2^{-K}A(1 + A/R)$. Under the displayed family bounds, the first tends to zero, and the second is $O(p^2 2^{-K}(1 + A/R)) = o(1)$. These bounds depend on the supplied groups only through M, R, A, proving the asserted uniformity. □

Notation and source map

Added explanation Use this page while reading the technical sections.

Symbol or term	Meaning in this guide and the original argument
n	Bit length of each input integer.
b	Radix-digit bit width in the original assembly.
p	Working precision; originally $p = 6\lceil \log_2 n \rceil$.
V	Logical stored bit volume at a recursive node.
W	Number of all roles in a fixed finite network, including scratch.
m	Fixed network dimension and recursive width multiplier.
s	Number of paid smaller calls in the recurrence discussion; in resampling, the same letter instead denotes a source length.
d	Number of transform axes, growing slowly with precision.
K	Packed address-chunk width; scales like $p^{\epsilon c}$.
r, ℓ	Polynomial length and its binary logarithm.
T	Padded transform-box size; $Tp = \Theta(n)$. Do not confuse with the time function $T(n)$ or a triple named T.
τ, σ	Rational upper exponents for the bit and complex recurrences.
λ'	Exponent in the completed simultaneous-layer cost after overhead.
κ	Final saving in the exponent of $\log n$.
Role / row	Fixed circuit wire position / complete contiguous suffix block.
Frame	Invertible representation operator on one role's address-indexed array.
Rank deficit	Difference between a full linear call budget Wm and the paid budget s.
Gaussian dyadic	Complex number with power-of-two denominators in both components.
Conditional	The conclusion depends on retained interfaces whose full independent validation is not supplied by a finite certificate.

Where to find each layer of the original source. The filenames below are preserved in the accompanying package. Their theorem labels are more stable references than page numbers after expansion.

Question	Original source file
What machine is counted?	`02-streams.tex`
What does the finite network do?	`03-motifs.tex`
How does rank price movement?	`04-swap.tex`
How are many butterflies batched?	`05-layers.tex`
How are transforms and ring products done?	`06-transforms.tex`
How do grid sizes and permutations fit?	`07-resampling.tex`
Where are exactness and all costs combined?	`08-assembly.tex`
What are the further consequences?	`09-exact-arithmetic.tex`, `10-transposition.tex`

Primary sources and scope references.

1. OpenAI, *Integer multiplication below* $n \log n$, September 23, 2026. Frozen source in the OpenAI math repository.

2. CrocSwap/integer-mult-bounds, community research and attribution. Local scope reference consulted: `docs/audit.md` at commit `87aa555a455cca84a3e3296ced2440623d39fc4d`.

3. David Harvey and Joris van der Hoeven, *Integer multiplication in time* $O(n \log n)$, Annals of Mathematics 193 (2021). The original paper's bibliography and analytic references are retained in the full edition.

References

[1] R. C. Agarwal and J. W. Cooley. New algorithms for digital convolution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 25(5):392–410, 1977.

[2] Noga Alon. The Shannon capacity of a union. *Combinatorica*, 18(3):301–310, 1998.

[3] Ziv Bar-Yossef, Yitzhak Birk, T. S. Jayram, and Tomer Kol. Index coding with side information. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science*, pages 197–206. IEEE Computer Society, 2006.

[4] L. I. Bluestein. A linear filtering approach to the computation of discrete Fourier transform. *IEEE Transactions on Audio and Electroacoustics*, 18(4):451–455, 1970.

[5] Richard P. Brent. Fast multiple-precision evaluation of elementary functions. *Journal of the ACM*, 23(2):242–251, 1976. Author-hosted copy: https://maths-people.anu.edu.au/~brent/pd/rpb034.pdf.

[6] Harry Buhrman, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. Computing with a full memory: Catalytic space. In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing*, pages 857–866. ACM, 2014.

[7] Anindya De, P. P. Kurur, Chandan Saha, and Ramprasad Saptharishi. Fast integer multiplication using modular arithmetic. *SIAM Journal on Computing*, 42(2):685–699, 2013.

[8] A. Dutt and V. Rokhlin. Fast Fourier transforms for nonequispaced data. *SIAM Journal on Scientific Computing*, 14(6):1368–1393, 1993.

[9] Peter Frankl and Richard M. Wilson. Intersection theorems with geometric consequences. *Combinatorica*, 1(4):357–368, 1981.

[10] Martin Fürer. Faster integer multiplication. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*, pages 57–66. ACM, 2007.

[11] Martin Fürer. Faster integer multiplication. *SIAM Journal on Computing*, 39(3):979–1005, 2009.

[12] D. Yu. Grigor'ev. Additive complexity in directed computations. *Theoretical Computer Science*, 19:39–67, 1982.

[13] Matt Groff. Faster integer multiplication using preprocessing, 2019. Version 2, 11 December 2019.

[14] David Harvey and Joris van der Hoeven. Faster integer multiplication using short lattice vectors. In Renate Scheidler and Jonathan Sorenson, editors, *Proceedings of the Thirteenth Algorithmic Number Theory Symposium*, volume 2 of *The Open Book Series*, pages 293–310. Mathematical Sciences Publishers, 2019. Preprint posted 22 February 2018.

[15] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021. Result numbering refers to the 45-page author-hosted manuscript, https://www.texmacs.org/joris/nlogn/nlogn.pdf, accessed 4 October 2026.

[16] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. In *Proceedings of the 66th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1366–1381. IEEE Computer Society, 2025. Cited theorem numbering follows arXiv version 1, 28 March 2025, https://arxiv.org/abs/2503.22848v1.

[17] David Harvey, Joris van der Hoeven, and Grégoire Lecerf. Even faster integer multiplication. *Journal of Complexity*, 36:1–30, 2016.

[18] A. Karatsuba and Yu. Ofman. Multiplication of many-digital numbers by automatic computers. *Doklady Akademii Nauk SSSR*, 145(2):293–294, 1962. In Russian.

[19] Eyal Lubetzky and Uri Stav. Nonlinear index coding outperforming the linear optimum. *IEEE Transactions on Information Theory*, 55(8):3544–3551, 2009. Result numbering refers to the 16-page author preprint, https://arxiv.org/pdf/0806.1919, accessed 4 October 2026.

[20] H. J. Nussbaumer. Fast polynomial transform algorithms for digital convolution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(2):205–215, 1980.

[21] H. J. Nussbaumer and P. Quandalle. Computation of convolutions and discrete Fourier transforms by polynomial transforms. *IBM Journal of Research and Development*, 22(2):134–144, 1978.

[22] OpenAI. An explicit power saving for the exact discrete Fourier transform. OpenAI Math Release preprint OAI:An-explicit-power-saving-for-the-exact-discrete-Fourier-transform-September-25-2026, 2026.

[23] J. Barkley Rosser and Lowell Schoenfeld. Approximate formulas for some functions of prime numbers. *Illinois Journal of Mathematics*, 6:64–94, 1962.

[24] Arnold Schönhage. Storage modification machines. *SIAM Journal on Computing*, 9(3):490–508, 1980.

[25] Arnold Schönhage and Volker Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, 7:281–292, 1971.

[26] A. L. Toom. The complexity of a scheme of functional elements simulating the multiplication of integers. *Doklady Akademii Nauk SSSR*, 150(3):496–498, 1963. In Russian.